cppad_py

Brad Bell

Sep 13, 2020

CONTENTS

1 Index **1**

2 Table of Contents **3**

3 cppad_py **9**

A C++ Object Library and Python Interface to CppAD 9

Index **265**

CHAPTER

ONE

INDEX

- Link To Index

CHAPTER

TWO

TABLE OF CONTENTS

A C++ Object Library and Python Interface to CppAD

1 Configure and Build the cppad_py Python Module

1.1 Error Messages During Installation
1.2 Get Cppad

2 Numerical Examples

2.1 An AD Compatible Matrix Inverse Routine
2.1.1 Example Computing Derivatives of Matrix Inversion
2.2 One Fourth Order Runge-Kutta ODE Step
2.2.1 Example Computing Derivative A Runge-Kutta Ode Solution
2.3 One Third Order Rosenbrock ODE Step
2.3.1 Example Computing Derivative A Rosenbrock Ode Solution
2.4 Multiple Ode Steps
2.4.1 Example Computing Derivative A Runge-Kutta Ode Solution
2.5 A Helper Class That Defines Functions Needed for Optimization
2.5.1 Example Using optimize_fun_class with Scipy Optimization
2.6 A Susceptible Exposed Infectious Recovered and Death Model
2.6.1 Example Using seris_model
2.7 Example Fitting an SEIRWD Model for Covid-19

3 The Cppad Py Libraries

3.1 The Python Library

3.1.1 Cppad Py AD Functions

3.1.1.1 Declare Independent Variables and Start Recording
3.1.1.1.1 Python: Using Dynamic Parameters: Example and Test
3.1.1.2 Abort Recording
3.1.1.2.1 Python: Abort Recording a_double Operations: Example and Test
3.1.1.3 Stop Current Recording and Store Function Object

3.1.1.3.1 Python: Purpose of a_fun Objects: Example and Test
3.1.1.4 Properties of a Function Object
3.1.1.4.1 Python: d_fun Properties: Example and Test
3.1.1.5 New Dynamic Parameters
3.1.1.6 Jacobian of an AD Function
3.1.1.6.1 Python: Dense Jacobian Using AD: Example and Test
3.1.1.7 Hessian of an AD Function
3.1.1.7.1 Python: Dense Hessian Using AD: Example and Test
3.1.1.8 Forward Mode AD
3.1.1.8.1 Python: Forward Mode AD: Example and Test
3.1.1.9 Reverse Mode AD
3.1.1.9.1 Python: Reverse Mode AD: Example and Test
3.1.1.10 Optimize an AD Function
3.1.1.10.1 Python: Optimize an d_fun: Example and Test
3.1.1.11 Json Representation of AD Computation Graph

3.1.1.11.1 Python to_json: Example and Test
3.1.1.11.2 Python from_json: Example and Test

3.1.2 Python Sparsity Routines

3.1.2.1 Sparsity Patterns
3.1.2.1.1 Python: Sparsity Patterns: Example and Test
3.1.2.2 Sparse Matrices
3.1.2.2.1 Python: Sparsity Patterns: Example and Test
3.1.2.3 Jacobian Sparsity Patterns
3.1.2.3.1 Python: Jacobian Sparsity Patterns: Example and Test
3.1.2.4 Hessian Sparsity Patterns
3.1.2.4.1 Python: Hessian Sparsity Patterns: Example and Test
3.1.2.5 Computing Sparse Jacobians
3.1.2.5.1 Python: Computing Sparse Jacobians: Example and Test
3.1.2.6 Computing Sparse Hessians
3.1.2.6.1 Python: Hessian Sparsity Patterns: Example and Test

3.1.3 Python Utilities

3.1.3.1 Convert a Numpy Array to a cppad_py Vector
3.1.3.2 Convert a cppad_py Vector to a Numpy Array

3.1.4 Steps To Add More Python Functions

3.2 The C++ Library

3.2.1 Cppad Py AD Scalars

3.2.1.1 The a_double Constructor
3.2.1.2 a_double Unary Plus and Minus

3.2.1.2.1 C++: a_double Unary Plus and Minus: Example and Test
3.2.1.2.2 Python: a_double Unary Plus and Minus: Example and Test

3.2.1.3 Properties of an a_double Object

3.2.1.3.1 C++: a_double Properties: Example and Test
3.2.1.3.2 Python: a_double Properties: Example and Test

3.2.1.4 a_double Binary Operators with an AD Result

3.2.1.4.1 C++: a_double Binary Operators With AD Result: Example and Test
3.2.1.4.2 Python: a_double Binary Operators With AD Result: Example and Test

3.2.1.5 a_double Comparison Operators

3.2.1.5.1 C++: a_double Comparison Operators: Example and Test
3.2.1.5.2 Python: a_double Comparison Operators: Example and Test

3.2.1.6 a_double Assignment Operators

3.2.1.6.1 C++: a_double Assignment Operators: Example and Test
3.2.1.6.2 Python: a_double Assignment Operators: Example and Test

3.2.1.7 Unary Functions with AD Result

3.2.1.7.1 C++: a_double Unary Functions with AD Result: Example and Test
3.2.1.7.2 Python: a_double Unary Functions with AD Result: Example and Test

3.2.1.8 AD Conditional Assignment

3.2.1.8.1 C++: a_double Conditional Assignment: Example and Test
3.2.1.8.2 Python: a_double Conditional Assignment: Example and Test

3.2.2 Cppad Py Vectors

3.2.2.1 Cppad Py Vector Constructors
3.2.2.2 Size of a Vector

3.2.2.2.1 C++: Size of Vectors: Example and Test
3.2.2.2.2 Python: Size of Vectors: Example and Test

3.2.2.3 Setting and Getting Vector Elements

3.2.2.3.1 C++: Setting and Getting Vector Elements: Example and Test
3.2.2.3.2 Python: Setting and Getting Vector Elements: Example and Test

3.2.3 Cppad Py AD Functions

3.2.3.1 Declare Independent Variables and Start Recording
3.2.3.1.1 C++: Using Dynamic Parameters: Example and Test
3.2.3.2 Abort Recording
3.2.3.2.1 C++: Abort Recording a_double Operations: Example and Test
3.2.3.3 Stop Current Recording and Store Function Object
3.2.3.4 Properties of a Function Object
3.2.3.4.1 C++: function Properties: Example and Test
3.2.3.5 Change The Dynamic Parameters
3.2.3.6 Jacobian of an AD Function
3.2.3.6.1 C++: Dense Jacobian Using AD: Example and Test
3.2.3.7 Hessian of an AD Function
3.2.3.7.1 C++: Dense Hessian Using AD: Example and Test
3.2.3.8 Forward Mode AD
3.2.3.8.1 C++: Forward Mode AD: Example and Test
3.2.3.9 Reverse Mode AD
3.2.3.9.1 C++: Reverse Mode AD: Example and Test
3.2.3.10 Optimize an AD Function
3.2.3.10.1 C++: Optimize an d_fun: Example and Test
3.2.3.11 Json Representation of AD Computational Graph

3.2.3.11.1 C++: to_json: Example and Test
3.2.3.11.2 C++: from_json: Example and Test

3.2.4 Cppad Py Sparse Calculation

3.2.4.1 Sparsity Patterns
3.2.4.1.1 C++: Sparsity Patterns: Example and Test
3.2.4.2 Sparse Matrices
3.2.4.2.1 C++: Sparsity Patterns: Example and Test
3.2.4.3 Jacobian Sparsity Patterns
3.2.4.3.1 C++: Jacobian Sparsity Patterns: Example and Test
3.2.4.4 Hessian Sparsity Patterns
3.2.4.4.1 C++: Hessian Sparsity Patterns: Example and Test
3.2.4.5 Computing Sparse Jacobians
3.2.4.5.1 C++: Computing Sparse Jacobians: Example and Test
3.2.4.6 Computing Sparse Hessians
3.2.4.6.1 C++: Hessian Sparsity Patterns: Example and Test

3.2.5 C++ Utilities

3.2.5.1 Convert an a_double Vector to a CppAD::AD<double> Vector
3.2.5.2 Convert a CppAD::AD<double> Vector to an a_double Vector
3.2.5.3 Exception Handling

3.2.5.3.1 C++: Cppad Py Exception Handling: Example and Test

3.2.5.3.2 Python: Cppad Py Exception Handling: Example and Test

3.2.6 Steps To Add More C++ Functions

4 Extract Sphinx RST

4.1 Begin and End Commands
4.2 Children Commands
4.2.1 No Parent Example
4.2.1.1 No Parent Result

4.2.1.1.1 Indent and Children Example
4.2.1.1.2 Indent and Children Result
4.2.1.1.2.1 Indent Example
4.2.1.1.2.1.1 Indent Result

4.3 Spell Command
4.3.1 Spell Example
4.3.1.1 Spell Result
4.4 Suspend and Resume Commands
4.4.1 Suspend Example
4.4.1.1 Suspend Result
4.5 Code Command
4.5.1 Code Example
4.5.1.1 Code Result
4.6 File Command
4.6.1 File Example
4.6.1.1 File Result
4.7 Comment Character Command
4.7.1 Comment Character Example
4.7.1.1 Comment Character Result
4.8 Heading Example
4.8.1 Heading Result
4.9 Example xsrst Configuration Files

4.9.1 Example conf.py
4.9.2 Example index.rst
4.9.3 Example Spelling File
4.9.4 Example Keyword File

5 CppAD Py Changes During 2020
5.1 Cppad Py Changes During 2018

CHAPTER

THREE

CPPAD_PY

A C++ Object Library and Python Interface to CppAD

- *Version 2020.9.13*
- *Git Repository*
- *Purpose*
- *Getting Started*
- *Numerical Examples*
- *C++ Function Speed*
- *License*
- *Children*

Version 2020.9.13

Git Repository

https://github.com/bradbell/cppad_py

Purpose

1. Provide a connection from Python to the Algorithmic Differentiation (AD) package Cppad; see *py_lib*.
2. Provide an AD object library; see *cpp_lib*.
3. Provide a concrete example of how cppad_swig can be used to connect any scripting language to CppAD.

Getting Started

The following is a good place to get started using cppad_py: *python*.

Numerical Examples

The following is a link to some numerical examples: *numeric_xam*.

C++ Function Speed

One can use Cppad Py to get faster function evaluation in scripting Python, when the sequence of floating point operations does not depend on the independent variables. Once an *py_fun* is recorded, zero order *forward_mode* can be used to effectively evaluate the function in C++ instead of Python.

License

This program is distributed under the terms of the GNU General Public License version 3.0 or later see gpl-3.0.txt.

Children

setup_py

1 Configure and Build the cppad_py Python Module

- *Syntax*
- *External Requirements*
- *Install Using Pip*
- *Install Errors*
- ***Download***
  - *Top Source Directory*
- *Configure*
- *Get cppad*
- ***Test***
  - *Build cppad_py*
  - *python*
  - *c++*
  - *import*
- *Install*
- *Python Path*

Syntax

```
python setup.py build_ext
```

External Requirements

1. python version 3.
2. bash
3. c++.
4. cmake
5. git.
6. swig: issue 3.
7. numpy.

Install Using Pip

There is a preliminary version of cppad_py available using `pip` . To install for you entire system use:

```
pip install -i https://test.pypi.org/simple/ cppad_py
```

install_error

1.1 Error Messages During Installation

- ***Solved***
  - *swig*
  - *Permissions*
- *CppAD Library Missing*
- *Fortify Source*
- ***Unsolved***
  - *Travis*

Solved

swig

If the error message below occurs, try installing http://www.swig.org/ on you system:

```
FileNotFoundError: [Errno 2] No such file or directory: 'swig'
```

The following message as also been seen:

```
Error: Unable to find 'python.swg'
```

In this case try installing `swig-python` .

Permissions

If you try to do a system wide install, and do not have permission, you will get the following message:

```
ERROR: Could not install packages due to... Permission denied:
```

Try install in your local user space. For example, if you are using pip,

```
pip install -i https://test.pypi.org/simple/ cppad_py --user --verbose
```

CppAD Library Missing

If the error message below occurs, it is because the CppAD library is not in your `LD_LIBRARY_PATH` :

```
ImportError: libcppad_lib.so... can not open shared object file
```

If you run the `setup.py` script directory, or if you included the `--verbose` option in your `pip` command, there should be text at the end that tells you how to modify your `LD_LIBRARY_PATH` . If you have a Mac, you may instead need to modify `DYLD_LIBRARY_PATH` .

Fortify Source

If you set *build_type* to `debug` , you may get the following warning during the build:

```
#warning _FORTIFY_SOURCE requires compiling with optimization
```

This is a problem with the python setuptools, one can un-define a macro, but it does not remove a original definition.

Unsolved

If you know how to fix and of he errors below, please open an issue and put your solution there.

Travis

The following message occurs on a Travis system of unknown configuration; i.e., it has not been reproduced outside of Travis.

```
return alpha * p * a_exp( - z * z ) / numpy.sqrt(numpy.pi)[0m
   TypeError:  unsupported operand type(s) for /:  'a_double' and 'float'
```

Note the same code works on other systems. The `.travis.yml` file is included below:

```
language:  python
python:
   - "3.7"
install:
   - sudo apt-get install swig
   - python setup.py install
   - pip install -i https://test.pypi.org/simple/ cppad_py
script:
   - pytest tests
   - make examples
```

---

xsrst input file: `install_error.xsrst`

get_cppad_sh

1.2 Get Cppad

- *Syntax*
- *Top Source Directory*
- ***Settings***
  - *cppad_prefix*
  - *extra_cxx_flags*
  - *build_type*
  - *test_cppad*
- *Caching*

Syntax

```
bin/get_cppad.sh
```

Top Source Directory

This program must be run from the *top_source_directory*. Each time it is run it removes the old `build` directory and starts over.

Settings

If you change any of these settings, you must re-run `get_cppad.sh` .

cppad_prefix

This prefix is used to install CppAD may be a local director; e.g., `build/prefix` or an absolute path; e.g., `/usr/local` , it may include the shell variable `$HOME` but no other variables:

```
cppad_prefix="$HOME/prefix/cppad"
```

extra_cxx_flags

Extra compiler false used when compiling c++ code not including the debugging and optimization flags. The ones below are example flags are used by g++:

```
extra_cxx_flags='-Wall -pedantic-errors -Wno-unused-result -std=c++11'
```

build_type

This must be must `debug` or `release` . The debug version has more error messaging while the release version runs faster.

```
build_type='release'
```

If you used the `debug` build type you may get the following warning from the compiler (because the optimization is totally turned off):

```
warning _FORTIFY_SOURCE requires compiling with optimization
```

test_cppad

This must be must `true` or `false` . Cppad has a huge test suite and this can take a significant amount of time, but it may be useful if you have problems.

```
test_cppad='false'
```

Caching

This procedure cashes previous builds so that when you re-run this script it does not re-do all the work. If you have trouble, try deleting the directory

`build/cppad` - *yyyymmdd* `.git`

and re-running this script.

---

xsrst input file: `bin/get_cppad.sh`

Install Errors

If you get an error message during the install procedure above, or the one below, see *install_error*. This will only install the release version. Installing a debug version is discussed below in the instructions for downloading and building from the source code.

Download

Use the following command to download the current version of cppad_py:

```
git clone https://github.com/bradbell/cppad_py.git top_srcdir
```

Top Source Directory

The directory you choose for *top_srcdir* is referred to as your top source directory.

Configure

Before running `setup.py` or `bin/get_cppad.sh` , you should check and possibly change the *settings* in `bin/get_cppad.sh` .

Get cppad

The next step is to get a copy of cppad using *get_cppad_sh*.

Test

These steps are optional if you already know that cppad_py works on your system.

Build cppad_py

Build the Python cppad_py module using the command:

```
python setup.py build_ext
```

python

The next step is to test the cppad_py on your system by executing the following commands starting in *top_srcdir* :

```
cd example/python
python check_all.py
```

C++

After `setup.py` has run, you can also test the cppad_py c++ interface *cpp_lib* on your system by executing the following commands starting in *top_srcdir* :

```
cd build
make check_lib_cplusplus
```

import

If you are in the *top_srcdir* directory, you should be able to import cppad_py using the following commands:

```
python
import cppad_py
quit ()
```

We need to install cppad_py so you can import it from any directory.

Install

Use the following command to build and install cppad_py:

`python setup.py install --prefix` = *prefix*

This will install `cppad_py` in the directory

prefix / *lib* /`python` *major* . *minor* /`site_packages/cppad_py`

where *lib* is `lib` or `lib64` , *major* (*minor*) is the major (minor) version of *python* .

Python Path

Check that the directory

prefix / *lib* /`python` *major* . *minor* /`site_packages`

is in your python path. Once it is, you should be able to execute the following commands:

```
python
```

```
import sys
print(sys.path)
quit()
```

---

xsrst input file: `setup.py`

numeric_xam

2 Numerical Examples

- *Children*

Children

numeric_simple_inv

2.1 An AD Compatible Matrix Inverse Routine

- *Syntax*
- *Purpose*
- *A*
- *Ainv*
- *Example*
- *Source Code*

Syntax

Ainv = `simple_inv` (*A*)

Purpose

This routine can be used with `ad_double`

A

This must be an invertible square matrix (no singular detection is done). The type of its elements can be `float` or `a_double` .

Ainv

This is the matrix inverse of A .

numeric_simple_inv_xam_py

2.1.1 Example Computing Derivatives of Matrix Inversion

- *Problem*
- *Source Code*

Problem

We define

$$A(x) = \begin{pmatrix} x_0 & x_1 \\ x_2 & x_3 \end{pmatrix}$$

It follows that

$$A^{-1}(x) = \frac{1}{x_3 * x_0 - x_1 * x_2} \begin{pmatrix} x_3 & -x_1 \\ -x_2 & x_0 \end{pmatrix}$$

We define

$$f(x) = (x_3 * x_0 - x_1 * x_2)[A^{-1}_{0,0}(x), A^{-1}_{0,1}(x), A^{-1}_{1,0}(x), A^{-1}_{1,1}(x)] = [x_3, -x_1, -x_2, x_0]$$

The following example below check the derivative of $f(x)$

Source Code

```
import numpy
import cppad_py
from simple_inv import simple_inv
def simple_inv_xam() :
    ok = True
    x  = numpy.array( [1.0, 2.0, 3.0, 4.0] )
    ax = cppad_py.independent(x)
    #
    # create the matrix [ [x_0, x_1], [x_2, x_3] ]
    aA = numpy.reshape(ax, (2,2), order='C' )
    #
    # compute inverse of A
    aAinv = simple_inv(aA)
    #
    # create f(x)
    ay = numpy.reshape(aAinv, 4, order='C' )
    ay = (ax[3] * ax[0] - ax[1] * ax[2]) * ay
    f  = cppad_py.d_fun(ax, ay)
    #
    # evaluate the derivative f'(x)
    J      = f.jacobian(x)
```

(continues on next page)

(continued from previous page)

```
    #
    # Check the derivative
    check      = numpy.zeros( (4, 4) )
    check[0,3] =  1.0
    check[1,1] = -1.0
    check[2,2] = -1.0
    check[3,0] =  1.0
    #
    eps99 = 99.0 * numpy.finfo(float).eps
    ok   &= numpy.all( numpy.fabs(J - check) < eps99 )
    return ok
```

---

xsrst input file: `example/python/numeric/simple_inv_xam.py`

Example

numeric_simple_inv_xam_py

Source Code

When viewing the source code below it is important to know that optimizes out multiplication by the constant one while recording a function. It also optimizes out both addition and multiplication by the constant zero.

```
import numpy
import copy
from cppad_py import a_double
def simple_inv(A) :
    nr, nc = A.shape
    has_a_double = False
    for e in A.flatten() :
        has_a_double = has_a_double or type(e) == a_double
    assert nr == nc
    #
    if has_a_double :
        e_type = a_double
    else :
        e_type = float
    #
    # initialize Tablue = [A | I]
    Tablue = numpy.empty( (nr, 2 * nr), dtype=e_type )
    Tablue[:,0:nr] = A
    for i in range(nr) :
        for j in range(nr) :
            Tablue[i, j]    = e_type( A[i, j] )
            Tablue[i,nr+j] = float(i == j)
    # ---------------------------------------------------------------------------
    # Use row reduction to get convert A to an upper traingular matrix
    # with ones along the diagonal
    # ---------------------------------------------------------------------------
    # for each column (except the last) of the matrix
    for j in range(nr - 1) :
```

(continues on next page)

(continued from previous page)

```
        # next row is one with maximum absolute element in column j
        if has_a_double :
            i_max = numpy.argmax( numpy.fabs( Tablue[:,j] ) )
        else :
            i_max = numpy.argmax( numpy.abs( Tablue[:,j] ) )
        #
        # swap row j and row i_max
        if i_max != j :
            tmp               = copy.copy( Tablue[j,:] )
            Tablue[j,:]       = Tablue[i_max,:]
            Tablue[i_max,:]   = tmp
        #
        # divide row j by Tablue[j,j]
        Tablue[j,:] = Tablue[j,:] / Tablue[j,j]
        Tablue[j,j] = e_type(1.0)
        #
        # zero out column j in rows after j
        for k in range(nr - j - 1) :
            i = j + k + 1
            Tablue[i,:] = Tablue[i,:] - Tablue[i,j] * Tablue[j,:]
            Tablue[i,j]     = e_type( 0.0 )
    # divide the last row of Tablue by the last pivot element
    Tablue[nr-1,:]      = Tablue[nr-1,:] / Tablue[nr-1,nr-1]
    Tablue[nr-1, nr-1] = e_type( 1.0 )
    # ------------------------------------------------------------------------
    # Use row reduction to get convert upper traingular matrix to the identity
    # ------------------------------------------------------------------------
    for jm in reversed(range(nr - 1) ) :
        j = jm + 1
        # zero out entries in column j and row less than j
        for i in range(j) :
            Tablue[i,:] = Tablue[i,:] - Tablue[i,j] * Tablue[j,:]
            Tablue[i,j] = e_type( 0.0 )
    # ------------------------------------------------------------------------
    Ainv = Tablue[:, nr:]
    return Ainv
```

xsrst input file: `example/python/numeric/simple_inv.py`

numeric_runge4_step

2.2 One Fourth Order Runge-Kutta ODE Step

- *Syntax*
- *Purpose*
- *fun*
- *ti*
- *yi*
- *h*
- *yf*

- *Example*
- *Source Code*

Syntax

yf = `runge4_step` (*fun* , *ti* , *yi* , *h*)

Purpose

The routine can be used with `ad_double` to solve the initial value ODE

$$y^{(1)}(t) = f(t, y)$$

fun

This is a function that evaluates the ordinary differential equation using the syntax

yp = *fun* . `f` *t* , *y*)

where *t* # is the current time, *y* is the current value of $y(t)$, and *yp* is the current derivative $y^{(1)}(t)$. The type of the elements of *t* and *y* can be `float` or `ad_double` .

ti

This is the initial time for the Runge-Kutta step. It can have type `float` or `a_double` .

yi

This is the numpy vector containing the value of $y(t)$ at the initial time. The type of its elements can be `float` or `a_double` .

h

This is the step size in time; i.e., the time at the end of the step minus the initial time. It can have type `float` or `a_double` .

yf

This is the approximate solution for $y(t)$ at the final time as a numpy vector. This solution is 4-th order accurate in time t; e.g., if $y(t)$ is a polynomial in t of order four or lower, the solution has no truncation error, only round off error.

numeric_runge4_step_xam_py

2.2.1 Example Computing Derivative A Runge-Kutta Ode Solution

- *ODE*
- *Solution*
- *Derivative of Solution*
- *Source Code*

ODE

$$\partial_t y_i(t,x) = f(t,y,x) \left\{ \begin{array}{rl} x_0 & \text{if } i = 0 \\ x_i y_{i-1}(t) & \text{otherwise} \end{array} \right.$$

with the initial condition $y(0) = 0$

Solution

This is a special case for which we know the solution

$$y_i(t,x) = \left\{ \begin{array}{rl} t x_0 & \text{if } i = 0 \\ (t^i/(i+1)!) \prod_{j=0}^{i} x_j & \text{otherwise} \end{array} \right.$$

Derivative of Solution

For this special case, the partial derivative of the solution with respect to the j-th component of the vector x is

$$\partial_{x(j)} y_i(t,x) = \left\{ \begin{array}{rl} y_i(t,x)/x_j & \text{if } j \leq i \\ 0 & \text{otherwise} \end{array} \right.$$

Source Code

```
import numpy
from  scipy.special import factorial
import cppad_py
from runge4_step import runge4_step
#
```

(continues on next page)

(continued from previous page)

```
class fun_class :
    #
    def __init__(self, x) :
        self.x = x
    #
    def f(self, t, y) :
        y_shift = numpy.concatenate( ( [1.0] , y[0:-1] ) )
        return self.x * y_shift
#
def runge4_step_xam() :
    ok    = True
    nx    = 4
    eps99 = 99.0 * numpy.finfo(float).eps
    #
    # independent variables for this g(x) = y(1, x)
    x  = numpy.array( nx * [ 1.0 ] )
    ax = cppad_py.independent(x)
    #
    # function to pass to runge4_step
    fun = fun_class(ax)
    #
    # initiali value for the ODE
    ay_start =  numpy.array( nx * [ cppad_py.a_double(0.0) ] )
    t_start  = 0.0
    t_step   = 0.75
    #
    # take one step
    ay = runge4_step(fun, t_start, ay_start, t_step)
    #
    # g(x) = y(t_step, x)
    g = cppad_py.d_fun(ax, ay)
    #
    # Check g_i (x) = prod_{j=0}^i x[j] t^(i+1) / (i+1) !
    # 4th order method should be very accutate for functions
    # or order 4 or less in t.
    x  = numpy.arange(0, nx) + 1.0
    gx = g.forward(0, x)
    prod = 1.0
    for i in range(nx) :
        prod      = prod * x[i]
        check     = prod * numpy.power(t_step, i+1) / factorial(i+1)
        rel_error = gx[i] / check - 1.0
        ok       &= abs(rel_error) < eps99
    #
    # Check derivative of g_i (x) w.r.t x_j
    # is zero for i < j and  g_i(x) / x_j otherwise
    J = g.jacobian(x)
    for j in range(nx) :
        for i in range(nx) :
            if i < j :
                ok &= J[i,j] == 0.0
            else :
                check     = gx[i] / x[j]
                rel_error = J[i,j] / check - 1.0
                ok       &= abs(rel_error) < eps99

    return ok
```

---

xsrst input file: `example/python/numeric/runge4_step_xam.py`

Example

numeric_runge4_step_xam_py

Source Code

```
def runge4_step(fun, ti, yi, h) :
    k1      = h * fun.f(ti,           yi)
    k2      = h * fun.f(ti + h / 2.0, yi + k1 / 2.0)
    k3      = h * fun.f(ti + h / 2.0, yi + k2 / 2.0)
    k4      = h * fun.f(ti + h,       yi + k3 )
    yf      = yi + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
    return yf
```

---

xsrst input file: `example/python/numeric/runge4_step.py`

numeric_rosen3_step

2.3 One Third Order Rosenbrock ODE Step

- *Syntax*
- *Purpose*
- *Reference*
- ***fun***
  - *t*
  - *y*
  - *fun.f*
  - *fun.f_t*
  - *fun.f_y*
- *ti*
- *yi*
- *h*
- *yf*
- *ok*
- *Example*
- ***Source Code***
  - *rosen3_step*
  - *check_rosen3_step*

Syntax

yf = `rosen3_step` (*fun* , *ti* , *yi* , *h*)
ok = `check_rosen3_step` (*fun* , *ti* , *yi* , *h*)

Purpose

The routine `rosen3_step` can be used with `ad_double` to solve an initial value ODE

$$y^{(1)}(t) = f(t, y)$$

Reference

The formulas in this method are taken from page 100 of the following reference (except that 98/108 was correct to 97/108): Shampine, L.F., *Implementation of Rosenbrock Methods* , ACM Transactions on Mathematical Software, Vol. 8, No. 2, June 1982.

fun

This is a function that evaluates the ordinary differential equation, and its partial derivatives,

t

The argument *t* below is the current time. It can be a `float` or `a_double` .

y

The argument *y* below is the current value of $y(t)$. The type of the elements of *y* can be `float` or `ad_double` .

fun.f

The syntax `yp` = `fun` . *f* `t` , `y`) returns *yp* , the current function value $f(t, y)$.

fun.f_t

The syntax `yp_t` = `fun` . *f_t* (`t` , `y`) returns *yp_t* , the current partial value $\partial_t f(t, y)$.

fun.f_y

The syntax `yp_y = fun . f_y ( t , y )` returns *yp_y* , the current partial value $\partial_y f(t, y)$.

ti

This is the initial time for the Rosenbrock step. It can have type `float` or `a_double` . (For `check_rosen3_step` only `float` is allowed.)

yi

This is the numpy vector containing the value of $y(t)$ at the initial time. The type of its elements can be `float` or `a_double` . (For `check_rosen3_step` only `float` is allowed.)

h

This is the step size in time; i.e., the time at the end of the step minus the initial time. It can have type `float` or `a_double` . (This is not used by `check_rosen3_step` .)

yf

This is the approximate solution for $y(t)$ at the final time as a numpy vector. This solution is 3-th order accurate in time t; e.g., if $y(t)$ is a polynomial in t of order three or lower, the solution has no truncation error, only round off error.

ok

This is true if the function *fun* . `f` *t* , *y*) and the partials *fun* . `f_t` (*t* , *y*) , *fun* . `f_y` (*t* , *y*) agree. Otherwise AD has detected an error in these functions.

numeric_rosen3_step_xam_py

2.3.1 Example Computing Derivative A Rosenbrock Ode Solution

- *y(t, x)*
- *First ODE*
- *Second ODE*
- *Derivative of Solution*
- *Source Code*

y(t, x)

The two initial value problems below have the following solution:

$$y_i(t,x) = (t^{i+1}/(i+1)!) \prod_{j=0}^{i} x_j$$

First ODE

$$f(t,y,x) = \left\{ \begin{array}{rl} x_0 & \text{if } i = 0 \\ x_i y_{i-1}(t) & \text{otherwise} \end{array} \right.$$

with the initial condition $y(0) = 0$

Second ODE

$$f(t,y,x) = (t^i/i!) \prod_{j=0}^{i} x_j$$

with the initial condition $y(0) = 0$

Derivative of Solution

For this special case, the partial derivative of the solution with respect to the j-th component of the vector x is

$$\partial_{x(j)} y_i(t,x) = \left\{ \begin{array}{rl} y_i(t,x)/x_j & \text{if } j \leq i \\ 0 & \text{otherwise} \end{array} \right.$$

Source Code

```
import numpy
from  scipy.special import factorial
import cppad_py
from rosen3_step import rosen3_step
from rosen3_step import check_rosen3_step
# ----------------------------------------------------------------------------
class first_fun_class :
    def __init__(self, x) :
        self.x = x

    # fun.f
    def f(self, t, y) :
        y_shift = numpy.concatenate( ( [1.0] , y[0:-1] ) )
        return self.x * y_shift
```

(continues on next page)

(continued from previous page)

```
    #
    # fun.f_t
    def f_t(self, t, y) :
        ny = y.size
        return numpy.zeros( ny, dtype=type(y[0]) )
    #
    # fun.f_y
    def f_y(self, t, y) :
        ny = y.size
        J  = numpy.zeros( (ny, ny), dtype=type(y[0]) )
        for i in range(ny - 1) :
            J[i+1, i] = self.x[i+1]
        return J
# ----------------------------------------------------------------------------
class second_fun_class :
    def __init__(self, x) :
        self.x  = x
    #
    # fun.f
    def f(self, t, y) :
        ny        = y.size
        type_y    = type(y[0])
        result    = numpy.empty(ny, dtype=type(y[0]) )
        result[0] = type_y( self.x[0] )
        for i in range(ny - 1) :
            result[i+1] = result[i] * self.x[i + 1] * t / float(i+1)
        return result
    #
    # fun.f_t
    def f_t(self, t, y) :
        ny = y.size
        type_y    = type(y[0])
        result    = numpy.empty(ny, dtype=type(y[0]) )
        result[0] = type_y( 0.0 )
        prod      = type_y( self.x[0] )
        for i in range(ny - 1) :
            prod        = prod * self.x[i+1]
            result[i+1] = prod
            prod        = prod * t / (i + 1)
        return result
    #
    # fun.f_y
    def f_y(self, t, y) :
        ny = y.size
        return numpy.zeros( ny, dtype=type(y[0]) )
# ----------------------------------------------------------------------------
# Check derivative claculations in first_fun_class and second_fun_class
def test_derivative() :
    ok       = True
    nx       = 3
    eps99    = 99.0 * numpy.finfo(float).eps
    t_start  = 0.0
    t_step   = 0.75
    x        = numpy.array( nx * [ 1.0 ] )
    y_start  = numpy.array( nx * [ 0.0 ] )
    #
    fun_1    = first_fun_class(x)
```

(continues on next page)

(continued from previous page)

```
    ok       = ok and check_rosen3_step(fun_1, t_start, y_start, t_step)
    #
    fun_2    = second_fun_class(x)
    ok       = ok and check_rosen3_step(fun_1, t_start, y_start, t_step)
    return ok

def test_case(case) :
    ok    = True
    nx    = 3
    eps99 = 99.0 * numpy.finfo(float).eps
    #
    # independent variables for this g(x) = y(1, x)
    x  = numpy.array( nx * [ 1.0 ] )
    ax = cppad_py.independent(x)
    #
    if case == 1 :
        fun = first_fun_class(ax)
    elif case == 2 :
        fun = second_fun_class(ax)
    else :
        assert(False)
    #
    # initiali value for the ODE
    ay_start =  numpy.array( nx * [ cppad_py.a_double(0.0) ] )
    t_start  = 0.0
    t_step   = 0.75
    #
    # take one step
    ay = rosen3_step(fun, t_start, ay_start, t_step)
    #
    # g(x) = y(t_step, x)
    g = cppad_py.d_fun(ax, ay)
    #
    # Check g_i (x) = prod_{j=0}^i x[j] t^(i+1) / (i+1) !
    # 3th order method should be very accutate for functions
    # or order 3 or less in t.
    x  = numpy.arange(0, nx) + 1.0
    gx = g.forward(0, x)
    prod = 1.0
    for i in range(nx) :
        prod      = prod * x[i]
        check     = prod * numpy.power(t_step, i+1) / factorial(i+1)
        rel_error = gx[i] / check - 1.0
        ok       &= abs(rel_error) < eps99
    #
    # Check derivative of g_i (x) w.r.t x_j
    # is zero for i < j and  g_i(x) / x_j otherwise
    J = g.jacobian(x)
    for j in range(nx) :
        for i in range(nx) :
            if i < j :
                ok &= J[i,j] == 0.0
            else :
                check     = gx[i] / x[j]
                rel_error = J[i,j] / check - 1.0
                ok       &= abs(rel_error) < eps99
```

(continues on next page)

(continued from previous page)

```
    return ok
# ------------------------------------------------------------------------------
def rosen3_step_xam() :
    ok = True
    ok = ok and test_derivative()
    ok = ok and test_case(1)
    ok = ok and test_case(2)
    return ok
```

---

xsrst input file: `example/python/numeric/rosen3_step_xam.py`

Example

numeric_rosen3_step_xam_py

Source Code

rosen3_step

```
import numpy
from simple_inv import simple_inv
import cppad_py
def rosen3_step(fun, ti, yi, h) :
    #
    r1_2     = 1.0   / 2.0
    r3_2     = 3.0   / 2.0
    r3_5     = 3.0   / 5.0
    r24_25   = 24.0  / 25.0
    r3_25    = 3.0   / 25.0
    r121_50  = 121.0 / 50.0
    r186_25  = 186.0 / 25.0
    r6_5     = 6.0   / 5.0
    r97_108  = 97.0  / 108.0
    r11_72   = 11.0  / 72.0
    r25_216  = 25.0  / 216.0
    #
    # Einv
    ny   = yi.size
    I    = numpy.identity(ny, dtype=float)
    E    = I - r1_2 * h * fun.f_y(ti, yi)
    Einv = simple_inv(E)
    #
    # f_t
    f_t  = fun.f_t(ti, yi)
    #
    # k1
    k1   = fun.f(ti, yi)
    k1  += r1_2 * h * f_t
    k1   = numpy.matmul(Einv , k1)
    #
```

(continues on next page)

(continued from previous page)

```
    # k2
    k2   = fun.f(ti + h, yi + h * k1)
    k2  -= r3_2 * h * f_t
    k2  -= 4.0 * k1
    k2   = numpy.matmul(Einv , k2)
    #
    # k3
    t    = ti + r3_5 * h
    y    = yi + r24_25 * h * k1 + r3_25 * h * k2
    k3   = fun.f(t, y)
    k3  += r121_50 * h * f_t
    k3  += r186_25 * k1 + r6_5 * k2
    k3   = numpy.matmul(Einv , k3)
    #
    yf   = yi + h * ( r97_108 * k1 + r11_72 * k2 + r25_216 * k3)
    #
    return yf
```

check_rosen3_step

```
def check_rosen3_step(fun, ti, yi, h) :
    ok    = True
    eps99 = 99.0 * numpy.finfo(float).eps
    #
    ny    = yi.size
    both  = numpy.concatenate( (yi, [ti]) )
    aboth = cppad_py.independent(both)
    az    = fun.f(aboth[ny], aboth[0:ny] )
    fun_d = cppad_py.d_fun(aboth, az)
    #
    J     = fun_d.jacobian(both)
    #
    # check fun.f_t(t, y)
    fun_t   = fun.f_t(ti, yi)
    printed = False
    for i in range(ny) :
        if J[i, ny] == 0.0 :
            ok = ok and fun_t[i] == 0.0
        else :
            rel_error = fun_t[i] / J[i,ny] - 1.0
            if abs(rel_error) > eps99 :
                ok = False
                if not printed :
                    print('check_rosen3_step: fun.f_t check failed')
                    printed = True
                print('fun_t[', i, '] = ', fun_t[i], ', check = ', J[i,ny])
    #
    # check fun.f_y(t, y)
    fun_y = fun.f_y(ti, yi)
    printed = False
    for i in range(ny) :
        for j in range(ny) :
            if J[i, j] == 0.0 :
                ok_ij =fun_y[i, j] == 0.0
```

(continues on next page)

(continued from previous page)

```
            else :
                rel_error = fun_y[i, j] / J[i, j] - 1.0
                ok_ij = abs( rel_error ) < eps99
            if (not ok_ij) and (not printed) :
                print('check_rosen3_step: fun.f_y check failed')
                printed = True
            if not ok_ij :
                msg  = 'fun_y[' + str(i) + ',' + str(j) + '] ='
                msg += str( fun_y[i, j] )
                msg += ', check = ' + str( J[i, j] )
                print(msg)
    #
    return ok
```

xsrst input file: `example/python/numeric/rosen3_step.py`

numeric_ode_multi_step

2.4 Multiple Ode Steps

- *Syntax*
- *Purpose*
- ***one_step***
  - *t0*
  - *y0*
  - *t_step*
  - *y1*
- ***fun***
  - *fun.set_t_all_index(index)*
  - *fun.f(t, y)*
  - *one_step*
- *t_all*
- *y_init*
- *y_all*
- *Example*
- *Source Code*

Syntax

y_all = `ode_multi_step` (*one_step* , *f* , *t_all* , *y_init*)

Purpose

The routine can be used with `ad_double` to solve an initial value ODE

$$y^{(1)}(t) = f(t, y)$$

one_step

This routine executes one step of an ODE approximation method with the following syntax:

y1 = *one_step* (*f* , *t0* , *y0* , *t_step*)

The elements of *y0* and the scalars above can be `float` or `a_double` .

t0

is the initial time value for the step.

y0

is the initial value of $y(t)$ for the step.

t_step

is the is the size of the step (in time).

y1

is the approximation for the ODE solution at time *t0* + *t_step* .

fun

fun.set_t_all_index(index)

Often, we use interpolation with knots to define $f(t, y)$. It is one of the subtitle issues of AD that even though values are the same, derivatives might not be the same; e.g., piecewise linear interpolation. We must break the integration of the ODE at each of the knot so we can use a method that assumes $f(t, y)$ is smooth. Also so that AD can be used to compute derivatives of our solutions. The function `set_t_all_index` informs *fun* that we are currently integrating the time interval

[*t_all* [*index*] , *t_all* [*index* +1]]

so that it know which smooth function to represent even if t is at a knot and it matters if it is the interval to the left or right of the knot. The function `set_t_all_index` is called at the start of each integration interval and before any of the other *fun* member functions.

fun.f(t, y)

This call evaluates the function that defines the ODE

$$y^{(1)}(t) = f[t, y(t)]$$

one_step

The routine *one_step* may put extra requirements on *fun* .

t_all

This is a numpy vector of time values at which the solution is calculated. The type of its elements can be `float` or `ad_double` . It must be either monotone increasing or decreasing.

y_init

This is the value of $y(t)$ at the initial time *t_all* [0] as a numpy vector. The type of its elements can be `float` or `ad_double` .

y_all

This is the approximate solution for $y(t)$ at all of the times specified by *t_all* as a numpy array. The value *y_all* [*i* , *j*] is the value of the j-th component of $y(t)$ at time *t_all* [*i*] .

numeric_ode_multi_step_xam_py

2.4.1 Example Computing Derivative A Runge-Kutta Ode Solution

- *ODE*
- *Solution*
- *Derivative of Solution*
- *Source Code*

ODE

$$\partial_t y_i(t,x) = f(t,y,x) \left\{ \begin{array}{rl} x_0 & \text{if } i = 0 \\ x_i y_{i-1}(t) & \text{otherwise} \end{array} \right.$$

with the initial condition $y(0) = 0$

Solution

This is a special case for which we know the solution

$$y_i(t,x) = \left\{ \begin{array}{rl} t x_0 & \text{if } i = 0 \\ (t^i/(i+1)!) \prod_{j=0}^{i} x_j & \text{otherwise} \end{array} \right.$$

Derivative of Solution

For this special case, the partial derivative of the solution with respect to the j-th component of the vector x is

$$\partial_{x(j)} y_i(t,x) = \left\{ \begin{array}{rl} y_i(t,x)/x_j & \text{if } j \leq i \\ 0 & \text{otherwise} \end{array} \right.$$

Source Code

```
import numpy
import scipy.special
import cppad_py
from ode_multi_step import ode_multi_step
from runge4_step import runge4_step
#
class fun_class :
    #
    def __init__(self, x) :
        self.x     = x
        self.index = None
    #
    def set_t_all_index(self, index) :
        self.index = index
    #
    def f(self, t, y) :
        assert self.index != None
        # This ode does not depend on the the time index
        #
        y_shift = numpy.concatenate( ( [1.0] , y[0:-1] ) )
        return self.x * y_shift
#
def ode_multi_step_xam() :
    ok    = True
    nx    = 4
```

(continues on next page)

(continued from previous page)

```
    eps99 = 99.0 * numpy.finfo(float).eps
    #
    # independent variables for this g(x) = y(1, x)
    x  = numpy.array( nx * [ 1.0 ] )
    ax = cppad_py.independent(x)
    #
    # function to pass to runge4_step
    fun = fun_class(ax)
    #
    # routine we are usning for each step
    one_step = runge4_step
    #
    # initiali value for the ODE
    n_step  = 10
    ay_init =  numpy.array( nx * [ cppad_py.a_double(0.0) ] )
    t_final = 0.75
    t_all   =  [ t_final * i / (n_step - 1) for i in range(n_step) ]
    t_all   =  numpy.array( t_all )
    #
    # take multiple steps
    ay = ode_multi_step(one_step, fun, t_all, ay_init)
    ay_final = ay[n_step - 1,:]
    #
    # g(x) = y(t_final, x)
    g = cppad_py.d_fun(ax, ay_final)
    #
    # Check g_i (x) = prod_{j=0}^i x[j] t^(i+1) / (i+1) !
    # 4th order method should be very accutate for functions
    # or order 4 or less in t.
    x  = numpy.arange(0, nx) + 1.0
    gx = g.forward(0, x)
    prod = 1.0
    for i in range(nx) :
        prod      = prod * x[i]
        power     = numpy.power(t_final, i+1)
        factorial = scipy.special.factorial(i+1)
        check     = prod * power / factorial
        rel_error = gx[i] / check - 1.0
        ok       &= abs(rel_error) < eps99
    #
    # Check derivative of g_i (x) w.r.t x_j
    # is zero for i < j and  g_i(x) / x_j otherwise
    J = g.jacobian(x)
    for j in range(nx) :
        for i in range(nx) :
            if i < j :
                ok &= J[i,j] == 0.0
            else :
                check     = gx[i] / x[j]
                rel_error = J[i,j] / check - 1.0
                ok       &= abs(rel_error) < eps99

    return ok
```

xsrst input file: `example/python/numeric/ode_multi_step_xam.py`

Example

numeric_ode_multi_step_xam_py

Source Code

```
import numpy
import copy
def ode_multi_step(one_step, fun, t_all, y_init ) :
    dtype      = type(y_init[0])
    n_var      = y_init.size
    n_step     = t_all.size - 1
    y_all      = numpy.empty( (n_step+1, n_var), dtype = dtype  )
    y1         = y_init
    t1         = t_all[0]
    y_all[0,:] = y1
    for i in range(n_step) :
        fun.set_t_all_index(i)
        y0            = y1
        t0            = t1
        t1            = t_all[i+1]
        t_step        = t1 - t0
        y1            = one_step(fun, t0, y0, t_step)
        y_all[i+1,:]  = copy.copy(y1)
    return y_all
```

---

xsrst input file: `example/python/numeric/ode_multi_step.py`

numeric_optimize_fun_class

2.5 A Helper Class That Defines Functions Needed for Optimization

- *Syntax*
- *Purpose*
- *objective_ad*
- *constraint_ad*
- ***optimize_fun***
  - *objective_fun*
  - *objective_grad*
  - *objective_hess*
  - *constraint_fun*
  - *constraint_jac*
  - *constraint_hess*
- *Example*
- *Source Code*

Syntax

optimize_fun = `optimize_fun_class` (*objective_ad* , *constraint_ad*)

Purpose

This class is an aid solving optimization problems of the form

$$\begin{array}{rl} \text{minimize} & f(x) \text{ w.r.t } x \\ \text{subject to} & a \leq g(x) \leq b \end{array}$$

where x is a vector, $f(x)$ is a scalar, and $a, g(x), b$ are all vectors with the same length. We use n, m for the length of the vectors x and $g(x)$ respectively.

objective_ad

This is a *d_fun* representation of the function $f(x)$.

constraint_ad

This is a `d_fun` representation of the function $g(x)$.

optimize_fun

This class object has the following functions defined:

objective_fun

The syntax

y = *optimize_fun* . `objective_fun` (*x*)

sets $y = f(x)$ where *x* is a numpy vector with length *n* and *y* is a scalar.

objective_grad

The syntax

z = *optimize_fun* . `objective_grad` (*x*)

sets $z = f^{(1)}(x)$ where *x* and *z* are numpy vectors with length *n* .

objective_hess

The syntax

h = *optimize_fun* . `objective_hess` (*x*)

sets $h = f^{(2)}(x)$ where *x* is a numpy vector with length *n* and *h* is a numpy *n* by *n* matrix.

constraint_fun

The syntax

y = *optimize_fun* . `constraint_fun` (*x*)

sets $y = g(x)$ where *x* (*y*) is a numpy vector with length *n* (*m*).

constraint_jac

The syntax

J = *optimize_fun* . `constraint_jac` (*x*)

sets $J = g^{(1)}(x)$ where *x* is a numpy vector with length *n* and *J* is a numpy *m* by *n* matrix.

constraint_hess

The syntax

H = *optimize_fun* . `constraint_hess` (*x* , *v*)

sets

$$H = \sum_{i=0}^{m-1} v_k g_i^{(2)}(x)$$

where *x* is a numpy vector with length *n* and *H* is a numpy *n* by *n* matrix.

numeric_optimize_fun_xam_py

2.5.1 Example Using optimize_fun_class with Scipy Optimization

- *Reference*
- *Problem*
- *trust_constr*
- *Source Code*

Reference

This problem comes form the Interfaces section of the Ipopt documentation.

Problem

$$
\begin{array}{rr}
\text{minimize} & x_0 x_3 (x_0 + x_1 + x_2) + x_2 \\
\text{subject to} & x_0 x_1 x_2 x_3 \geq 25 \\
& x_0^2 + x_1^2 + x_2^2 + x_3^2 = 40 \\
& 1 \leq x \leq 5
\end{array}
$$

with the starting point $x = (1, 5, 5, 1)$. The optimal value for x is

$$(1.00000000 \text{ , } 4.74299963 \text{ , } 3.82114998 \text{ , } 1.37940829)$$

trust_constr

This is one of the scipy.optimize methods.

Source Code

```
def optimize_fun_xam() :
    #
    import numpy
    import cppad_py
    import scipy.optimize
    from optimize_fun_class import optimize_fun_class
    #
    ok = True
    #
    def a_objective(ax)  :
        return ax[0] * ax[3] * ( ax[0] + ax[1] + ax[2] ) + ax[2]
    def a_constraint(ax) :
        return [ numpy.prod(ax), numpy.sum( ax * ax) ]
    #
    # objective_ad
```

(continues on next page)

(continued from previous page)

```
    x = numpy.array( [ 1.0, 2.0, 3.0, 4.0 ] )
    ax = cppad_py.independent(x)
    ay = numpy.array( [a_objective(ax)] )
    objective_ad = cppad_py.d_fun(ax, ay)
    #
    # constraint_ad
    ax = cppad_py.independent(x)
    ay = numpy.array( a_constraint(ax) )
    constraint_ad = cppad_py.d_fun(ax, ay)
    #
    # optimize_fun
    optimize_fun = optimize_fun_class(objective_ad, constraint_ad)
    #
    # constraints
    lower_bound = [      25.0, 40.0 ]
    upper_bound = [ numpy.inf, 40.0 ]
    nonlinear_constraint = scipy.optimize.NonlinearConstraint(
        optimize_fun.constraint_fun,
        lower_bound,
        upper_bound,
        jac           = optimize_fun.constraint_jac,
        hess          = optimize_fun.constraint_hess,
        keep_feasible = False
    )
    constraints       = [nonlinear_constraint]
    #
    # bounds
    lower_bound = 4 * [ 1.0 ]
    upper_bound = 4 * [ 5.0 ]
    bounds = scipy.optimize.Bounds(
        lower_bound,
        upper_bound,
        keep_feasible = False
    )
    #
    # start_point
    start_point = [ 1.0, 5.0, 5.0, 1.0 ]
    #
    options = {
                            'gtol' : 1e-8,
                            'xtol' : 1e-8,
                     'barrier_tol' : 1e-8,
               'initial_tr_radius' : 1.0,
          'initial_constr_penalty' : 1.0,
        'initial_barrier_tolerance' : 0.1,
        'initial_barrier_parameter' : 0.1,
             'factorization_method' : None,
             'finite_diff_rel_step' : None,
                          'maxiter' : 50,
                          'verbose' : 0,
    }
    #
    result = scipy.optimize.minimize(
        optimize_fun.objective_fun,
        start_point,
        method      = 'trust-constr',
        jac         = optimize_fun.objective_grad,
```

(continues on next page)

(continued from previous page)

```
        hess        = optimize_fun.objective_hess,
        constraints = constraints,
        options     = options,
        bounds      = bounds,
    )
    ok = ok and result.success
    #
    optimal_point = result.x
    check         = [ 1.00000000, 4.74299963, 3.82114998, 1.37940829 ]
    rel_error     = optimal_point / check - 1.0
    ok = ok and numpy.all( abs(rel_error) < 1e-5 )
    return ok
```

xsrst input file: `example/python/numeric/optimize_fun_xam.py`

Example

numeric_optimize_fun_xam_py

Source Code

```
import numpy
import cppad_py

class optimize_fun_class :
    def __init__(self, objective_ad, constraint_ad=None) :
        self.objective_ad  = objective_ad
        self.constraint_ad = constraint_ad
    #
    def objective_fun(self, x) :
        # objective as a vector
        y = self.objective_ad.forward(0, x)
        return y[0]
    #
    def objective_grad(self, x) :
        # Jacobian as a matrix
        J = self.objective_ad.jacobian(x)
        # change to a vector
        return J.flatten()
    #
    def objective_hess(self, x) :
        w = numpy.array( [ 1.0 ] )
        H = self.objective_ad.hessian(x, w)
        return H
    #
    def constraint_fun(self, x) :
        return self.constraint_ad.forward(0, x)
    #
    def constraint_jac(self, x) :
        # Jacobian as a matrix
        J = self.constraint_ad.jacobian(x)
        return J
```

(continues on next page)

(continued from previous page)

```
    #
    def constraint_hess(self, x, v) :
        H = self.constraint_ad.hessian(x, v)
        return H
```

---

xsrst input file: `example/python/numeric/optimize_fun_class.py`

numeric_seirwd_model

2.6 A Susceptible Exposed Infectious Recovered and Death Model

- *Purpose*
- *Syntax*
- *Notation*
- *ODE*
- *method*
- *t_all*
- ***p_all***
    - *alpha*
- *initial*
- *n_step*
- *seirwd_all*
- *Conservation of Mass*
- *Example*
- *Source Code*

Purpose

This routine can be used with `ad_double` .

Syntax

seirwd_all = `seirwd_model` (
    method , *t_all* , *p_all* , *initial* , *n_step* = 1
)

Notation

$S(t)$	size of the Susceptible group
$E(t)$	size of the Exposed group
$I(t)$	size of the Infectious group
$R(t)$	size Recovered group
$W(t)$	size of the group that will die
$D(t)$	size of the group that has died
$\alpha(t)$	infectious group size exponent
$\beta(t)$	infectious rate
$\sigma(t)$	incubation rate
$\gamma(t)$	recovery rate
$\xi(t)$	loss of immunity rate
$\chi(t)$	excess mortality rate
$\delta(t)$	delay between infectious and death

ODE

The ordinary differential equation for this model is:

$$
\begin{array}{rcll}
\dot{S} & = & -\beta S I^{\alpha} & +\xi R \\
\dot{E} & = & +\beta S I^{\alpha} & -\sigma E \\
\dot{I} & = & +\sigma E & -(\gamma + \chi) I \\
\dot{R} & = & +\gamma I & -\xi R \\
\dot{W} & = & +\chi I & -\delta W \\
\dot{D} & = & +\delta W &
\end{array}
$$

where we dropped the time dependence in the equations above. This model does not account for death by other causes. It is similar to the standard SEIRS model with the following differences:

1. The total population N is not included in this model, so the units for β are different.
2. This model tracks death due to the condition using the compartments W and D.

method

This `str` must be either `runge4` or `rosen3` . It determines if *runge4_step* or *rosen3_step* is used to solve the ODE.

t_all

The argument *t_all* is a vector that is monotone increasing or decreasing. The type of its elements can be `float` or `a_double` . The smaller the spacing between time points, the more accurate the approximation is. Note two points can be equal; i.e., no zero spacing. We call *t_all* [0] the initial time and *t_all* [-1] the final time.

p_all

This argument is a list of dictionaries. The i-th element of the list has the following *key* , *value* pairs:

key	*value*
`'alpha'`	$\alpha(t_i)$
`'beta'`	$\beta(t_i)$
`'sigma'`	$\sigma(t_i)$
`'gamma'`	$\gamma(t_i)$
`'xi'`	$\xi(t_i)$
`'chi'`	$\chi(t_i)$
`'delta'`	$\delta(t_i)$

where *t_i* is the time *t_all* [*i*] . The type of *value* can be `float` or `a_double` . Each of the these parameters will be linearly interpolated for times between the those in *t_all* .

alpha

There is a special restriction that $\alpha(t)$ must be constant; i.e. $\alpha(t_i) = \alpha(t_0)$ for all i. This is because talking the derivative of I^α respect to I has a special representation when $\alpha = 1$. Using a special representation for that case would not work with AD unless α is constant.

initial

is a vector of length four containing the initial values for S, E, I, R, W, D in that order. The type of its elements can be `float` or `a_double` .

n_step

This is the number of numerical integration steps to use for each time interval in the *t_all* array. It must be an `int` greater or equal one. The larger *n_step* the more computational effort and the more accurate the solution. The default value is for *n_step* is one.

seirwd_all

The return value *seirwd_all* is a numpy matrix with row dimension equal to the number of elements in *t_all* and column dimension equal to six. The value *seirwd_all* [*i* , *j*] is the approximate solution for the j-th compartment at time *t_all* [*i*] . The compartments have the same order as in *initial* and `seirwd` [0, *:* is equal to *initial* . The sequence of floating point operations only depends on *t_all* and the operations used to compute *p_fun* .

Conservation of Mass

Note that the sum of S, E, I, R, W, and D should be constant; i.e., up to numerical accuracy, it not depend on time.

numeric_seirwd_model_xam_py

2.6.1 Example Using seris_model

```
import numpy
from seirwd_model import seirwd_model
from seirwd_model import test_fun_derivatives
#
def check_method(method) :
    ok    = True
    #
    alpha = 9.5           # exponent in infectious group size
    sigma = 1.0 / 5.0     # average 5 days between exposure and infectious
    gamma = 1.0 / 20.0    # average 20 days between infectious and recovered
    xi    = 1.0 / 365.0   # average 1 year of immunity
    chi   = gamma / 10.0  # probabiliy of death is 1/10 that of recovery
    delta = 1.0 / 10.0    # to days between no-longer infectious and death
    #
    # t_all
    t_all    = numpy.array( [ 0.0, 0.1, 0.2 ] )
    beta_all =  [ 0.30, 0.20, 0.10 ]
    #
    # p_all
    p_all    = list()
    for i in range(t_all.size) :
        p = {
            'alpha' : alpha,
            'beta'  : beta_all[i],
            'sigma' : sigma ,
            'gamma' : gamma ,
            'xi'    : xi ,
            'chi'   : chi ,
            'delta' : delta ,
        }
        p_all.append(p)
    #
    # initial
    I0  = 0.02
    E0  = 0.02
    R0  = 0.02
    S0  = 1.0 - I0 - E0 - R0
    W0  = 0.0
    D0  = 0.0
    initial  = numpy.array( [ S0, E0, I0, R0, W0, D0 ] )
    #
    # check derivatives in seirwd_model needed when method is rosen3
    # (This is not part of the User API for seirwd_model)
    if method == 'rosen3' :
        ok = ok and test_fun_derivatives(t_all, p_all, initial)
    #
    # solve the ODE
```

(continues on next page)

(continued from previous page)

```
    seirwd_all = seirwd_model(method, t_all, p_all, initial)
    #
    # check solution using midpoint values
    for i in range( len(t_all) - 1 ) :
        # midpoint values
        t_mid      = (t_all[i+1] + t_all[i]) / 2.0
        seirwd_mid = (seirwd_all[i,:] + seirwd_all[i+1,:]) / 2.0
        S, E, I, R, W, D = seirwd_mid
        #
        # differential equation
        p       = dict()
        for key in ['alpha', 'beta', 'sigma', 'gamma', 'xi', 'chi', 'delta'] :
            p[key] = (p_all[i][key] + p_all[i+1][key]) / 2.0
        #
        dot     = numpy.empty( 6, dtype = float)
        Ia      = pow(I, p['alpha'])
        dot[0]  = - p['beta'] *  S * Ia  + p['xi']    * R
        dot[1]  = + p['beta'] *  S * Ia - p['sigma'] * E
        dot[2]  = + p['sigma'] * E      - (p['gamma'] + p['chi']) * I
        dot[3]  = + p['gamma'] * I      - p['xi']    * R
        dot[4]  = + p['chi']   * I      - p['delta'] * W
        dot[5]  = + p['delta'] * W
        #
        # difference over interval
        seirwd_delta = seirwd_all[i+1,:] - seirwd_all[i,:]
        t_delta      = t_all[i+1]      - t_all[i]
        #
        check  = seirwd_delta / t_delta
        rel_error = dot / check - 1.0
        # print(rel_error)
        ok        = ok and all( abs(rel_error) < 1e-2 )
    #
    # now check solution using twice as many Runge-Kutta steps
    n_step      = 2
    seirwd_check = seirwd_model(method, t_all, p_all, initial, n_step)
    error       = seirwd_all - seirwd_check
    # print(error)
    ok          = ok and numpy.all( abs(error) < 1e-7 )
    #
    return ok

def seirwd_model_xam() :
    ok = True
    ok = ok and check_method('runge4');
    ok = ok and check_method('rosen3');
    return ok
```

xsrst input file: `example/python/numeric/seirwd_model_xam.py`

Example

numeric_seirwd_model_xam_py

Source Code

```
import numpy
from ode_multi_step import ode_multi_step
from rosen3_step import rosen3_step
from runge4_step import runge4_step
from rosen3_step import check_rosen3_step
# ------------------------------------------------------------------------------
class fun_class :
    # ---------------------------------------------------------------------
    # init
    def __init__(self, t_all, p_all, n_step) :
        self.t_all  = t_all
        self.p_all  = p_all
        self.n_step = n_step
        self.index  = None
        self.p      = None
    # ---------------------------------------------------------------------
    # set_t_all_index
    def set_t_all_index(self, refine_index) :
        t_all_index  = int( numpy.floor( refine_index / self.n_step ) )
        self.index   = t_all_index
    # ---------------------------------------------------------------------
    # get_p
    def get_p(self, t, seirwd) :
        assert self.index != None
        t_all = self.t_all
        p_all = self.p_all
        index = self.index
        #
        # linearly interpolate the parameter values
        p      = dict()
        t_left = t_all[index]
        t_diff = t_all[index + 1] - t_left
        for key in ['alpha', 'beta', 'sigma', 'gamma', 'xi', 'chi', 'delta'] :
            p_left = p_all[index][key]
            p_diff = p_all[index + 1][key] - p_left
            p[key] = p_left + (t - t_left) * p_diff / t_diff
        #
        return p
    # ---------------------------------------------------------------------
    # get_p_t
    def get_p_t(self) :
        assert self.index != None
        t_all = self.t_all
        p_all = self.p_all
        index = self.index
        #
        # linearly interpolate the parameter values
        p_t    = dict()
        t_diff = t_all[index + 1] - t_all[index]
        for key in ['alpha', 'beta', 'sigma', 'gamma', 'xi', 'chi', 'delta'] :
```

(continues on next page)

(continued from previous page)

```
            p_diff   = p_all[index + 1][key] - p_all[index][key]
            p_t[key] = p_diff / t_diff
        #
        return p_t
    # ----------------------------------------------------------------------
    # f
    def f(self, t, seirwd) :
        p                = self.get_p(t, seirwd)
        S, E, I, R, W, D = seirwd
        #
        Ia     = pow(I, p['alpha'])
        #
        # compute f(t, y)
        Sdot   = - p['beta']  *  S * Ia  + p['xi']    * R
        Edot   = + p['beta']  *  S * Ia  - p['sigma'] * E
        Idot   = + p['sigma'] * E        - (p['gamma'] + p['chi']) * I
        Rdot   = + p['gamma'] * I        - p['xi']    * R
        Wdot   = + p['chi']   * I        - p['delta'] * W
        Ddot   = + p['delta'] * W
        #
        return numpy.array([ Sdot, Edot, Idot, Rdot, Wdot, Ddot])
    # ----------------------------------------------------------------------
    # f_t
    def f_t(self, t, seirwd) :
        p                = self.get_p(t, seirwd)
        p_t              = self.get_p_t()
        S, E, I, R, W, D = seirwd
        #
        Ia     = pow(I, p['alpha'])
        #
        #
        Sdot_t   = - p_t['beta']  *  S * Ia  + p_t['xi']    * R
        Edot_t   = + p_t['beta']  *  S * Ia  - p_t['sigma'] * E
        Idot_t   = + p_t['sigma'] * E        - (p_t['gamma'] + p_t['chi']) * I
        Rdot_t   = + p_t['gamma'] * I        - p_t['xi']    * R
        Wdot_t   = + p_t['chi']   * I        - p_t['delta'] * W
        Ddot_t   = + p_t['delta'] * W
        return numpy.array([ Sdot_t, Edot_t, Idot_t, Rdot_t, Wdot_t, Ddot_t])
    # ----------------------------------------------------------------------
    # f_y
    def f_y(self, t, seirwd) :
        p                = self.get_p(t, seirwd)
        S, E, I, R, W, D = seirwd
        #
        Ia     = pow(I, p['alpha'])
        if p['alpha'] == 1.0 :
            Ia_I = 1.0
        else :
            Ia_I = p['alpha'] * pow(I, p['alpha'] - 1.0 )
        #
        ny     = 6
        type_y = type( seirwd[0] )
        zero   = type_y( 0.0 )
        J      = numpy.empty( (ny,ny), dtype = type_y )
        #
        # partials of Sdot(t, y) w.r.t. y
        Sdot_S   = - p['beta']  *  Ia
```

(continues on next page)

(continued from previous page)

```
        Sdot_I    = - p['beta']  *  S * Ia_I
        Sdot_R    = + p['xi']
        J[0,:]    = [ Sdot_S, zero, Sdot_I, Sdot_R, zero, zero ]
        #
        # partial of Edot(t, y) w.r.t y
        Edot_S    = + p['beta']  *  Ia
        Edot_I    = + p['beta']  *  S * Ia_I
        Edot_E    =  - p['sigma']
        J[1,:]    = [ Edot_S, Edot_E, Edot_I, zero, zero, zero ]
        #
        # partial of Idot(t, y) w.r.t y
        Idot_E    = + p['sigma']
        Idot_I    = - (p['gamma'] + p['chi'])
        J[2,:]    = [ zero, Idot_E, Idot_I, zero, zero, zero ]
        #
        # partial Rdot(t, y) w.r.t y
        Rdot_I    = + p['gamma']
        Rdot_R    = - p['xi']
        J[3,:]    = [ zero, zero, Rdot_I, Rdot_R, zero, zero ]
        #
        # partial Wdot(t, y) w.r.t. y
        Wdot_I    = + p['chi']
        Wdot_W    = - p['delta']
        J[4,:]    = [ zero, zero, Wdot_I, zero, Wdot_W, zero ]
        #
        # partial Ddot(t, y) w.r.t y
        Ddot_W    = + p['delta']
        J[5,:]    = [ zero, zero, zero, zero, Ddot_W, zero ]
        #
        return J
# ------------------------------------------------------------------------------
# check the fun derivatives required by rosen3_step
def test_fun_derivatives(t_all, p_all, initial) :
    ok     = True
    n_step = 1
    fun  = fun_class(t_all, p_all, n_step)
    index  = 0
    fun.set_t_all_index(index)
    ti   = t_all[index]
    yi   = initial
    h    = t_all[index + 1] - t_all[index]
    ok   = ok and check_rosen3_step(fun, ti, yi, h)
    return ok
# ------------------------------------------------------------------------------
# seriwd_model
def seirwd_model(method, t_all, p_all, initial, n_step = 1) :
    for i in range( len(t_all) - 1 ) :
        assert p_all[i+1]['alpha'] == p_all[0]['alpha']
    #
    fun      = fun_class(t_all, p_all, n_step)
    if method == 'runge4' :
        one_step = runge4_step
    elif method == 'rosen3' :
        one_step = rosen3_step
    else :
        assert False
    if n_step == 1 :
```

(continues on next page)

(continued from previous page)

```
        seirwd_all  = ode_multi_step(one_step, fun, t_all, initial)
    else :
        # t_refine
        t_refine = list()
        for i in range( len(t_all) - 1 ) :
            step = (t_all[i+1] - t_all[i]) / n_step
            for j in range(n_step) :
                t_refine.append( t_all[i] + j * step )
        t_refine.append( t_all[-1] )
        t_refine = numpy.array( t_refine )
        #
        seirwd_refine   = ode_multi_step(one_step, fun, t_refine, initial)
        index_all   = [ i * n_step for i in range(len(t_all)) ]
        seirwd_all   = seirwd_refine[ index_all ]
    #
    return seirwd_all
```

---

xsrst input file: `example/python/numeric/seirwd_model.py`

numeric_covid_19_xam_py

2.7 Example Fitting an SEIRWD Model for Covid-19

- *Covariates*
- ***Model ODE***
  - *beta(t)*
  - *Other Rates*
  - *Initial Values*
  - *Ode Solver*
- ***Unknown Parameters***
  - *Maximum Likelihood*
  - *Model Bounds*
  - *Actual Bounds*
- ***Data***
  - *sample_interval*
  - *data_file*
  - *Coefficient of Variation*
  - *Simulation*
  - *Weighted Residuals*
- *Random Seed*
- *Random Start*
- ***Display Fit Results***

- *Plot*
- *Printout*

- *Debug Output*
- *Source Code*

Covariates

In this example there are two covariates that affect the infectious rate β: social mobility $c_0(t)$, Covid-19 testing $c_1(t)$, and scaled time $c_2(t)$. The covariates are known functions of time. The mobility covariate has been shifted and scaled so it is in the interval [-1, 0]. The testing covariate has been shifted and scaled so it is in the interval [0, 1]. Note that the maximum mobility and the minimum testing corresponds to the normal (baseline) condition. The scaled time covariate is shifted and scaled version of time so that it is in the interval [0, 1] with the first data point corresponding to time zero. It is assumed here that the baseline condition corresponds to time zero.

Model ODE

We use the *seirwd* model and notation.

beta(t)

Our model for the infectious rate is

$$\beta(t) = \bar{\beta} \exp[m_0 c_0(t) + m_1 c_1(t) + m_2 c_2(t)]$$

where $\bar{\beta}$ is the baseline value for the infectious rate, m_0 is the social covariate multiplier, and m_1 is the Covid-19 testing covariate multiplier. The baseline value $\bar{\beta}$ is the infectious rate corresponding to all the covariates being zero. The covariate multipliers, and the baseline infectious rate, are unknown.

Other Rates

The other rates $\alpha(t)$, $\sigma(t)$, $\gamma(t)$, $\xi(t)$, $\chi(t)$, $\delta(t)$, constant functions with known values:

```
alpha_known  = 0.95
sigma_known  = 0.2
gamma_known  = 0.1
chi_known    = 0.1
xi_known     = 0.00
delta_known  = 0.2
```

All of theses rates must be non-negative.

Initial Values

The initial size of the Recovered group $R(0)$ and of the Death group $D(0)$ is zero. We use fraction of the total population for sizes, so the sum of the other initial values is one. We treat the initial Infected group $I(0)$, and Will die group $W(0)$, as unknown parameters in the model. We would like to also solve for the initial exposed population but that model has identifiability problems, so we use the following approximation for the initial exposed group

$$E(0) = I(0)\gamma/\sigma$$

The initial Susceptible group $S(0)$ is expressed as a function of the other initial conditions:

$$S(0) = 1 - E(0) - I(0) - W(0)$$

Ode Solver

There are two choices for *ode_method* , the method used to solve the ODE: *runge4* and *rosen3*. In addition, we can choose *ode_n_step* , the number of step to take for each time interval in *t_all* , before it is sub-sampled using the *sample_interval*.

```
ode_method = 'runge4'
ode_n_step = 4
```

Unknown Parameters

The unknown parameter vector in this model is

$$x = [m_0, m_1, m_2, I(0), W(0), \bar{\beta}]$$

```
x_name = [ 'm_mobility', 'm_testing', 'm_stime', 'I(0)', 'W(0)', 'beta_bar' ]
```

Maximum Likelihood

We use a Gaussian likelihood for each of the differences in the cumulative deaths. The unknown parameters are estimated by maximizing the product of these likelihoods; i.e., the differences are modeled as being independent. The covariance of the estimates is approximated by the inverse of the observed information matrix. AD is used to compute first and second derivatives of the likelihood w.r.t. the unknown parameters x. These derivatives are used during optimization as well as for computing the observed information matrix.

Model Bounds

The infection rate $\beta(t)$ must be non-negative; i.e.,

$$0 \leq \bar{\beta} \exp[m_0 c_0(t) + m_1 c_1(t) + m_2 c_2(t)]$$

is true for all t. In addition, the size of the groups can not be negative. It is sufficient to enforce this constraint on the initial conditions; i.e.,

$$\begin{array}{rcr} 0 & \leq & \bar{\beta} \\ 0 & \leq & I(0) \\ 0 & \leq & W(0) \end{array}$$

Actual Bounds

The following actual upper and lower bounds for the unknown parameters are used as an as an aid to the optimizer:

```
    n_x = len(x_name)
    x_lower                 = numpy.zeros( n_x, dtype=float)
    x_upper                 = numpy.zeros( n_x, dtype=float)
    x_upper[ x_sim > 0.0 ] = actual_bound_factor * x_sim[ x_sim > 0.0 ]
    x_lower[ x_sim < 0.0 ] = actual_bound_factor * x_sim[ x_sim < 0.0 ]
```

where *x_sim* is the *simulation* value for the unknown parameters and *actual_bound_factor* is chosen below. The problem has not really been solved if bounds, other than the model bounds above, are active at the solution of the optimization problem.

```
actual_bound_factor = 5.0
```

Data

The data in this model is the cumulative number of deaths, as a fraction of the total population and as a function of time. We assume that new deaths are recorded for time intervals and the cumulative deaths is the sum of these recordings. For this reason, we model the difference of the cumulative deaths between time points as independent.

sample_interval

It is possible to sub-sample the data in order to reduce noise. The cumulative death data is just sub-sampled since the reduces noise by the summing the differences corresponding to a longer time period. The covariate data is averaged over the sample interval. The *sample_interval* must be either one or a positive even integer (even so an original data point corresponds to the center of the interval).

```
sample_interval = 1
```

data_file

If the data file name is the empty string, the cumulative death data, and corresponding covariates, are simulated by the program. Otherwise, the data file must be a CSV file with the following columns: *day* , *death* , *mobility* , *testing* . In this case the data file is used for the cumulative death and corresponding covariates.

```
data_file = '/home/bradbell/Downloads/561.csv'          # Pennsylvania
data_file = '/home/bradbell/trash/covid_19/seirwd.csv' # New York
data_file = ''                                          # empty string
```

Coefficient of Variation

This is the coefficient of variation for the differences in the cumulative death data as a fraction, not a percent. If this value is zero, a CV of zero is used for data simulation and a CV of one in the definition of the likelihood. This enables checking that the unknown parameters can be accurately identified using perfect data. For real data (when *data_file* is not empty) this value should be adjusted so that the average residual has variance one.

```
death_data_cv = 0.25
```

Note this is the noise level in the original data before it is sub-sampled using *sample_interval*.

Simulation

If *data_file* is the empty string, the data is simulated using the following values for the *unknown_parameters*:

```
m_mobility_sim    =   1.0  # m_0
m_testing_sim     = - 1.0  # m_1
m_stime_sim       = - 1.0  # m_2
I0_sim            =  2e-5  # I(0)
W0_sim            =  2e-5  # W(0)
beta_bar_sim      =  2.0   # baseline value for beta
```

Weighted Residuals

If *death_data_cv* is zero, $\lambda = 1$, otherwise λ is equal to

death_data_cv * `sqrt` (*sample_interval*)

. (Note that the standard deviation of a sum of independent values is the square root of the sum of the variance of each of the values.) Let y_i be the i-th value for the cumulative death data. The weighted residuals (some times referred to as just the residuals) are

$$r_i = \frac{(y_{i+1} - y_i) - [D(t_{i+1}) - D(t_i)]}{\lambda(y_{i+1} - y_i)}$$

where $D(t)$ is the model for the cumulative data given the fit results. The time corresponding to r_i is $(t_{i+1} + t_i)/2$. We put the data difference in the denominator, instead of the model difference, because it is constant with respect to the unknown parameters.

Random Seed

This is the random seed used to simulate noise in the data. If this value is zero, the system clock is used to choose the random seed.

```
random_seed = 0
```

Random Start

The optimizer needs a good starting point in order to succeed. This is the number of random points, between the lower and upper limits, that are checked. The point with the best objective value is chosen as the starting point for the optimization.

```
n_random_start = 4000
```

Display Fit Results

If you set this variable to True, a printout and a plot of the fit results is generated.

```
display_fit = False
```

Plot

There are three plots all with time on the x-axis. The first contains the size for all the compartments, except S, as a fraction of the total population. The second contains the model and data for the death difference values. The third contains the weighted residuals corresponding to the death difference data.

Printout

1. The following statistics for the weighted data residuals is printed: the maximum, minimum, average, and average of square.
2. A table with the following columns is printed:

x_name	name of the unknown parameter
x_fit	fit result for the unknown parameter
x_lower	lower bound used for the fit
x_upper	upper bound used for the fit
std_error	asymptotic standard error for the parameter

3. If *data_file* is empty, a table is printed with the following columns is also printed:

x_name	name of the unknown parameter
x_sim	known parameter value used during simulation
x_fit	fit result for the unknown parameter
rel_error	relative error for fit versus simulation
residual	*std_error* weighted residual for fit versus simulation

Debug Output

If this flag is true a lot of debugging output is printed.

```
debug_output = False
```

Source Code

```
from pdb import set_trace
from matplotlib import pyplot
from matplotlib import gridspec
import scipy.optimize
import numpy
import random
import time
import csv
import copy
#
import cppad_py
from optimize_fun_class import optimize_fun_class
from seirwd_model import seirwd_model
#
def subsample(file_data) :
    if sample_interval == 1 :
        return file_data
    assert sample_interval % 2 == 0
    sample_interval_2 = int( sample_interval / 2 )
    #
    n_data          = len(file_data)
    n_data_interval = n_data - 1
    n_sub_interval  = int( n_data_interval / sample_interval )
    n_sub           = n_sub_interval
    sub_data        = list()
    for j in range(n_sub) :
        i                = sample_interval * j + sample_interval_2
        row              = dict()
        row['day']       = file_data[i]['day']
        row['death']     = file_data[i]['death']
        for key in ['mobility', 'testing'] :
            row[key]  = 0.0
            for k in range(sample_interval + 1) :
                i         = sample_interval * j + k
                row[key] += file_data[i][key]
            row[key] /= float(sample_interval + 1)
        sub_data.append(row)
    return sub_data
#
# t_all, covariates
if data_file != '' :
    # getting cumulative death and covariates from data_file
    file_in   = open(data_file, 'r')
    file_data = list()
    reader    = csv.DictReader(file_in)
    for row in reader :
        for key in ['day', 'death', 'mobility', 'testing']  :
```

(continues on next page)

(continued from previous page)

```
            row[key] = float( row[key] )
        file_data.append(row)
    file_in.close()
    file_data = subsample(file_data)
    #
    n_time       = len(file_data)
    t_all        = numpy.empty(n_time, dtype=float)
    covariates   = numpy.empty( (n_time, 3) )
    for i in range(n_time) :
        t_all[i]        = file_data[i]['day']
        covariates[i,0] = file_data[i]['mobility']
        covariates[i,1] = file_data[i]['testing']
    for i in range(n_time) :
        stime           = (t_all[i] - t_all[0]) / (t_all[-1] - t_all[0])
        covariates[i,2] = stime
else :
    # simulating cumulative death and covariates
    t_start = 0.0
    t_stop  = 90.0
    t_step  = float(sample_interval)
    t_all = numpy.arange(t_start, t_stop, t_step)
    #
    n_time = t_all.size
    covariates   = numpy.empty( (n_time, 3) )
    for i in range(n_time) :
        t               = t_all[i]
        mobility        = 0.0 if t < 10.0 else -1.0
        testing         = 0.0 if t < 20.0 else 1.0
        stime           = t / t_stop
        covariates[i,:] = [ mobility, testing, stime ]
#
# mobility in [-1, 0]
assert numpy.all( covariates[:,0] <= 0.0 )
assert numpy.all( -1.0 <= covariates[:,0] )
#
# testing in [0, 1]
assert numpy.all( 0.0 <= covariates[:,1] )
assert numpy.all( covariates[:,1] <= 1.0 )
#
# stime in [0, 1]
assert numpy.all( 0.0 <= covariates[:,2] )
assert numpy.all( covariates[:,2] <= 1.0 )
#
# other initial conditions used to simulate data
E0_sim     = I0_sim * gamma_known / sigma_known
S0_sim     = 1.0 - E0_sim - I0_sim - W0_sim
R0_sim     = 0.0
D0_sim     = 0.0
#
# x_sim
x_sim = numpy.array( [
    m_mobility_sim, m_testing_sim, m_stime_sim, I0_sim, W0_sim, beta_bar_sim
] )
#
#
# actual_seed
# Make a list so that we can set its element value inside a routine
```

(continues on next page)

(continued from previous page)

```
actual_seed = [ random_seed ]
#
def x2seirwd_all(x) :
    #
    # unpack x
    cov_mul            = x[0 : 3]
    [I0, W0, beta_bar] = x[3 : 6]
    #
    # beta_all
    beta_all = beta_bar * numpy.exp( numpy.matmul(covariates, cov_mul) )
    #
    # p_all
    p_all = list()
    for i in range( beta_all.size ) :
        p = {
            'alpha' : alpha_known,
            'sigma' : sigma_known,
            'gamma' : gamma_known,
            'chi'   : chi_known,
            'xi'    : xi_known,
            'delta' : delta_known,
        }
        p['beta'] = beta_all[i]
        p_all.append( p )
    #
    # initial
    E0 = I0 * gamma_known / sigma_known
    S0 = 1.0 - E0 - I0 - W0
    R0 = 0.0
    D0 = 0.0
    initial  = numpy.array( [S0, E0, I0, R0, W0, D0] )
    #
    # seirwd_all
    n_step = sample_interval * ode_n_step
    seirwd_all = seirwd_model(ode_method, t_all, p_all, initial, n_step)
    #
    return seirwd_all
#
def weighted_data_residual(D_data, D_model) :
    Ddiff_data   = numpy.diff(D_data)
    if numpy.any( Ddiff_data <= 0 )  :
        print('covid_19_xam: cumutative death data not increasing at times: ')
        t_msg = t_all[0 : -1]
        print( t_msg[ Ddiff_data <= 0 ] )
    Ddiff_model  = numpy.diff(D_model)
    residual     = (Ddiff_data - Ddiff_model) / Ddiff_data
    if death_data_cv > 0.0 :
        residual = residual / (numpy.sqrt(sample_interval) * death_data_cv)
    return residual
#
# objective
# t_all and D_data, are constants relative to the objective function
def objective(t_all, D_data, x) :
    #
    # compute model for data
    seirwd_all = x2seirwd_all(x)
    #
```

(continues on next page)

(continued from previous page)

```
    # Model for the cumulative death as function of time
    D_model   = seirwd_all[:,5] # column order is S, E, I, R, W, D
    #
    # compute negative log Gaussian likelihood dropping variance terms
    # because they are constaint w.r.t the unknown parameters
    residual = weighted_data_residual(D_data, D_model)
    loss     = 0.5 * numpy.sum( residual * residual )
    return loss
#
# objective_d_fun
def objective_d_fun(t_all, D_data) :
    #
    n_x   = len(x_name)
    x     = 0.1 * numpy.ones(n_x)
    ax    = cppad_py.independent(x)
    aloss = objective(t_all, D_data, ax)
    aloss = numpy.array( [ aloss ] )
    #
    objective_ad = cppad_py.d_fun(ax, aloss)
    return objective_ad
#
def simulate_data() :
    #
    # noiseless simulated data
    seirwd_all_sim = x2seirwd_all(x_sim)
    #
    # rng: numpy random number generator
    rng = numpy.random.default_rng(actual_seed[0])
    #
    # D_data
    D_sim       = seirwd_all_sim[:,5]
    Ddiff_sim   = numpy.diff( D_sim )
    std         = numpy.sqrt(sample_interval) * death_data_cv * Ddiff_sim
    noise       = std * rng.standard_normal(size = t_all.size - 1)
    Ddiff_data  = Ddiff_sim + noise
    D_data      = numpy.cumsum(Ddiff_data)
    D_data      = numpy.concatenate( ([0.0], D_data) )
    #
    return D_data

def random_start(n_random, x_lower, x_upper, log_scale, objective) :
    # check beta constraints
    n_x      = len(x_lower)
    #
    x_best   = (x_lower + x_upper) / 2.0
    obj_best = objective(x_best)
    #
    x_low    = copy.copy(x_lower)
    x_up     = copy.copy(x_upper)
    for j in range(n_x) :
        if log_scale[j] :
            x_low[j] = numpy.log(x_lower[j])
            x_up[j]  = numpy.log(x_upper[j])
    #
    # rng: numpy random number generator
    rng = numpy.random.default_rng(actual_seed[0])
    #
```

(continues on next page)

(continued from previous page)

```
    if debug_output :
        print('random_start:')
    for i in range(n_random) :
        x_current   = rng.uniform(x_low, x_up)
        for j in range(n_x) :
            if log_scale[j] :
                x_current[j] = numpy.exp(x_current[j])
        obj_current = objective(x_current)
        if obj_current < obj_best :
            if debug_output :
                print( 'sample # =', i, ', objective = ', obj_current)
            x_best = x_current
            obj_best = obj_current
    #
    return x_best

def display_fit_results(D_data, x_fit, x_lower, x_upper, std_error) :
    n_x = len(x_fit)
    #
    # seirwd_all_fit
    seirwd_all_fit = x2seirwd_all(x_fit)
    #
    # D_fit
    D_fit = seirwd_all_fit[:,5]
    #
    # data_residual
    data_residual = weighted_data_residual(D_data, D_fit)
    #
    # ----------------------------------------------------------------------
    # printout
    residual_sq = data_residual * data_residual
    print('weighted data residuals')
    print('maximum           =', numpy.max( numpy.max(data_residual) ) )
    print('minimum           =', numpy.max( numpy.min(data_residual) ) )
    print('average           =', numpy.mean(data_residual) )
    print('average of square =', numpy.mean(residual_sq) )
    print('')
    #
    # table of fit values
    fmt = '{:<15s}{:>15s}{:>15s}{:>15s}{:>15s}'
    line = fmt.format(
        'x_name', 'x_fit','x_lower','x_upper', 'std_error'
    )
    print(line)
    for i in range( n_x ) :
        fmt = '{:<15s}{:+15.9f}{:+15.9f}{:+15.9f}{:+15.9f}'
        line = fmt.format(
            x_name[i], x_fit[i], x_lower[i], x_upper[i], std_error[i]
        )
        print(line)
    if data_file == '' :
        # table of fit versus simulation
        print('')
        fmt = '{:<15s}{:>15s}{:>15s}{:>15s}{:>15s}'
        line = fmt.format(
            'x_name', 'x_fit','x_sim','rel_error','residual'
        )
```

(continues on next page)

(continued from previous page)

```
        print(line)
        for i in range( n_x ) :
            if x_sim[i] == 0.0 :
                rel_error = 0.0
            else :
                rel_error = x_fit[i] / x_sim[i] - 1.0
            residual  = (x_fit[i] - x_sim[i]) / std_error[i]
            fmt = '{:<15s}{:+15.9f}{:+15.9f}{:+15.9f}{:+15.9f}'
            line = fmt.format(x_name[i],
                x_fit[i], x_sim[i], rel_error, residual
            )
            print(line)
    # ----------------------------------------------------------------------
    # plot
    # ----------------------------------------------------------------------
    #
    fig  = pyplot.figure(tight_layout = True)
    gs   = gridspec.GridSpec(3, 2)
    ax1  = fig.add_subplot(gs[:,0])
    ax2  = fig.add_subplot(gs[0,1])
    ax3  = fig.add_subplot(gs[1,1])
    ax4  = fig.add_subplot(gs[2,1])
    #
    ax1.plot(t_all, seirwd_all_fit[:,1], 'g-', label='E')
    ax1.plot(t_all, seirwd_all_fit[:,2], 'r-', label='I')
    ax1.plot(t_all, seirwd_all_fit[:,3], 'k-', label='R')
    ax1.plot(t_all, seirwd_all_fit[:,4], 'y-', label='W')
    ax1.plot(t_all, seirwd_all_fit[:,5], 'b-', label='D')
    ax1.legend()
    ax1.set_xlabel('time')
    ax1.set_ylabel('population fraction')
    #
    Ddiff_data = numpy.diff(D_data)
    Ddiff_fit  = numpy.diff(D_fit)
    t_mid      = (t_all[0 : -1] + t_all[1 :]) / 2.0
    ax2.plot(t_mid, Ddiff_data, 'k+' , label='data')
    ax2.plot(t_mid, Ddiff_fit,  'k-' , label='fit')
    ax2.legend()
    ax2.set_ylabel('death differences')
    #
    ax3.plot( t_mid,             data_residual,  'k+' )
    ax3.plot( [t_all[0], t_all[-1]], [0.0, 0.0], 'k-' )
    ax3.set_ylabel('weighted residuals')
    #
    cov_mul  = x_fit[0 : 3]
    beta_bar = x_fit[5]
    beta_all = beta_bar * numpy.exp( numpy.matmul(covariates, cov_mul) )
    ax4.plot(t_all, beta_all, 'k-')
    ax4.set_xlabel('time')
    ax4.set_ylabel('beta(t)')
    #
    pyplot.show()

def covid_19_xam(call_count = 0) :
    ok = True
    #
    # if random seed is zero, use the clock to see the generator.
```

(continues on next page)

(continued from previous page)

```
    if random_seed == 0 :
        actual_seed[0] = int( 13 * time.time() )
    if debug_output :
        print('actual_seed = ', actual_seed[0])
    #
    # D_data
    if data_file == '' :
        D_data = simulate_data()
    else :
        D_data = list()
        for row in file_data :
            D_data.append( float( row['death'] ) )
        D_data = numpy.array(D_data)
    #
    # objective_ad
    objective_ad = objective_d_fun(t_all, D_data)
    #
    # objective: fun, grad, hess
    optimize_fun = optimize_fun_class(objective_ad)
    #
    # options
    options = {
        'gtol'    : 1e-10,
        'xtol'    : 1e-8,
        'maxiter' : 300,
        'verbose' : 0,
    }
    # ------------------------------------------------------------------------
    # This code is used by the $subhead Actual Bounds$$ documentation
# BEGIN_ACTUAL_BOUNDS
    n_x = len(x_name)
    x_lower                 = numpy.zeros( n_x, dtype=float)
    x_upper                 = numpy.zeros( n_x, dtype=float)
    x_upper[ x_sim > 0.0 ] = actual_bound_factor * x_sim[ x_sim > 0.0 ]
    x_lower[ x_sim < 0.0 ] = actual_bound_factor * x_sim[ x_sim < 0.0 ]
# END_ACTUAL_BOUNDS
    #
    # currently not using log-scaling
    log_scale    = numpy.array( n_x * [ False ] )
    #
    assert numpy.all( x_lower <= x_sim )
    assert numpy.all( x_sim <= x_upper )
    #
    bounds = scipy.optimize.Bounds(
        x_lower,
        x_upper,
        keep_feasible = True
    )
    if debug_output :
        print('x_lower =', x_lower)
        print('x_upper =', x_upper)
    # ------------------------------------------------------------------------
    # optimizer loop over beta constraints
    start_point = random_start(
        n_random_start,
        x_lower,
        x_upper,
```

(continues on next page)

(continued from previous page)

```
        log_scale,
        optimize_fun.objective_fun
    );
    if debug_output :
        print( 'start_point = ', start_point )
        print( 'objective   = ', optimize_fun.objective_fun(start_point) )
        print( 'check       = ', objective(t_all, D_data, start_point) )
    constraints  = list()
    result = scipy.optimize.minimize(
        optimize_fun.objective_fun,
        start_point,
        method        = 'trust-constr',
        jac           = optimize_fun.objective_grad,
        hess          = optimize_fun.objective_hess,
        constraints   = constraints,
        options       = options,
        bounds        = bounds,
    )
    x_fit = result.x
    if debug_output :
        print('optimizer success = ', result.success )
        print('optimal objective = ', result.fun )
    # check optimizer status
    ok = ok and result.success
    #
    # H: the observed infromation matrix
    H = optimize_fun.objective_hess(x_fit)
    #
    # Hinv: approxiamtion for covariance of the estimate x_fit
    Hinv = numpy.linalg.inv(H)
    #
    # standard devaition for each component of x_fit
    std_error = numpy.sqrt( numpy.diag(Hinv) )
    #
    # seirwd_all_fit
    seirwd_all_fit = x2seirwd_all(x_fit)
    #
    # D_fit
    D_fit = seirwd_all_fit[:,5]
    #
    # data_resdidual
    data_residual = weighted_data_residual(D_data, D_fit)
    #
    # display_fit_results
    if display_fit :
        display_fit_results(D_data, x_fit, x_lower, x_upper, std_error)
    #
    # check that all the data residuals an less than 3.0
    if numpy.any( numpy.abs( data_residual ) >= 4.0 ) :
        print('covid_19_xam: a weighted data residual >= 4.0')
        ok = False
    #
    # compare fit to simulation truth
    if data_file == '' :
        x_residual = x_fit / x_sim - 1.0
        x_residual[ x_sim == 0.0 ] = 0.0
        if death_data_cv > 0.0 :
```

(continues on next page)

(continued from previous page)

```
            if numpy.any( abs(x_residual) >= 2.0) :
                print('covid_19_xam: a weight x residual >= 2.0')
                ok = False
        else:
            if numpy.any( abs(x_residual) >= 1e-5 ) :
                print('covid_19_xam: an x residual >= 1e-5')
                print('should have prefect fit because death_data_cv = 0')
                ok = False
        #
        if not ok :
            msg  = 'covid_19_xam: Correctness test failed, '
            msg += 'actual random seed = ' + str(actual_seed[0])
            print( msg )
            call_count += 1
            if call_count < 3 and random_seed == 0 :
                print( 're-trying with a differenent random seed')
                ok = covid_19_xam(call_count)
    #
    # check conservation of mass in the compartmental model
    sum_all_fit = numpy.sum(seirwd_all_fit, axis=1)
    eps99       = 99.0 * numpy.finfo(float).eps
    if numpy.any( abs(sum_all_fit - 1.0) > eps99 ) :
        print('covid_19_xam: sum of all compartments not equal 1.0')
        ok = False
    #
    return ok
```

---

xsrst input file: `example/python/numeric/covid_19_xam.py`

1. numeric_simple_inv: *2.1 An AD Compatible Matrix Inverse Routine*
2. numeric_runge4_step: *2.2 One Fourth Order Runge-Kutta ODE Step*
3. numeric_rosen3_step: *2.3 One Third Order Rosenbrock ODE Step*
4. numeric_ode_multi_step: *2.4 Multiple Ode Steps*
5. numeric_optimize_fun_class: *2.5 A Helper Class That Defines Functions Needed for Optimization*
6. numeric_seirwd_model: *2.6 A Susceptible Exposed Infectious Recovered and Death Model*
7. numeric_covid_19_xam_py: *2.7 Example Fitting an SEIRWD Model for Covid-19*

---

xsrst input file: `example/python/numeric/numeric.xsrst`

library

3 The Cppad Py Libraries

- *Children*

Children

py_lib

3.1 The Python Library

- *Children*

Children

py_fun

3.1.1 Cppad Py AD Functions

- *Children*

Children

py_independent

3.1.1.1 Declare Independent Variables and Start Recording

- *Syntax*
- *x*
- *dynamic*
- *ax*
- *adynamic*
- *Purpose*
- *Example*

Syntax

ax = `cppad_py.independent` (*x*)
(*ax* , *adynamic*) = `cppad_py.independent` (*x* , *dynamic*)

x

This argument is a numpy vector with `float` elements. It specifies the number of independent variables and their values during the recording. We use *nx* = *x* . `size` to denote the number of independent variables.

dynamic

This argument is a numpy vector with `float` elements. It specifies the number of independent dynamic parameters and their values during the recording. We use *nd* = *dynamic* . `size` to denote the number of independent dynamic parameters.

ax

This result is a numpy vector with `a_double` elements. This is the vector of independent variables. It has size *nx* and for *i* = 0 to *n* -1

ax [*i*]. `value` () == *x* [*i*]

adynamic

This result is a numpy vector with `a_double` elements. This is the vector of independent dynamic parameters. It has size *nd* and for *i* = 0 to *n* -1

adynamic [*i*]. `value` () == *dynamic* [*i*]

Purpose

This starts a recording of the *a_double* operations. This recording is terminated, and the information is stored, by calling the *d_fun_constructor*. It is terminated, and the information is lost, by calling *abort_recording*.

fun_dynamic_xam_py

3.1.1.1.1 Python: Using Dynamic Parameters: Example and Test

```
import numpy
import cppad_py
def fun_dynamic_xam() :
    ok  = True
    nx  = 2
    nd  = 2
    #
    # value of independent variables during recording
    x = numpy.empty(nx, dtype=float)
```

(continues on next page)

(continued from previous page)

```
    x[0] = 1.0
    x[1] = 2.0
    #
    # value of independent dynamic parameters during recording
    dynamic = numpy.empty(nd, dtype=float)
    dynamic[0] = 3.0
    dynamic[1] = 4.0
    #
    # start recording
    (ax, adynamic) = cppad_py.independent(x, dynamic)
    #
    # create another dynamic paramerer
    adyn  = adynamic[0] + adynamic[1]
    #
    # create another variable
    avar  = ax[0] + ax[1] + adyn
    #
    # create f(x) = x[0] + x[1] + dynamic[0] + dynamic[1]
    ay    = numpy.empty(1, dtype=cppad_py.a_double)
    ay[0] = avar
    f     = cppad_py.d_fun(ax, ay)
    #
    # check some properties of f
    ok = ok and f.size_domain() == nx
    ok = ok and f.size_order()  == 0
    #
    # zero order forward mode using same values as during the recording
    y  = f.forward(0, x)
    ok = ok and y[0] == (x[0] + x[1] + dynamic[0] + dynamic[1])
    #
    #  zero order forward mode using different value for dynamic parameters
    dynamic[0] = dynamic[0] + 1.0
    dynamic[1] = dynamic[1] + 1.0
    f.new_dynamic(dynamic)
    y   = f.forward(0, x)
    ok  = ok and y[0] == (x[0] + x[1] + dynamic[0] + dynamic[1])
    #
    return ok
```

---

xsrst input file: `example/python/core/fun_dynamic_xam.py`

Example

Most of the python `d_fun` examples use this function. The *fun_dynamic_xam_py* uses the syntax that includes dynamic parameters.

---

xsrst input file: `lib/python/cppad_py/independent.py`

py_abort_recording

3.1.1.2 Abort Recording

- *Syntax*
- *Purpose*
- *Example*

Syntax

`cppad_py.abort_recording()`

Purpose

This aborts the current recording (if it exists) started by the most recent call to *independent*.

fun_abort_xam_py

3.1.1.2.1 Python: Abort Recording a_double Operations: Example and Test

```
def fun_abort_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    n_ind = 2
    #
    # create ax
    x = numpy.empty(n_ind, dtype=float)
    for i in range( n_ind  ) :
        x[i] = i + 1.0
    #
    ax = cppad_py.independent(x)
    #
    # preform some a_double operations
    ax0 = ax[0]
    ax1 = ax[1]
    ay = ax0 + ax1
    #
    # check that ay is a variable; its value depends on the value of ax
    ok = ok and ay.variable()
    #
    # abort this recording
    cppad_py.abort_recording()
    #
    # check that ay is now a parameter, no longer a variable.
    ok = ok and ay.parameter()
```

(continues on next page)

(continued from previous page)

```
    #
    # since it is a parameter, we can retrieve its value
    y = ay.value()
    #
    # its value should be x0 + x1
    ok = ok and y  == x[0] + x[1]
    #
    # an abort when not recording has no effect
    cppad_py.abort_recording()
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/fun_abort_xam.py`

Example

python

---

xsrst input file: `lib/python/cppad_py/abort_recording.xsrst`

py_fun_ctor

3.1.1.3 Stop Current Recording and Store Function Object

- ***Syntax***
  - *d_fun*
  - *a_fun*
- *ax*
- *ay*
- ***f***
  - *Empty Function*
- *af*
- *Example*

Syntax

d_fun

f = `cppad_py.d_fun` (ax , ay)
f = `cppad_py.d_fun` ()

a_fun

af = `cppad_py.a_fun(`*f* `)`

ax

This argument must be the same as *ax* returned by the previous call to `independent` ; i.e., it must be the independent variable vector. We use *n* to denote the number of independent variables (the size of *ax*).

ay

This argument is a numpy array with `a_double` elements. It specifies the dependent variables. We use *m* to denote the number of dependent variables (the size of *ay*).

f

This result is a function object that has a representation for the floating point operations that mapped the independent variables *ax* to the dependent variables *ay* . This object computes function and derivative values using `double`

Empty Function

In the case where *ax* and *ay* are not present the function is 'empty' and all its sizes are zero; see *cpp_fun_property*.

af

This result is a function object that has a representation for the same function as *f* . This object computes function and derivative values using `a_double` Initially, there are not Taylor coefficient stored in *af* ; i.e., *af_size_order()* is zero.

a_fun_xam_py

3.1.1.3.1 Python: Purpose of a_fun Objects: Example and Test

```
def a_fun_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    # number of dependent and independent variables
    m = 1
    n = 3
    #
    # Record the function
    #   f(x) = 0.5 * ( x[0]^2 + ... + x[n-1]^2 )
```

(continues on next page)

(continued from previous page)

```
    x = numpy.empty(n, dtype=float)
    for i in range(n) :
        x[i] = i
    ax   = cppad_py.independent(x)
    ay   = numpy.empty(m, dtype=cppad_py.a_double)
    asum = cppad_py.a_double(0.0)
    for i in range(n) :
        asum = asum + ax[i] * ax[i]
    ay[0] = cppad_py.a_double(0.5) * asum
    f     = cppad_py.d_fun(ax, ay)
    af    = cppad_py.a_fun(f)
    #
    # Record the function
    #   g(x) = f'(x) * v
    au   = numpy.empty(m, dtype=cppad_py.a_double)
    av   = numpy.empty(n, dtype=cppad_py.a_double)
    for i in range(n) :
        av[i] = cppad_py.a_double(i)
    ax   = cppad_py.independent(x)
    af.forward(0, ax)
    au = af.forward(1, av)
    g  = cppad_py.d_fun(ax, au)
    #
    # compute g(x)
    u  = g.forward(0, x)
    #
    # compute g'(x)
    uq       = numpy.empty( (m, 1), dtype=float )
    uq[0, 0] = 1.0
    xq       = g.reverse(1, uq)
    #
    # g'(x) = f''(x) * v = v
    for i in range(n) :
        ok = ok and cppad_py.a_double(xq[i, 0]) == av[i]
    return ok
```

---

xsrst input file: `example/python/core/a_fun_xam.py`

Example

All of the examples use the `d_fun` constructor. The example *a_fun_xam_py* demonstrates the purpose of `a_fun` objects.

---

xsrst input file: `lib/python/cppad_py/fun_ctor.py`

py_fun_property

3.1.1.4 Properties of a Function Object

- *Syntax*
- *f*
- *size_domain*
- *size_range*
- *size_var*
- *size_op*
- *size_order*
- *Example*

Syntax

$n = f$. `size_domain` ()
$m = f$. `size_range` ()
$v = f$. `size_var` ()
$p = f$. `size_op` ()
$q = f$. `size_order` ()

f

This is either a *d_fun* or *a_fun* function object and is constant; i.e., not changed.

size_domain

The return value n is an `int` and is the size of the vector *ax* in the function constructor; i.e., the number of independent variables.

size_range

The return value m is an `int` and is the size of the vector *ay* in the function constructor; i.e., the number of dependent variables.

size_var

The return value v is an `int` and is the number of variables in the function. This includes the independent variables, dependent variables, and any variables that are used to compute the dependent variables from the independent variables.

size_op

The return value p is an `int` and is the number of atomic operations that are used to express the dependent variables as a function of the independent variables.

size_order

The return value q is an `int` and is the number of Taylor coefficients currently stored in f , for every variable in the operation sequence corresponding to f . These coefficients are computed by *py_fun_forward*. This is different from the other function properties in that it can change after each call to f . `forward` ; see *size_order* in the forward mode section.

fun_property_xam_py

3.1.1.4.1 Python: d_fun Properties: Example and Test

```
def fun_property_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------------
    n_ind = 1 # number of independent variables
    n_dep = 2 # number of dependent variables
    n_var = 1 # phantom variable at address 0
    n_op  = 1 # special operator at beginning
    #
    # dimension some vectors
    # x  = numpy.empty(n_ind, dtype=float)
    x  = numpy.empty(n_ind, dtype=float)
    ay = numpy.empty(n_dep, dtype=cppad_py.a_double)
    #
    # independent variables
    x[0]  = 1.0
    ax    = cppad_py.independent(x)
    n_var = n_var + n_ind # one for each indpendent
    n_op  = n_op + n_ind
    #
    # first dependent variable
    ay[0] = ax[0] + ax[0]
    n_var = n_var + 1 # one variable and operator
    n_op  = n_op + 1
    #
    # second dependent variable
```

(continues on next page)

(continued from previous page)

```
    ax0   = ax[0]
    ay[1] = ax0.sin()
    n_var = n_var + 2 # two varialbes, one operator
    n_op  = n_op + 1
    #
    # define f(x) = y
    f  = cppad_py.d_fun(ax, ay)
    n_op = n_op + 1 # speical operator at end
    #
    # check af properties except f.to_json
    ok = ok and f.size_domain() == n_ind
    ok = ok and f.size_range()  == n_dep
    ok = ok and f.size_var()    == n_var
    ok = ok and f.size_op()     == n_op
    ok = ok and f.size_order()  == 0
    #
    # compute zero order Taylor coefficients
    y  = f.forward(0, x)
    ok = ok and f.size_order() == 1
    # ------------------------------------------------------------------
    af = cppad_py.a_fun(f)
    #
    # check af properties
    ok = ok and af.size_domain() == n_ind
    ok = ok and af.size_range()  == n_dep
    ok = ok and af.size_var()    == n_var
    ok = ok and af.size_op()     == n_op
    ok = ok and af.size_order()  == 0
    # ------------------------------------------------------------------
    # The empty function
    f = cppad_py.d_fun()
    ok = ok and f.size_domain() == 0
    ok = ok and f.size_range()  == 0
    ok = ok and f.size_var()    == 0
    ok = ok and f.size_op()     == 0
    ok = ok and f.size_order()  == 0
    #
    return( ok  )
#
```

---

xsrst input file: `example/python/core/fun_property_xam.py`

Example

fun_property_xam_py

---

xsrst input file: `lib/python/cppad_py/fun_property.xsrst`

py_fun_new_dynamic

3.1.1.5 New Dynamic Parameters

- *Syntax*
- *f*
- ***dynamic***
  - *size_order*
- *Example*

Syntax

f . `new_dynamic` (*dynamic*)

f

This is either a *d_fun* or *a_fun*. The independent *dynamic* parameters are changed to have the specified values. The other dynamic parameters are then computed.

dynamic

If *f* is a `d_fun` (`a_fun`) object, *dynamic* is a numpy vector with `float` (`a_double`) elements and its size must be the same as the size of *dynamic* in the corresponding call to `independent` . It specifies new values for the dynamic parameters in *f* .

size_order

After this call, *f_size_order()* is zero.

Example

See *fun_dynamic_xam_py*

---

xsrst input file: `lib/python/cppad_py/fun_new_dynamic.py`

py_fun_jacobian

3.1.1.6 Jacobian of an AD Function

- *Syntax*
- *f*
- *f(x)*

- *x*
- *J*
- *Example*

Syntax

J = *f* . `jacobian` (*x*)

f

This is either a *d_fun* or *a_fun* function object. Upon return, the zero order *taylor_coefficients* in *f* correspond to the value of *x* . The other Taylor coefficients in *f* are unspecified.

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

x

If *f* is a `d_fun` or `a_fun` , *x* is a numpy vector with `float` (`a_double`) elements and size *n* . It specifies the argument value at we are computing the Jacobian $f'(x)$.

J

The result is a numpy matrix with `float` (`a_double`) elements, *m* rows, and `n` columns. For *i* between zero and *m* -1 and *j* between zero and *n* -1 ,

$$J[i, j] = \frac{\partial f_i}{\partial x_j}(x)$$

fun_jacobian_xam_py

3.1.1.6.1 Python: Dense Jacobian Using AD: Example and Test

```
def fun_jacobian_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # number of dependent and independent variables
    n_dep = 1
    n_ind = 3
    #
```

(continues on next page)

(continued from previous page)

```
    # create the independent variables ax
    x = numpy.empty(n_ind, dtype=float)
    for i in range( n_ind  ) :
        x[i] = i + 2.0
    #
    ax = cppad_py.independent(x)
    #
    # create dependent variables ay with ay0 = ax_0 * ax_1 * ax_2
    ax_0  = ax[0]
    ax_1  = ax[1]
    ax_2  = ax[2]
    ay    = numpy.empty(n_dep, dtype=cppad_py.a_double)
    ay[0] = ax_0 * ax_1 * ax_2
    #
    # define af corresponding to f(x) = x_0 * x_1 * x_2
    f  = cppad_py.d_fun(ax, ay)
    #
    # compute the Jacobian f'(x) = ( x_1*x_2, x_0*x_2, x_0*x_1 )
    fp = f.jacobian(x)
    #
    # check Jacobian
    x_0 = x[0]
    x_1 = x[1]
    x_2 = x[2]
    ok = ok and fp[0, 0] == x_1 * x_2
    ok = ok and fp[0, 1] == x_0 * x_2
    ok = ok and fp[0, 2] == x_0 * x_1
    # ------------------------------------------------------------------------
    af = cppad_py.a_fun(f)
    #
    ax   = numpy.empty(n_ind, dtype=cppad_py.a_double)
    for i in range( n_ind ) :
        ax[i] = x[i]
    #
    # compute the Jacobian f'(x) = ( x_1*x_2, x_0*x_2, x_0*x_1 )
    afp = af.jacobian(ax)
    #
    # check Jacobian
    ok = ok and afp[0, 0] == cppad_py.a_double(x_1 * x_2)
    ok = ok and afp[0, 1] == cppad_py.a_double(x_0 * x_2)
    ok = ok and afp[0, 2] == cppad_py.a_double(x_0 * x_1)
    #
    return( ok )
#
```

xsrst input file: `example/python/core/fun_jacobian_xam.py`

Example

fun_jacobian_xam_py

---

xsrst input file: `lib/python/cppad_py/fun_jacobian.py`

py_fun_hessian

3.1.1.7 Hessian of an AD Function

- *Syntax*
- *f*
- *f(x)*
- *g(x)*
- *x*
- *w*
- *H*
- *Example*

Syntax

H = *f* . `hessian` (*x* , *w*)

f

This is either a *d_fun* or *a_fun* function object. Upon return, the zero order *taylor_coefficients* in *f* correspond to the value of *x* . The other Taylor coefficients in *f* are unspecified.

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

g(x)

We use the notation $g : \mathbf{R}^n \to \mathbf{R}$ for the function defined by

$$g(x) = \sum_{i=0}^{n-1} w_i f_i(x)$$

x

If f is a `d_fun` or `a_fun` , x is a numpy vector with `float` (`a_double`) elements and size n . It specifies the argument value at we are computing the Hessian $g^{(2)}(x)$.

w

If f is a `d_fun` or `a_fun` , w is a numpy vector with `float` (`a_double`) elements and size m . It specifies the vector w in the definition of $g(x)$ above.

H

The result is a numpy matrix with `float` (`a_double`) elements, n rows and `n` columns. For i between zero and n -1 and j between zero and n -1 ,

$$H[i,j] = \frac{\partial^2 g}{\partial x_i \partial x_j}(x)$$

fun_hessian_xam_py

3.1.1.7.1 Python: Dense Hessian Using AD: Example and Test

```
def fun_hessian_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # number of dependent and independent variables
    n_dep = 1
    n_ind = 3
    #
    # create the independent variables ax
    x = numpy.empty(n_ind, dtype=float)
    for i in range( n_ind  ) :
        x[i] = i + 2.0
    #
    ax = cppad_py.independent(x)
    #
    # create dependent variables ay with ay0 = ax_0 * ax_1 * ax_2
    ax_0 = ax[0]
    ax_1 = ax[1]
    ax_2 = ax[2]
    ay   = numpy.empty(n_dep, dtype=cppad_py.a_double)
    ay[0] = ax_0 * ax_1 * ax_2
    #
    # define af corresponding to f(x) = x_0 * x_1 * x_2
    f  = cppad_py.d_fun(ax, ay)
    #
    # g(x) = w_0 * f_0 (x) = f(x)
```

(continues on next page)

(continued from previous page)

```
    w = numpy.empty(n_dep, dtype=float)
    w[0] = 1.0
    #
    # compute Hessian
    fpp = f.hessian(x, w)
    #
    #           [ 0.0 , x_2 , x_1 ]
    # f''(x) = [ x_2 , 0.0 , x_0 ]
    #           [ x_1 , x_0 , 0.0 ]
    ok = ok and fpp[0, 0] == 0.0
    ok = ok and fpp[0, 1] == x[2]
    ok = ok and fpp[0, 2] == x[1]
    #
    ok = ok and fpp[1, 0] == x[2]
    ok = ok and fpp[1, 1] == 0.0
    ok = ok and fpp[1, 2] == x[0]
    #
    ok = ok and fpp[2, 0] == x[1]
    ok = ok and fpp[2, 1] == x[0]
    ok = ok and fpp[2, 2] == 0.0
    # ---------------------------------------------------------------------
    af = cppad_py.a_fun(f)
    #
    # compute and check Hessian
    aw    = numpy.empty(n_dep, dtype=cppad_py.a_double)
    aw[0] = w[0]
    afpp  = af.hessian(ax, aw)
    ok    = ok and afpp.shape == fpp.shape
    (nr, nc) = fpp.shape
    for i in range(nr) :
        for j in range(nc) :
            ok = ok and afpp[i, j] == cppad_py.a_double( fpp[i, j] )
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/fun_hessian_xam.py`

Example

fun_hessian_xam_py

---

xsrst input file: `lib/python/cppad_py/fun_hessian.py`

py_fun_forward

3.1.1.8 Forward Mode AD

- *Syntax*
- *Taylor Coefficient*
- *f*
- *f(x)*
- *X(t)*
- *Y(t)*
- ***p***
  - *size_order*
- *xp*
- *yp*
- *Example*

Syntax

yp = *f* . `forward` (*p* , *xp*)

Taylor Coefficient

For a function $g(t)$ of a scalar argument $t \in \mathbf{R}$, the *p*-th order Taylor coefficient is its p -th order derivative divided by *p* factorial and evaluated at $t = 0$; i.e.,

$$g^{(p)}(0)/p!$$

f

This is either a *d_fun* or *a_fun* function object. Note that its state is changed by this operation because all the Taylor coefficient that it calculates for every variable in recording are stored. See more discussion of this fact under the heading *p* below.

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

X(t)

We use the notation $X : \mathbf{R} \to \mathbf{R}^n$ for a function that the calling routine chooses.

Y(t)

We define the function $Y : \mathbf{R} \to \mathbf{R}^n$ by $Y(t) = f(X(t))$.

p

This argument has type `int` and is non-negative. Its value is the order of the Taylor coefficient being calculated. If there was no call to `forward` for this *f* , the value of *p* must be zero. Otherwise, it must be between zero and one greater that its value for the previous call using this *f* . After this call, the Taylor coefficients for orders zero though *p* , and for every variable in the recording, will be stored in *f* .

size_order

After this call, *f_size_order()* is *p* +1 .

xp

If *f* is a `d_fun` (`a_fun`) object, *xp* is a numpy vector with `float` (`a_double`) elements and size *n* . It specifies the *p*-th order Taylor coefficients for *X(t*) .

yp

The result is a numpy vector with `float` (`a_double`) elements and size *m* . It is the *p*-th order Taylor coefficients for $Y(t)$.

fun_forward_xam_py

3.1.1.8.1 Python: Forward Mode AD: Example and Test

```
def fun_forward_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # number of dependent and independent variables
    n_dep = 1
    n_ind = 2
    #
    # create the independent variables ax
    xp = numpy.empty(n_ind, dtype=float)
```

(continues on next page)

(continued from previous page)

```
    for i in range( n_ind  ) :
        xp[i] = i + 1.0
    #
    ax = cppad_py.independent(xp)
    #
    # create dependent varialbes ay with ay0 = ax0 * ax1
    ax0    = ax[0]
    ax1    = ax[1]
    ay    = numpy.empty(n_dep, dtype=cppad_py.a_double)
    ay[0] = ax0 * ax1
    #
    # define af corresponding to f(x) = x0 * x1
    f  = cppad_py.d_fun(ax, ay)
    #
    # define X(t) = (3 + t, 2 + t)
    # it follows that Y(t) = f(X(t)) = (3 + t) * (2 + t)
    #
    # Y(0) = 6 and p ! = 1
    p     = 0
    xp[0] = 3.0
    xp[1] = 2.0
    yp = f.forward(p, xp)
    ok = ok and yp[0] == 6.0
    #
    # first order Taylor coefficients for X(t)
    p     = 1
    xp[0] = 1.0
    xp[1] = 1.0
    #
    # first order Taylor coefficient for Y(t)
    # Y'(0) = 3 + 2 = 5 and p ! = 1
    yp = f.forward(p, xp)
    ok = ok and yp[0] == 5.0
    #
    # second order Taylor coefficients for X(t)
    p     = 2
    xp[0] = 0.0
    xp[1] = 0.0
    #
    # second order Taylor coefficient for Y(t)
    # Y''(0) = 2.0 and p ! = 2
    yp = f.forward(p, xp)
    ok = ok and yp[0] == 1.0
    # ----------------------------------------------------------------------
    af = cppad_py.a_fun(f)
    ok = ok and af.size_order() == 0
    #
    # zero order forward
    p   = 0
    axp = numpy.empty(n_ind, dtype=cppad_py.a_double)
    axp[0] = 3.0
    axp[1] = 2.0
    ayp    = af.forward(p, axp)
    ok     = ok and ayp[0] == cppad_py.a_double(6.0)
    ok     = ok and af.size_order() == 1
    #
    return( ok )
```

(continues on next page)

(continued from previous page)

```
#
```

---

xsrst input file: `example/python/core/fun_forward_xam.py`

Example

fun_forward_xam_py

---

xsrst input file: `lib/python/cppad_py/fun_forward.py`

py_fun_reverse

3.1.1.9 Reverse Mode AD

- *Syntax*
- *f*
- ***Notation***
  - *f(x)*
  - *X(t), S*
  - *Y(t), T*
  - *G(T)*
- *q*
- *yq*
- *xq*
- *Example*

Syntax

$xq = f$. `reverse` (q , yq)

f

This is either a *d_fun* or *a_fun* function object and is effectively constant; i.e., not changed.

Notation

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

X(t), S

This is the same function as *x(t)* in the previous call to *f* . `forward` . We use $S \in \mathbf{R}^{n \times q}$ to denote the Taylor coefficients of $X(t)$.

Y(t), T

This is the same function as *y(t)* in the previous call to *f* . `forward` . We use $T \in \mathbf{R}^{m \times q}$ to denote the Taylor coefficients of $Y(t)$. We also use the notation $T(S)$ to express the fact that the Taylor coefficients for $Y(t)$ are a function of the Taylor coefficients of $X(t)$.

G(T)

We use the notation $G : \mathbf{R}^{m \times p} \to \mathbf{R}$ for a function that the calling routine chooses.

q

This argument has type `int` and is positive. It is the number of the Taylor coefficient (for each variable) that we are computing the derivative with respect to. It must be greater than zero, and less than or equal the number of Taylor coefficient stored in *f* ; i.e., *f_size_order()*.

yq

If *f* is a `d_fun` (`a_fun`) object, *yq* is a numpy vector with `float` (`a_double`) elements, *m* rows and *q* columns. For 0 <= *i* < *m* and 0 <= *k* < *q* , *yq* [*i* , *k*] is the partial derivative of $G(T)$ with respect to the *k*-th order Taylor coefficient for the *i*-th component function; i.e., the partial derivative of $G(T)$ w.r.t. $Y_i^{(k)}(t)/k!$.

xq

The result is a numpy vector with `float` (`a_double`) elements, *n* rows and *q* columns. For 0 <= *j* < *n* and 0 <= *k* < *q* , *xq* [*j* , *k*] is the partial derivative of $G(T(S))$ with respect to the *k*-th order Taylor coefficient for the *j*-th component function; i.e., the partial derivative of $G(T(S))$ w.r.t. $S_j^{(k)}(t)/k!$.

fun_reverse_xam_py

3.1.1.9.1 Python: Reverse Mode AD: Example and Test

```
def fun_reverse_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    # number of dependent and independent variables
    n_dep = 1
    n_ind = 3
    #
    # create the independent variables ax
    xp = numpy.empty(n_ind, dtype=float)
    for i in range( n_ind  ) :
        xp[i] = i
    #
    ax = cppad_py.independent(xp)
    #
    # create dependent variables ay with ay0 = ax_0 * ax_1 * ax_2
    ax_0  = ax[0]
    ax_1  = ax[1]
    ax_2  = ax[2]
    ay    = numpy.empty(n_dep, dtype=cppad_py.a_double)
    ay[0] = ax_0 * ax_1 * ax_2
    #
    # define af corresponding to f(x) = x_0 * x_1 * x_2
    f  = cppad_py.d_fun(ax, ay)
    # ----------------------------------------------------------------------
    # define          X(t) = (x_0 + t, x_1 + t, x_2 + t)
    # it follows that Y(t) = f(X(t)) = (x_0 + t) * (x_1 + t) * (x_2 + t)
    # and that       Y'(0) = x_1 * x_2 + x_0 * x_2 + x_0 * x_1
    # ----------------------------------------------------------------------
    # zero order forward mode
    p     = 0
    xp[0] = 2.0
    xp[1] = 3.0
    xp[2] = 4.0
    yp = f.forward(p, xp)
    ok = ok and yp[0] == 24.0
    # ----------------------------------------------------------------------
    # first order reverse (derivative of zero order forward)
    # define G( Y ) = y_0 = x_0 * x_1 * x_2
    m         = f.size_range()
    q         = 1
    yq1       = numpy.empty( (m, q), dtype=float)
    yq1[0, 0] = 1.0
    xq1       = f.reverse(q, yq1)
    # partial G w.r.t x_0
    ok = ok and xq1[0,0] == 3.0 * 4.0
    # partial G w.r.t x_1
    ok = ok and xq1[1,0] == 2.0 * 4.0
    # partial G w.r.t x_2
```

(continues on next page)

(continued from previous page)

```
    ok = ok and xq1[2,0] == 2.0 * 3.0
    # ----------------------------------------------------------------------
    # first order forward mode
    p     = 1
    xp[0] = 1.0
    xp[1] = 1.0
    xp[2] = 1.0
    yp    = f.forward(p, xp)
    ok    = ok and yp[0] == 3.0*4.0 + 2.0*4.0 + 2.0*3.0
    # ----------------------------------------------------------------------
    # second order reverse (derivative of first order forward)
    # define G( y_0^0 , y_0^1 ) = y_0^1
    # = x_1^0 * x_2^0  +  x_0^0 * x_2^0  +  x_0^0  *  x_1^0
    q         = 2
    yq2       = numpy.empty( (m, q), dtype=float)
    yq2[0, 0] = 0.0 # partial of G w.r.t y_0^0
    yq2[0, 1] = 1.0 # partial of G w.r.t y_0^1
    xq2       = f.reverse(q, yq2)
    # partial G w.r.t x_0^0
    ok = ok and xq2[0, 0] == 3.0 + 4.0
    # partial G w.r.t x_1^0
    ok = ok and xq2[1, 0] == 2.0 + 4.0
    # partial G w.r.t x_2^0
    ok = ok and xq2[2, 0] == 2.0 + 3.0
    # ----------------------------------------------------------------------
    af = cppad_py.a_fun(f)
    #
    # zero order forward
    axp   = numpy.empty(n_ind, dtype=cppad_py.a_double)
    p     = 0
    axp[0] = 2.0
    axp[1] = 3.0
    axp[2] = 4.0
    ayp = af.forward(p, axp)
    ok = ok and ayp[0] == cppad_py.a_double(24.0)
    #
    # first order reverse
    q          = 1
    ayq1       = numpy.empty( (m, q), dtype=cppad_py.a_double)
    ayq1[0, 0] = 1.0
    axq1       = af.reverse(q, ayq1)
    # partial G w.r.t x_0
    ok = ok and axq1[0,0] == cppad_py.a_double(3.0 * 4.0)
    # partial G w.r.t x_1
    ok = ok and axq1[1,0] == cppad_py.a_double(2.0 * 4.0)
    # partial G w.r.t x_2
    ok = ok and axq1[2,0] == cppad_py.a_double(2.0 * 3.0)
    #
    return( ok )
#
```

xsrst input file: `example/python/core/fun_reverse_xam.py`

Example

fun_reverse_xam_py

---

xsrst input file: `lib/python/cppad_py/fun_reverse.py`

py_fun_optimize

3.1.1.10 Optimize an AD Function

- *Syntax*
- *Purpose*
- *f*
- *Example*

Syntax

f `.optimize()`

Purpose

This reduces the number of operations (hence to time and memory) used to compute the function stored in *f* On the other hand, the optimization may take a significant amount of time and memory.

f

This is a *d_fun*. Optimizing this *f* also optimizes the corresponding *a_fun*.

fun_optimize_xam_py

3.1.1.10.1 Python: Optimize an d_fun: Example and Test

```
def fun_optimize_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    n_ind = 1 # number of independent variables
    n_dep = 1 # number of dependent variables
    n_var = 1 # phantom variable at address 0
    n_op  = 1 # special operator at beginning
    #
```

(continues on next page)

(continued from previous page)

```
    # dimension some vectors
    x  = numpy.empty(n_ind, dtype=float)
    ay = numpy.empty(n_dep, dtype=cppad_py.a_double)
    #
    # independent variables
    x[0]  = 1.0
    ax    = cppad_py.independent(x)
    n_var = n_var + n_ind # one for each indpendent
    n_op  = n_op + n_ind
    #
    # accumulate summation
    ax0   = ax[0]
    csum  = cppad_py.a_double(0.0)
    csum  = ax0 + ax0 + ax0 + ax0
    n_var = n_var + 3 # one per + operator
    n_op  = n_op + 3
    #
    # define f(x) = y_0 = csum
    ay[0] = csum
    f     = cppad_py.d_fun(ax, ay)
    n_op  = n_op + 1 # speical operator at end
    #
    # check number of variables and operators
    ok = ok and f.size_var() == n_var
    ok = ok and f.size_op() == n_op
    #
    # optimize
    f.optimize()
    #
    # number of variables and operators has decreased by two
    ok = ok and f.size_var() == n_var-2
    ok = ok and f.size_op() == n_op-2
    #
    return( ok  )
#
```

---

xsrst input file: `example/python/core/fun_optimize_xam.py`

Example

fun_optimize_xam_py

---

xsrst input file: `lib/python/cppad_py/fun_optimize.xsrst`

py_fun_json

3.1.1.11 Json Representation of AD Computation Graph

- *Syntax*
- *f*
- *json*
- *to_json*
- *from_json*
- *Examples*

Syntax

json = *f* . `to_json` ()
f . `from_json(json)`

f

This is a *d_fun* function object.

json

is a `str` containing a Json representation of the computation graph corresponding to *f* ; see the CppAD documentation for json_ad_graph.

to_json

In this case, the function object *f* is constant and the return value *json* is created.

from_json

In this case, the argument *json* is constant and the function *f* is changed so it corresponds to *json* .

fun_to_json_xam_py

3.1.1.11.1 Python to_json: Example and Test

```
def to_json_xam() :
    #
    import numpy
    import cppad_py
    import re
```

(continues on next page)

(continued from previous page)

```
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    n = 1 # number of independent variables
    m = 2 # number of dependent variables
    #
    # independent variables
    x  = numpy.array( [ 1.0 ] )
    ax = cppad_py.independent(x)
    #
    # f(x) = [ x0 + x0, sin(x0) ]
    ay = numpy.empty(m, dtype=cppad_py.a_double)
    ay[0] = ax[0] + ax[0]
    ay[1] = ax[0].sin()
    f     = cppad_py.d_fun(ax, ay)
    #
    # check f.to_json
    json     = f.to_json()
    pattern  = r'"op_code" *: *([^,]*),'
    match    = re.search(pattern, json)
    op_code  = int( match.group(1) )
    ok      &= op_code == 1
    pattern  = r'"name" *: *"([^"]*)" *,'
    match    = re.search(pattern, json)
    name     = match.group(1);
    ok      &= name == 'add' or name == 'sub'
    #
    return ok
```

xsrst input file: `example/python/core/fun_json_xam.py`

fun_from_json_xam_py

3.1.1.11.2 Python from_json: Example and Test

```
def from_json_xam() :
    #
    import numpy
    import cppad_py
    import math
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # AD graph repersentation of f(x) = sin(x) / cos(x)
    #
    # node_1 : x[0]
    # node_2 : sin( x[0] )
    # node_3 : cos( x[0] )
    # node_4 : sin( x[0] ) / cos( x[0] )
    # y[0]   = sin( x[0] ) / cos( x[0] )
    json = '''
```

(continues on next page)

(continued from previous page)

```
        {
            "function_name" : "tangent function",
            "op_define_vec" : [ 3, [
                { "op_code":1, "name":"sin", "n_arg":1 } ,
                { "op_code":2, "name":"cos", "n_arg":1 } ,
                { "op_code":3, "name":"div", "n_arg":2 } ]
            ],
            "n_dynamic_ind"  : 0,
            "n_variable_ind" : 1,
            "constant_vec"   : [ 0, [ ] ],
            "op_usage_vec"   : [ 3, [
                [ 1, 1 ]   ,
                [ 2, 1 ]   ,
                [ 3, 2, 3] ]
            ],
            "dependent_vec" : [ 1, [4] ]
        };
    '''
    # convert json to a fucntion object
    f = cppad_py.d_fun();
    f.from_json(json);
    #
    # compute y = f(x)
    x  = numpy.array( [ 1.0 ] );
    y  = f.forward(0, x);
    #
    # check the function value
    eps99     = 99.0 * numpy.finfo(float).eps
    check     = math.tan(x[0]);
    rel_error = y[0] / check - 1.0;
    ok       &= abs( rel_error ) < eps99;
    #
    return ok
```

---

xsrst input file: `example/python/core/fun_json_xam.py`

Examples

fun_to_json_xam_py, *fun_from_json_xam_py*.

---

xsrst input file: `lib/python/cppad_py/fun_json.xsrst`

1. py_independent: *3.1.1.1 Declare Independent Variables and Start Recording*
2. py_abort_recording: *3.1.1.2 Abort Recording*
3. py_fun_ctor: *3.1.1.3 Stop Current Recording and Store Function Object*
4. py_fun_property: *3.1.1.4 Properties of a Function Object*
5. py_fun_new_dynamic: *3.1.1.5 New Dynamic Parameters*
6. py_fun_jacobian: *3.1.1.6 Jacobian of an AD Function*
7. py_fun_hessian: *3.1.1.7 Hessian of an AD Function*

8. py_fun_forward: *3.1.1.8 Forward Mode AD*
9. py_fun_reverse: *3.1.1.9 Reverse Mode AD*
10. py_fun_optimize: *3.1.1.10 Optimize an AD Function*
11. py_fun_json: *3.1.1.11 Json Representation of AD Computation Graph*

---

xsrst input file: `lib/python/cppad_py/fun.py`

py_sparse

3.1.2 Python Sparsity Routines

- *Children*

Children

py_sparse_rc

3.1.2.1 Sparsity Patterns

- *Syntax*
- *pattern*
- *nr*
- *nc*
- *nnz*
- *resize*
- ***put***
  - *k*
  - *r*
  - *c*
- *row*
- *col*
- *row_major*
- *col_major*
- *Example*

Syntax

pattern = `cppad_py.sparse_rc`()
pattern . `resize` (*nr* , *nc* , *nnz*)
nr = *pattern* . `nr` ()
nc = *pattern* . `nc` ()
nnz = *pattern* . `nnz` ()
pattern . `put` (*k* , *r* , *c*)
row = *pattern* . `row` ()
col = *pattern* . `col` ()
row_major = *pattern* . `row_major` ()
col_major = *pattern* . `col_major` ()

pattern

This result is used to hold a sparsity pattern for a matrix. It is constant except during the `resize` and `put` operations.

nr

This argument is a non-negative `int` and is the number of rows in the sparsity pattern. The function `nr()` returns the value of *nr* in the previous `resize` operation.

nc

This argument is a non-negative `int` and is the number of columns in the sparsity pattern. The function `nc()` returns the value of *nc* in the previous `resize` operation.

nnz

This argument is a non-negative `int` and is the number of possibly non-zero index pairs in the sparsity pattern. The function `nnz()` returns the value of *nnz* in the previous `resize` operation.

resize

The current sparsity pattern is lost and a new one is started with the specified parameters. After each `resize` , the elements in the *row* and *col* vectors should be assigned using `put` .

put

This function sets the values

$$row[k] = r$$
$$col[k] = c$$

(The name `set` is used by Cppad, but not used here, because `set` it is a built-in name in Python.)

k

This argument is a non-negative `int` and must be less than *nnz* .

r

This argument is a non-negative `int` and must be less than *nr* . It specifies the value assigned to *row* [*k*] .

c

This argument is a non-negative `int` and must be less than *nc* . It specifies the value assigned to *col* [*k*] .

row

This result is a numpy vector with `int` elements and its size is *nnz* . For $k = 0, \ldots, nnz - 1$, *row* [*k*] is the row index for the *k*-th possibly non-zero entry in the matrix.

col

This result is a numpy vector with `int` elements and its size is *nnz* . For $k = 0, \ldots, nnz - 1$, *col* [*k*] is the column index for the *k*-th possibly non-zero entry in the matrix.

row_major

This result is a numpy vector with `int` elements and its size *nnz* . It sorts the sparsity pattern in row-major order. To be specific,

$$col[row_major[k]] <= col[row_major[k+1]]$$

and if $col[row_major[k]] == col[row_major[k+1]]$,

$$row[row_major[k]] < row[row_major[k+1]]$$

This routine generates an assert if there are two entries with the same row and column values (if NDEBUG is not defined).

col_major

This result is a numpy vector with int elements and its size *nnz* . It sorts the sparsity pattern in column-major order. To be specific,

$$row\ [\ col_major\ [\ k\ ]\ ] <= row\ [\ col_major\ [\ k+1]\ ]$$

and if $row\ [\ col_major\ [\ k\ ]\ ] == row\ [\ col_major\ [\ k+1]\ ]$,

$$col\ [\ col_major\ [\ k\ ]\ ] < col\ [\ col_major\ [\ k+1]\ ]$$

This routine generates an assert if there are two entries with the same row and column values (if NDEBUG is not defined).

sparse_rc_xam_py

3.1.2.1.1 Python: Sparsity Patterns: Example and Test

```
def sparse_rc_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    #
    # create a sparsity pattern
    pattern = cppad_py.sparse_rc()
    #
    nr = 6
    nc = 5
    nnz = 4
    #
    # resize
    pattern.resize(nr, nc, nnz)
    #
    ok = ok and pattern.nr()  == nr
    ok = ok and pattern.nc()  == nc
    ok = ok and pattern.nnz() == nnz
    #
    # indices corresponding to upper-diagonal
    for k in range( nnz ) :
        pattern.put(k, k, k+1)
    #
    #
```

(continues on next page)

(continued from previous page)

```
    # row and column indices
    row = pattern.row()
    col = pattern.col()
    #
    # check row and column indices
    for k in range( nnz ) :
        ok = ok and row[k] == k
        ok = ok and col[k] == k+1
    #
    #
    # For this case, row_major and col_major order are same as original order
    row_major = pattern.row_major()
    col_major = pattern.col_major()
    for k in range( nnz ) :
        ok = ok and row_major[k] == k
        ok = ok and col_major[k] == k
    #
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/sparse_rc_xam.py`

Example

sparse_rc_xam_py

---

xsrst input file: `lib/python/cppad_py/sparse_rc.py`

py_sparse_rcv

3.1.2.2 Sparse Matrices

- *Syntax*
- *pattern*
- *matrix*
- *nr*
- *nc*
- *nnz*
- ***put***
    - *k*
    - *v*
- *row*
- *col*

- *val*
- *row_major*
- *col_major*
- *Example*

Syntax

matrix = `cppad_py::sparse_rcv` (*pattern*)
nr = *matrix* . `nr` ()
nc = *matrix* . `nc` ()
nnz = `matrix` . *nnz* ()
matrix . `put` (*k* , *v*)
row = *matrix* . `row` ()
col = *matrix* . `col` ()
val = *matrix* . `val` ()
row_major = *matrix* . `row_major` ()
col_major = *matrix* . `col_major` ()

pattern

This argument has prototype

```
const sparse_rc& pattern
```

It specifies the number of rows, number of columns and the possibly non-zero entries in the *matrix* .

matrix

This is a sparse matrix object with the sparsity specified by *pattern* . Only the *val* vector can be changed. All other values returned by *matrix* are fixed during the constructor and constant there after. The *val* vector is only changed by the constructor and the `set` function.

nr

This return value is an `int` and is the number of rows in the matrix.

nc

This return value is an `int` and is the number of columns in the matrix.

nnz

This return value is an `int` and is the number of possibly non-zero values in the matrix.

put

This function sets the value

$$val\ [\ k\ ] = v$$

(The name `set` is used by Cppad, but not used here, because `set` it is a built-in name in Python.)

k

This is a non-negative `int` and must be less than nnz .

v

This argument has type `float` and specifies the value assigned to $val\ [\ k\ ]$.

row

This result is a numpy vector with `int` elements and its size is nnz . For $k = 0, \ldots , nnz - 1$, $row\ [\ k\ ]$ is the row index for the k-th possibly non-zero entry in the matrix.

col

This result is a numpy vector with `int` elements and its size is nnz . For $k = 0, \ldots , nnz - 1$, $col\ [\ k\ ]$ is the column index for the k-th possibly non-zero entry in the matrix.

val

This result is a numpy vector with `float` elements and its size is nnz . For $k = 0, \ldots , nnz - 1$, $val\ [\ k\ ]$ is the value of the k-th possibly non-zero entry in the matrix (the value may be zero).

row_major

This result is a numpy vector with `int` elements and its size *nnz* . It sorts the sparsity pattern in row-major order. To be specific,

$$col\ [\ row_major\ [\ k\ ]\ ] <= col\ [\ row_major\ [\ k+1]\ ]$$

and if $col\ [\ row_major\ [\ k\ ]\ ] == col\ [\ row_major\ [\ k+1]\ ]$,

$$row\ [\ row_major\ [\ k\ ]\ ] < row\ [\ row_major\ [\ k+1]\ ]$$

This routine generates an assert if there are two entries with the same row and column values (if `NDEBUG` is not defined).

col_major

This result is a numpy vector with `int` elements and its size *nnz* . It sorts the sparsity pattern in column-major order. To be specific,

$$row\ [\ col_major\ [\ k\ ]\ ] <= row\ [\ col_major\ [\ k+1]\ ]$$

and if $row\ [\ col_major\ [\ k\ ]\ ] == row\ [\ col_major\ [\ k+1]\ ]$,

$$col\ [\ col_major\ [\ k\ ]\ ] < col\ [\ col_major\ [\ k+1]\ ]$$

This routine generates an assert if there are two entries with the same row and column values (if `NDEBUG` is not defined).

sparse_rcv_xam_py

3.1.2.2.1 Python: Sparsity Patterns: Example and Test

```
def sparse_rcv_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    #
    # create sparsity pattern for n by n identity matrix
    pattern = cppad_py.sparse_rc()
```

(continues on next page)

(continued from previous page)

```
    n = 5
    pattern.resize(n, n, n)
    for k in range( n ) :
        pattern.put(k, k, k)
    #
    #
    # create n by n sparse representation of identity matrix
    # (temporarly use pattern.rc untile sparse_rcv wrapper is built)
    matrix = cppad_py.sparse_rcv(pattern)
    for k in range( n ) :
        matrix.put(k, 1.0)
    #
    #
    # row, column indices
    row = matrix.row()
    col = matrix.col()
    val = matrix.val()
    #
    # check results
    for k in range( n ) :
        ok = ok and row[k] == k
        ok = ok and col[k] == k
        ok = ok and val[k] == 1.0
    #
    #
    # For this case, row_major and col_major order are same as original order
    row_major = matrix.row_major()
    col_major = matrix.col_major()
    for k in range( n ) :
        ok = ok and row_major[k] == k
        ok = ok and col_major[k] == k
    #
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/sparse_rcv_xam.py`

Example

sparse_rcv_xam_py

---

xsrst input file: `lib/python/cppad_py/sparse_rcv.py`

py_jac_sparsity

3.1.2.3 Jacobian Sparsity Patterns

- *Syntax*
- ***Purpose***
  - *for_jac_sparsity*
  - *rev_jac_sparsity*
- *x*
- *f*
- *pattern_in*
- *pattern_out*
- *Sparsity for Entire Jacobian*
- *Example*

Syntax

f . `for_jac_sparsity` (*pattern_in* , *pattern_out*)
f . `rev_jac_sparsity` (*pattern_in* , *pattern_out*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to the operation sequence stored in *f* .

for_jac_sparsity

Fix $R \in \mathbf{R}^{n \times \ell}$ and define the function

$$J(x) = F^{(1)}(x) * R$$

Given a sparsity pattern for R, `for_jac_sparsity` computes a sparsity pattern for $J(x)$.

rev_jac_sparsity

Fix $R \in \mathbf{R}^{\ell \times m}$ and define the function

$$J(x) = R * F^{(1)}(x)$$

Given a sparsity pattern for R, `rev_jac_sparsity` computes a sparsity pattern for $J(x)$.

x

Note that a sparsity pattern for $J(x)$ corresponds to the operation sequence stored in f and does not depend on the argument x .

f

This object must have been returned by a previous call to the python *d_fun* constructor. The object f is not constant when using `for_jac_sparsity` . After a call to `for_jac_sparsity` , a sparsity pattern for each of the variables in the operation sequence is held in f for possible later use during reverse Hessian sparsity calculations.

pattern_in

This argument must have be a *pattern* returned by the `sparse_rc` constructor. It specifies the sparsity pattern for R.

pattern_out

This argument must have be a *pattern* returned by the `sparse_rc` constructor. This input value of *pattern_out* does not matter. Upon return *pattern_out* is a sparsity pattern for $J(x)$.

Sparsity for Entire Jacobian

Suppose that R is the identity matrix. In this case, *pattern_out* is a sparsity pattern for $F^{(1)}(x)$.

sparse_jac_pattern_xam_py

3.1.2.3.1 Python: Jacobian Sparsity Patterns: Example and Test

```
def sparse_jac_pattern_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # number of dependent and independent variables
    n = 3
    #
    # create the independent variables ax
    x = numpy.empty(n, dtype=float)
    for i in range( n  ) :
        x[i] = i + 2.0
    #
    ax = cppad_py.independent(x)
    #
    # create dependent variables ay with ay[i] = ax[j]
    # where i = mod(j + 1, n)
    ay = numpy.empty(n, dtype=cppad_py.a_double)
```

(continues on next page)

(continued from previous page)

```
    for j in range( n  ) :
        i = j+1
        if i >= n  :
            i = i - n
        #
        ay_i = ax[j]
        ay[i] = ay_i
    #
    #
    # define af corresponding to f(x)
    f  = cppad_py.d_fun(ax, ay)
    #
    # sparsity pattern for identity matrix
    pat_in = cppad_py.sparse_rc()
    pat_in.resize(n, n, n)
    for k in range( n ) :
        pat_in.put(k, k, k)
    #
    #
    # loop over forward and reverse mode
    for mode in range( 2 ) :
        pat_out = cppad_py.sparse_rc()
        if mode == 0  :
            f.for_jac_sparsity(pat_in, pat_out)
        #
        if mode == 1  :
            f.rev_jac_sparsity(pat_in, pat_out)
        #
        #
        # check that result is sparsity pattern for Jacobian
        ok = ok and n == pat_out.nnz()
        col_major = pat_out.col_major()
        row = pat_out.row()
        col = pat_out.col()
        for k in range( n ) :
            ell = col_major[k]
            r = row[ell]
            c = col[ell]
            i = c+1
            if i >=  n  :
                i = i - n
            #
            ok = ok and c == k
            ok = ok and r == i
        #
    #
    #
    return( ok )
#
```

xsrst input file: `example/python/core/sparse_jac_pattern_xam.py`

Example

python

---

xsrst input file: `lib/python/cppad_py/jac_sparsity.xsrst`

py_hes_sparsity

3.1.2.4 Hessian Sparsity Patterns

- *Syntax*
- *Purpose*
- *x*
- *f*
- *select_domain*
- *select_range*
- *pattern_out*
- *Sparsity for Component Wise Hessian*
- *Example*

Syntax

f . `for_hes_sparsity` (*select_domain* , *select_range* , *pattern_out*)

f . `rev_hes_sparsity` (*select_domain* , *select_range* , *pattern_out*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to the operation sequence stored in *f* . Fix a diagonal matrix $D \in \mathbf{R}^{n \times n}$, fix a vector $r \in \mathbf{R}^m$, and define

$$H(x) = D(r^{\mathrm{T}} F)^{(2)}(x) D$$

Given a sparsity pattern for D and r, these routines compute a sparsity pattern for $H(x)$.

x

Note that a sparsity pattern for $H(x)$ corresponds to the operation sequence stored in *f* and does not depend on the argument *x* .

f

This object must have been returned by a previous call to the python *d_fun* constructor.

select_domain

The argument is a numpy vector with `bool` elements. It has size *n* and is a sparsity pattern for the diagonal of D; i.e., *select_domain* [*j*] is true if and only if $D_{j,j}$ is possibly non-zero.

select_range

The argument is a numpy vector with `bool` elements. It has size *m* and is a sparsity pattern for the vector r; i.e., *select_range* [*i*] is true if and only if r_i is possibly non-zero.

pattern_out

This argument must have be a *pattern* returned by the `sparse_rc` constructor. This input value of *pattern_out* does not matter. Upon return *pattern_out* is a sparsity pattern for $H(x)$.

Sparsity for Component Wise Hessian

Suppose that D is the identity matrix, and only the *i*-th component of *r* is possibly non-zero. In this case, *pattern_out* is a sparsity pattern for $F_i^{(2)}(x)$.

sparse_hes_pattern_xam_py

3.1.2.4.1 Python: Hessian Sparsity Patterns: Example and Test

```
def sparse_hes_pattern_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # number of dependent and independent variables
    n = 3
    #
    # create the independent variables ax
    x = numpy.empty(n, dtype=float)
    for i in range( n  ) :
        x[i] = i + 2.0
    #
    ax = cppad_py.independent(x)
    #
    # create dependent variables ay with ay[i] = ax[j] * ax[i]
    # where i = mod(j + 1, n)
    ay = numpy.empty(n, dtype=cppad_py.a_double)
```

(continues on next page)

(continued from previous page)

```
    for j in range( n  ) :
        i = j+1
        if i >= n  :
            i = i - n
        #
        ay_i = ax[i] * ax[j]
        ay[i] = ay_i
    #
    #
    # define af corresponding to f(x)
    f  = cppad_py.d_fun(ax, ay)
    #
    # Set select_d (domain) to all true, initial select_r (range) to all false
    select_d = numpy.empty(n, dtype=bool)
    select_r = numpy.empty(n, dtype=bool)
    for i in range( n ) :
        select_d[i] = True
        select_r[i] = False
    #
    #
    # only select component 0 of the range function
    # f_0 (x) = x_0 * x_{n-1}
    select_r[0] = True
    #
    # loop over forward and reverse mode
    for mode in range( 2 ) :
        pat_out = cppad_py.sparse_rc()
        if mode == 0  :
            f.for_hes_sparsity(select_d, select_r, pat_out)
        #
        if mode == 1  :
            f.rev_hes_sparsity(select_d, select_r, pat_out)
        #
        #
        # check that result is sparsity pattern for Hessian of f_0 (x)
        ok = ok and pat_out.nnz() == 2
        row = pat_out.row()
        col = pat_out.col()
        for k in range( 2 ) :
            r = row[k]
            c = col[k]
            if r <= c  :
                ok = ok and r == 0
                ok = ok and c == n-1
            #
            if r >= c  :
                ok = ok and r == n-1
                ok = ok and c == 0
            #
        #
    #
    #
    return( ok )
#
```

xsrst input file: `example/python/core/sparse_hes_pattern_xam.py`

Example

python

---

xsrst input file: `lib/python/cppad_py/hes_sparsity.py`

py_sparse_jac

3.1.2.5 Computing Sparse Jacobians

- *Syntax*
- *Purpose*
- *sparse_jac_for*
- *sparse_jac_rev*
- *f*
- *subset*
- *x*
- *pattern*
- *work*
- *n_sweep*
- *Uses Forward*
- *Example*

Syntax

work = `cppad_py.sparse_jac_work` ()
n_sweep = *f* . `sparse_jac_for` (*subset* , *x* , *pattern* , *work*)

n_sweep = *f* . `sparse_jac_rev` (*subset* , *x* , *pattern* , *work*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to *f* . The syntax above takes advantage of sparsity when computing the Jacobian

$$J(x) = F^{(1)}(x)$$

In the sparse case, this should be faster and take less memory than *py_fun_jacobian*. We use the notation $J_{i,j}(x)$ to denote the partial of $F_i(x)$ with respect to x_j.

sparse_jac_for

This function uses first order forward mode sweeps *py_fun_forward* to compute multiple columns of the Jacobian at the same time.

sparse_jac_rev

This function uses first order reverse mode sweeps *py_fun_reverse* to compute multiple rows of the Jacobian at the same time.

f

This object must have been returned by a previous call to the python *d_fun* constructor. Note that the Taylor coefficients stored in *f* are affected by this operation; see *uses_forward* below.

subset

This argument must have be a *matrix* returned by the `sparse_rcv` constructor. Its row size is *subset* . `nr` () == *m* , and its column size is *subset* . `nc` () == *n* . It specifies which elements of the Jacobian are computed. The input value of its value vector *subset* . `val` () does not matter. Upon return it contains the value of the corresponding elements of the Jacobian. All of the row, column pairs in *subset* must also appear in *pattern* ; i.e., they must be possibly non-zero.

x

This argument is a numpy vector with `float` elements and size *n* . It specifies the point at which to evaluate the Jacobian $J(x)$.

pattern

This argument must have be a *pattern* returned by the `sparse_rc` constructor. Its row size is *pattern* . `nr` () == *m* , and its column size is *pattern* . `nc` () == *n* . It is a sparsity pattern for the Jacobian $J(x)$. This argument is not used (and need not satisfy any conditions), when *work* is non-empty.

work

This argument must have been constructed by the call

work = `cppad_py.sparse_jac_work` ()

We refer to its initial value, and its value after *work* . `clear` () , as empty. If it is empty, information is stored in *work* . This can be used to reduce computation when a future call is for the same object *f* , the same member function `sparse_jac_for` or `sparse_jac_rev` , and the same subset of the Jacobian. If any of these values change, use *work* . `clear` () to empty this structure.

n_sweep

This return value is and `int` . If `sparse_jac_for` (`sparse_jac_rev`) is used, *n_sweep* is the number of first order forward (reverse) sweeps used to compute the requested Jacobian values. This is proportional to the total computational work, not counting the zero order forward sweep, or combining multiple columns (rows) into a single sweep.

Uses Forward

After each call to *py_fun_forward*, the object f contains the corresponding Taylor coefficients for all the variables in the operation sequence.. After a call to `sparse_jac_forward` or `sparse_jac_rev` , the zero order coefficients correspond to

f `.forward(0,`x`)`

All the other forward mode coefficients are unspecified.

sparse_jac_xam_py

3.1.2.5.1 Python: Computing Sparse Jacobians: Example and Test

```
def sparse_jac_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    # number of dependent and independent variables
    n = 3
    # one
    aone = cppad_py.a_double(1.0)
    #
    # create the independent variables ax
    x = numpy.empty(n, dtype=float)
    for i in range( n  ) :
        x[i] = i + 2.0
    #
    ax = cppad_py.independent(x)
    #
    # create dependent variables ay with ay[i] = (j+1) * ax[j]
    # where i = mod(j + 1, n)
    ay = numpy.empty(n, dtype=cppad_py.a_double)
    for j in range( n  ) :
        i = j+1
        if i >= n  :
            i = i - n
        #
        aj = cppad_py.a_double(j)
        ay_i = (aj + aone) * ax[j]
```

(continues on next page)

(continued from previous page)

```
        ay[i] = ay_i
    #
    #
    # define af corresponding to f(x)
    f  = cppad_py.d_fun(ax, ay)
    #
    # sparsity pattern for identity matrix
    pat_eye = cppad_py.sparse_rc()
    pat_eye.resize(n, n, n)
    for k in range( n ) :
        pat_eye.put(k, k, k)
    #
    #
    # sparsity pattern for the Jacobian
    pat_jac = cppad_py.sparse_rc()
    f.for_jac_sparsity(pat_eye, pat_jac)
    #
    # loop over forward and reverse mode
    for mode in range( 2 ) :
        # compute all possibly non-zero entries in Jacobian
        subset = cppad_py.sparse_rcv(pat_jac)
        # work space used to save time for multiple calls
        work = cppad_py.sparse_jac_work()
        if mode == 0  :
            f.sparse_jac_for(subset, x, pat_jac, work)
        #
        if mode == 1  :
            f.sparse_jac_rev(subset, x, pat_jac, work)
        #
        #
        # check result
        ok = ok and n == subset.nnz()
        col_major = subset.col_major()
        row = subset.row()
        col = subset.col()
        val = subset.val()
        for k in range( n ) :
            ell = col_major[k]
            r = row[ell]
            c = col[ell]
            v = val[ell]
            i = c+1
            if i >=  n  :
                i = i - n
            #
            ok = ok and c == k
            ok = ok and r == i
            ok = ok and v == c + 1.0
        #
    #
    #
    return( ok )
#
```

xsrst input file: `example/python/core/sparse_jac_xam.py`

Example

sparse_jac_xam_py

---

xsrst input file: `lib/python/cppad_py/sparse_jac.py`

py_sparse_hes

3.1.2.6 Computing Sparse Hessians

- *Syntax*
- *Purpose*
- *f*
- *subset*
- *x*
- *r*
- *pattern*
- *work*
- *n_sweep*
- *Uses Forward*
- *Example*

Syntax

work = `cppad_py.sparse_hes_work` ()
n_sweep = *f* . `sparse_hes` (*subset* , *x* , *r* , *pattern* , *work*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to *f* . Given a vector $r \in \mathbf{R}^m$, define

$$H(x) = (r^{\mathrm{T}} F)^{(2)}(x)$$

This routine takes advantage of sparsity when computing elements of the Hessian $H(x)$.

f

This object must have been returned by a previous call to the python *d_fun* constructor. Note that the Taylor coefficients stored in *f* are affected by this operation; see *uses_forward* below.

subset

This argument must have be a *matrix* returned by the `sparse_rcv` constructor. Its row size and column size is *n* ; i.e., *subset* . `nr` () == *n* and *subset* . `nc` () == *n* . It specifies which elements of the Hessian are computed. The input value of its value vector *subset* . `val` () does not matter. Upon return it contains the value of the corresponding elements of the Jacobian. All of the row, column pairs in *subset* must also appear in *pattern* ; i.e., they must be possibly non-zero.

x

This argument is a numpy vector with `float` elements and size *n* . It specifies the point at which to evaluate the Hessian $H(x)$.

r

This argument is a numpy vector with `float` elements and size *m* . It specifies the multiplier for each component of $F(x)$; i.e., r_i is the multiplier for $F_i(x)$.

pattern

This argument must have be a *pattern* returned by the `sparse_rc` constructor. Its row size and column sizes are *n* ; i.e., *pattern* . `nr` () == *n* and *pattern* . `nc` () == *n* . It is a sparsity pattern for the Hessian $H(x)$. This argument is not used (and need not satisfy any conditions), when *work* is non-empty.

work

This argument must have been constructed by the call

work = `cppad_py.sparse_hes_work` ()

We refer to its initial value, and its value after *work* . `clear` () , as empty. If it is empty, information is stored in *work* . This can be used to reduce computation when a future call is for the same object *f* , and the same subset of the Hessian. If either of these values change, use *work* . `clear` () to empty this structure.

n_sweep

The return value *n_sweep* has prototype

`int` *n_sweep*

It is the number of first order forward sweeps used to compute the requested Hessian values. Each first forward sweep is followed by a second order reverse sweep so it is also the number of reverse sweeps. This is proportional to the total computational work, not counting the zero order forward sweep, or combining multiple columns and rows into a single sweep.

Uses Forward

After each call to *py_fun_forward*, the object f contains the corresponding Taylor coefficients for all the variables in the operation sequence.. After a call to `sparse_hes` the zero order coefficients correspond to

f`.forward(0,`x`)`

All the other forward mode coefficients are unspecified.

sparse_hes_xam_py

3.1.2.6.1 Python: Hessian Sparsity Patterns: Example and Test

```
def sparse_hes_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    # number of dependent and independent variables
    n = 3
    #
    # create the independent variables ax
    x = numpy.empty(n, dtype=float)
    for i in range( n  ) :
        x[i] = i + 2.0
    #
    ax = cppad_py.independent(x)
    #
    # ay[i] = j * ax[j] * ax[i]
    # where i = mod(j + 1, n)
    ay = numpy.empty(n, dtype=cppad_py.a_double)
    for j in range( n  ) :
        i = j+1
        if i >= n  :
```

(continues on next page)

(continued from previous page)

```
            i = i - n
        #
        aj = cppad_py.a_double(j)
        ax_j = ax[j]
        ax_i = ax[i]
        ay[i] = aj * ax_j * ax_i
    #
    #
    # define af corresponding to f(x)
    f  = cppad_py.d_fun(ax, ay)
    #
    # Set select_d (domain) to all true,
    # initial select_r (range) to all false
    # initialize r to all zeros
    select_d = numpy.empty(n, dtype=bool)
    select_r = numpy.empty(n, dtype=bool)
    r = numpy.empty(n, dtype=float)
    for i in range( n ) :
        select_d[i] = True
        select_r[i] = False
        r[i] = 0.0
    #
    #
    # only select component n-1 of the range function
    # f_0 (x) = (n-1) * x_{n-1} * x_0
    select_r[0] = True
    r[0] = 1.0
    #
    # sparisty pattern for Hessian
    pattern = cppad_py.sparse_rc()
    f.for_hes_sparsity(select_d, select_r, pattern)
    #
    # compute all possibly non-zero entries in Hessian
    # (should only compute lower triangle becuase matrix is symmetric)
    subset = cppad_py.sparse_rcv(pattern)
    #
    # work space used to save time for multiple calls
    work = cppad_py.sparse_hes_work()
    #
    f.sparse_hes(subset, x, r, pattern, work)
    #
    # check that result is sparsity pattern for Hessian of f_0 (x)
    ok = ok and subset.nnz() == 2
    row = subset.row()
    col = subset.col()
    val = subset.val()
    for k in range( 2 ) :
        i = row[k]
        j = col[k]
        v = val[k]
        if i <= j  :
            ok = ok and i == 0
            ok = ok and j == n-1
        #
        if i >= j  :
            ok = ok and i == n-1
            ok = ok and j == 0
```

(continues on next page)

(continued from previous page)

```
        #
        ok = ok and v == n-1
    #
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/sparse_hes_xam.py`

Example

sparse_hes_xam_py

---

xsrst input file: `lib/python/cppad_py/sparse_hes.py`

1. py_sparse_rc: *3.1.2.1 Sparsity Patterns*
2. py_sparse_rcv: *3.1.2.2 Sparse Matrices*
3. py_jac_sparsity: *3.1.2.3 Jacobian Sparsity Patterns*
4. py_hes_sparsity: *3.1.2.4 Hessian Sparsity Patterns*
5. py_sparse_jac: *3.1.2.5 Computing Sparse Jacobians*
6. py_sparse_hes: *3.1.2.6 Computing Sparse Hessians*

---

xsrst input file: `lib/python/cppad_py/sparse.xsrst`

py_utility

3.1.3 Python Utilities

- *Children*

Children

numpy2vec

3.1.3.1 Convert a Numpy Array to a cppad_py Vector

- *Syntax*
- *array*
- *dtype*
- *shape*
- *context*

- *name*
- *vec*

Syntax

vec = `cppad_py.utility.numpy2vec` (
    array , *dtype* , *shape* , *context* , *name*
)

array

This is either a vector (only one index) or a matrix (has two indices) that we are converting to a vector. If this array does not match the conditions below, an exception is raised with an appropriate error message.

dtype

This is the expected data type for the elements of the array. It must be one of the following: `bool` , `int` , `float` or `cppad_py.a_double` .

shape

This either a `int` or a tuple of `int` with length one or two. If it is an `int` , *array* is expected to be a vector. If it is a tuple with length one, *array* is expected to be a vector. Otherwise a matrix is expected.

context

This is the context that *array* appears in (often to syntax for some other operation). It is a `str` that is used for error reporting.

name

This is the name used for *array* in *context* . It is a `str` that is used for error reporting.

vec

This is the values of *array* as a vector is row major order. It has type `vec_double` if *dtype* is `float` , and `vec_a_double` if *dtype* is `a_double` .

---

xsrst input file: `lib/python/cppad_py/utility.py`

vec2numpy

3.1.3.2 Convert a cppad_py Vector to a Numpy Array

- *Syntax*
- *vec*
- *nr*
- *nc*
- *array*

Syntax

array = `cppad_py.utility.vec2numpy` (*vec* , *nr*)
array = `cppad_py.utility.vec2numpy` (*vec* , *nr* , *nc*)

vec

This must have one of the following types: `cppad_py.vec_int` , `cppad_py.vec_double` , `vec_py.vec_a_double` . with size equal to *nr* * *nc* .

nr

This is an `int` equal to the number of rows in the array. If the argument *nc* is not present, the array is a vector; i.e., `len` (*array* `.shape ) == 1` .

nc

If this argument is present, it is an `int` equal to the number of columns in the array. In this case the array is a matrix; i.e., `len` (*array* `.shape ) == 2` .

array

This is the array corresponding to *vec* in row major order. Note that this array can be used after the vector *vec* drops out of scope (is deleted).

---

xsrst input file: `lib/python/cppad_py/utility.py`

1. numpy2vec: *3.1.3.1 Convert a Numpy Array to a cppad_py Vector*
2. vec2numpy: *3.1.3.2 Convert a cppad_py Vector to a Numpy Array*

---

xsrst input file: `lib/python/cppad_py/utility.xsrst`

more_py

3.1.4 Steps To Add More Python Functions

- *Purpose*
- ***Documentation***
  - *independent*
  - *new_dynamic*
  - *Example*
- ***Implementation***
  - *independent*
  - *new_dynamic*
  - *fun_new_dynamic.py*
  - *__init__.py*
- *Example*
- *Testing*

Purpose

This section outlines the steps for adding more python functionality to cppad_py. This is done by example showing how the *py_fun_new_dynamic* was added. This example case was chosen because it required both changing one python function, *py_independent*, and adding a new python function, *py_fun_new_dynamic*.

Documentation

independent

The python file `lib/python/cppad_py/independent.py` was edited to add the following syntax documentation:

(*ax* , *adynamic*) = `cppad_py::independent` (*x* , *dynamic*)

and the extra return value was documented; see *adynamic*.

new_dynamic

The *cpp_fun_new_dynamic* documentation was added in the file `lib/python/cppad_py/fun_new_dynamic.py` . In addition, the OMhelp command

```
%lib/python/cppad_py/fun_new_dynamic.py
```

was added to the file `lib/python/cppad_py/fun.py` .

Example

An example file was added to the documentation, below the *py_independent* section, using the OMhelp commands:

```
{ xsrst_dollar}children%
   example/python/core/fun_dynamic_xam.py
%${xsrst_dollar }
```

In addition, a reference to this example was added under the *example* heading in the `independent` documentation.

Implementation

independent

The `independent` function in `lib/python/cppad_py/independent.py` was changed to handle dynamic parameters as follows:

```
import cppad_py
import numpy
def independent(x, dynamic = None) :
    """
    ax = independent(x)
    creates the indepedent numpy vector ax, with value equal numpy vector x,
    and starts recording a_double operations.
    """
    # convert x -> u
    dtype    = float
    #
    nx = x.size
    if dynamic is None :
        syntax   = 'independent(x)'
        u = cppad_py.utility.numpy2vec(x, dtype, nx, syntax, 'x')
        av = cppad_py.swig.independent(u)
        ax = cppad_py.utility.vec2numpy(av, av.size());
        return ax
    #
    nd       = dynamic.size
    syntax   = 'independent(x, dynamic)'
    u = cppad_py.utility.numpy2vec(x, dtype, nx, syntax, 'x')
```

(continues on next page)

(continued from previous page)

```
    v = cppad_py.utility.numpy2vec(dynamic, dtype, nd, syntax, 'dynamic')
    a_both   = cppad_py.swig.independent(u, v)
    ax       = numpy.empty(nx,       dtype=cppad_py.a_double)
    adynamic = numpy.empty(nd, dtype=cppad_py.a_double)
    # use copy constructor so a separate copy is made for numpy arrays
    for i in range(nx) :
        ax[i] = cppad_py.a_double( a_both[i] )
    for i in range(nd) :
        adynamic[i] = cppad_py.a_double( a_both[nx + i] )
    #
    return (ax, adynamic)
```

new_dynamic

The following function declaration was added to the `d_fun` class in the `lib/python/cppad_py/fun.py` file:

```
def new_dynamic(self, dynamic) :
    return cppad_py.d_fun_new_dynamic(self.f, dynamic)
```

A similar declaration was added to the `a_fun` class.

fun_new_dynamic.py

The implementation of `d_fun_new_dynamic` and `a_fun_new_dynamic` were added to the file `fun_new_dynamic.py`

__init__.py

The following code was added to the file `lib/python/cppad_py/__init__.py`:

```
from cppad_py.fun_new_dynamic import a_fun_new_dynamic
from cppad_py.fun_new_dynamic import d_fun_new_dynamic
```

Example

The file `example/python/core/fun_dynamic_xam.py` was added with the following contents: *fun_dynamic_xam_py*. In addition, in the file

```
example/python/check_all.py.in
```

the following text was added to the list of python example files.

```
    'fun_dynamic_xam',
%
```

Testing

You must do a git add for all of the new files before running `bin/check_all.sh` After all the changes above were implemented, `bin/check_all.sh` was run and the changes were made until the warnings and errors were fixed. The command

```
grep 'fun_dynamic_xam' check_all.log
```

was used to make sure that the new python example / test was run. Note that the python files in `cppad_py` are copies of the python files in `lib/python` . So when you fix errors during testing, you need to fix the `lib/python` file. Also note that if a particular step in `bin/check_all.sh` is failing, you can just re-run that step to see if a particular fix works. Once the tests were working, the changes where checked into using `git` .

---

xsrst input file: `lib/python/cppad_py/more_py.xsrst`

1. py_fun: *3.1.1 Cppad Py AD Functions*
2. py_sparse: *3.1.2 Python Sparsity Routines*
3. py_utility: *3.1.3 Python Utilities*
4. more_py: *3.1.4 Steps To Add More Python Functions*

---

xsrst input file: `lib/python/cppad_py/py_lib.xsrst`

cpp_lib

3.2 The C++ Library

- *Children*

Children

a_double

3.2.1 Cppad Py AD Scalars

- *Children*

Children

a_double_ctor

3.2.1.1 The a_double Constructor

- *Syntax*

- *Purpose*
- *d*
- *a_other*
- *ad*
- *Example*

Syntax

ad = `cppad_py.a_double` ()
ad = `cppad_py.a_double` (*d*)
ad = `cppad_py.a_double` (*a_other*)

Purpose

Creates a `cppad_py::a_double` object that can be use to track floating point operations and perform algorithmic differentiation.

d

This argument has prototype

```
const double& d
```

The resulting *ad* variable represents a constant function equal to *d* .

a_other

This argument has prototype

```
const a_double& a_other
```

The resulting *ad* variable is the same function of the independent variables as *a_other* .

ad

is the `a_double` object that is constructed.

Example

All of the other `a_double` examples use an `a_double` constructor.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_unary_op

3.2.1.2 a_double Unary Plus and Minus

- *Syntax*
- *ax*
- *ay*
- *Example*

Syntax

$ay = + ax$
$ay = - ax$

ax

This object has prototype

```
const a_double& ax
```

ay

If the operator is + , the result is equal to ax . If it is − , the result is the negative of ax .

a_double_unary_op_xam_cpp

3.2.1.2.1 C++: a_double Unary Plus and Minus: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_unary_op_xam(void) {
    using cppad_py::a_double;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    a_double a2(2.0);
    a_double aplus2 = + a2 ;
    a_double aminus2 = - a2 ;
    //
    ok = ok && aplus2 == 2.0;
    ok = ok && aminus2 == -2.0;
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/a_double_unary_op_xam.cpp`

a_double_unary_op_xam_py

3.2.1.2.2 Python: a_double Unary Plus and Minus: Example and Test

```
def a_double_unary_op_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ---------------------------------------------------------------------
    a2 = cppad_py.a_double(2.0)
    aplus2 = + a2
    a2_minus = - a2
    #
    ok = ok and aplus2 == 2.0
    ok = ok and a2_minus == -2.0
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/a_double_unary_op_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_property

3.2.1.3 Properties of an a_double Object

- *Syntax*
- *ad*
- ***value***
  - *Restriction*
- *parameter*
- *variable*
- *near_equal*
- *var2par*
- *Example*

Syntax

d = *ad* . `value` ()
p = *ad* . `parameter` ()
v = *ad* . `variable` ()
e = *ad* . `near_equal` (*aother*)
ap = *ad* . `var2par` ()

ad

This object has prototype

```
const a_double& ad
```

value

The result *d* has prototype

```
double d
```

It is the value of *ad* , as a constant function.

Restriction

The object *ad* must not depend on the *independent* variables when *ad* . `value` () is called. If it does depend on the independent variables, you will have to wait until the current recording is terminated before you can access its value; see *var2par* below.

parameter

The result *p* has prototype

```
bool p
```

It is true if *ad* represent a constant functions; i.e., *ad* not depend on the independent variables.

variable

The result *v* has prototype

```
bool v
```

It is true if *ad* is not a constant function; i.e., *ad* depends on the independent variables.

near_equal

The argument *aother* , and the result *e* , have prototype

```
const a_double& aother
bool e
```

The result is true if *ad* is nearly equal to *aother* . To be specific, the result is

$$|d - o| \leq 100 \, \varepsilon \, (|d| + |o|)$$

where d and o are the value corresponding to *ad* and *aother* and ε is machine epsilon corresponding to the type `double` .

var2par

The result has prototype

a_double *ap*

It has the same value as *ad* and is sure to be a parameter (*ad* may or may not be a variable). This can be useful when you want to access the value of *ad* while is a variable; *value* above.

a_double_property_xam_cpp

3.2.1.3.1 C++: a_double Properties: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_property_xam(void) {
    using cppad_py::a_double;
    //
    // initialize return variable
    bool ok = true;
    //------------------------------------------------------------------------
    a_double a3(3.0);
    //
    ok = ok && a3   == 3.0;
    ok = ok && a3.parameter();
    ok = ok && ! a3.variable();
    //
    // near_equal
    a_double r3 = a3.sqrt() ;
    ok = ok && a3.near_equal( r3 * r3) ;;
    //
    // var2par
    a_double p3 = a3.var2par();
    ok &= p3.value() == 3.0;
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/a_double_property_xam.cpp`

a_double_property_xam_py

3.2.1.3.2 Python: a_double Properties: Example and Test

```
def a_double_property_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    a3 = cppad_py.a_double(3.0)
    #
    ok = ok and a3   == 3.0
    ok = ok and a3.parameter()
    ok = ok and not a3.variable()
    #
    # near_equal
    r3 = a3.sqrt()
    ok = ok and a3.near_equal( r3 * r3)
    #
    # var2par
    p3  = a3.var2par()
    ok  = ok and p3.value() == 3.0
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/a_double_property_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_binary

3.2.1.4 a_double Binary Operators with an AD Result

- *Syntax*
- *op*
- *ax*
- *ay*
- *y*
- *az*
- *Example*

Syntax

az = *ax op ay*
az = *ax op y*
az = *y op ax*

op

The binary operator *op* is one of the following: addition + , subtraction − , multiplication ⋆ , division / , or exponentiation ⋆⋆ . Note that exponentiation is a function is c++ and an operator in python.

ax

This object has prototype

```
const a_double& ax
```

ay

This object has prototype

```
const a_double& ay
```

y

This object has prototype

```
const double& y
```

az

The result has prototype

```
a_double az
```

a_double_binary_xam_cpp

3.2.1.4.1 C++: a_double Binary Operators With AD Result: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_binary_xam(void) {
    using cppad_py::a_double;
    bool ok      = true;
    a_double a2  = 2.0;
    a_double a3(3.0);
    // ------------------------------------------------------------------------
    // a_double op a_double
    a_double a5       = a2 + a3;
    a_double a6       = a2 * a3;
    a_double a1_minus = a2 - a3;
    a_double a23      = a2 / a3;
    //
    ok = ok && a5 == 5.0;
    ok = ok && a6 == 6.0;
    ok = ok && a1_minus == -1.0;
    ok = ok && a23.near_equal( a_double(2.0 / 3.0 ) );
    // ------------------------------------------------------------------------
    // a_double op double
    a5       = a2 + 3.0;
    a6       = a2 * 3.0;
    a1_minus = a2 - 3.0;
    a23      = a2 / 3.0;
    //
    ok = ok && a5 == 5.0;
    ok = ok && a6 == 6.0;
    ok = ok && a1_minus == -1.0;
    ok = ok && a23.near_equal( a_double(2.0 / 3.0 ) );
    // ------------------------------------------------------------------------
    // double op a_double
    a5           = cppad_py::radd(3.0, a2);
    a6           = cppad_py::rmul(3.0, a2);
    a_double a1  = cppad_py::rsub(3.0,  a2);
    a_double a32 = cppad_py::rdiv(3.0, a2);
    //
    ok = ok && a5 == 5.0;
    ok = ok && a6 == 6.0;
    ok = ok && a1 == 1.0;
    ok = ok && a32.near_equal( a_double(3.0 / 2.0 ) );
    // ------------------------------------------------------------------------
    // pow
    a_double a8 = cppad_py::pow(a2, a3);
    a_double a9 = cppad_py::pow(a3, 2.0);
    a_double a4 = cppad_py::pow(2.0, a2);
    ok = ok && a8.near_equal( a_double(8.0) );
    ok = ok && a9.near_equal( a_double(9.0) );
    ok = ok && a4.near_equal( a_double(4.0) );
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/a_double_binary_xam.cpp`

a_double_binary_xam_py

3.2.1.4.2 Python: a_double Binary Operators With AD Result: Example and Test

```
def a_double_binary_xam() :
    #
    import numpy
    from cppad_py import a_double
    ok = True
    a2 = a_double(2.0)
    a3 = a_double(3.0)
    # ------------------------------------------------------------------
    # a_double op a_double
    a5       = a2 + a3
    a6       = a2 * a3
    a1_minus = a2 - a3
    a23      = a2 / a3
    #
    ok = ok and a5 == 5.0
    ok = ok and a6 == 6.0
    ok = ok and a1_minus == -1.0
    ok = ok and a23.near_equal( a_double(2.0 / 3.0) )
    # ------------------------------------------------------------------
    # a_double op double
    a5       = a2 + 3.0
    a6       = a2 * 3.0
    a1_minus = a2 - 3.0
    a23      = a2 / 3.0
    #
    ok = ok and a5 == 5.0
    ok = ok and a6 == 6.0
    ok = ok and a1_minus == -1.0
    ok = ok and a23.near_equal( a_double(2.0 / 3.0) )
    # ------------------------------------------------------------------
    # double op a_double
    a5       = 3.0 + a2
    a6       = 3.0 * a2
    a1       = 3.0 - a2
    a32      = 3.0 / a2
    #
    ok = ok and a5 == 5.0
    ok = ok and a6 == 6.0
    ok = ok and a1 == 1.0
    ok = ok and a32.near_equal( a_double(3.0 / 2.0) )
    # ------------------------------------------------------------------
    # pow
    a8 = a2  ** a3
    a9 = a3  ** 2.0
    a4 = 2.0 ** a2
    ok = ok and a8.near_equal( a_double(8.0) )
    ok = ok and a9.near_equal( a_double(9.0) )
    ok = ok and a4.near_equal( a_double(4.0) )
    # ------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/a_double_binary_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_compare

3.2.1.5 a_double Comparison Operators

- *Syntax*
- *op*
- *ax*
- *ay*
- *y*
- *b*
- *Example*

Syntax

b = *ax op ay*
b = *ax op y*

op

The binary operator *op* is one of the following: < (less than), <= (less than or equal), > (greater than), >= (greater than or equal), == (equal), != (not equal).

ax

This object has prototype

`const a_double&` *ax*

ay

This object has prototype

`const a_double&` *ay*

y

This object has prototype

`const double&` *y*

b

The result has prototype

`bool` *b*

a_double_compare_xam_cpp

3.2.1.5.1 C++: a_double Comparison Operators: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_compare_xam(void) {
    using cppad_py::a_double;
    bool ok = true;
    a_double a2(2.0);
    a_double a3(3.0);
    //------------------------------------------------------------------------
    ok = ok && a2   <  a3;
    ok = ok && a2   <= a3;
    ok = ok && a3 >  a2;
```

(continues on next page)

(continued from previous page)

```
    ok = ok && a3 >= a2;
    ok = ok && a3 != a2;
    ok = ok && a3 == a3;
    //
    ok = ok && ! (a2 >  a3) ;
    ok = ok && ! (a2 >= a3) ;
    ok = ok && ! (a2 == a3) ;
    //-----------------------------------------------------------------------
    ok = ok && a2   <  3.0;
    ok = ok && a2   <= 3.0;
    ok = ok && a3 >  2.0;
    ok = ok && a3 >= 2.0;
    ok = ok && a3 != 2.0;
    ok = ok && a3 == 3.0;
    //
    ok = ok && ! (a2 >  3.0) ;
    ok = ok && ! (a2 >= 3.0) ;
    ok = ok && ! (a2 == 3.0) ;
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/a_double_compare_xam.cpp`

a_double_compare_xam_py

3.2.1.5.2 Python: a_double Comparison Operators: Example and Test

```
def a_double_compare_xam() :
    #
    import numpy
    import cppad_py
    ok = True
    a2 = cppad_py.a_double(2.0)
    a3 = cppad_py.a_double(3.0)
    # ---------------------------------------------------------------------
    ok = ok and a2   <  a3
    ok = ok and a2   <= a3
    ok = ok and a3 >  a2
    ok = ok and a3 >= a2
    ok = ok and a3 != a2
    ok = ok and a3 == a3
    #
    ok = ok and not (a2 >  a3)
    ok = ok and not (a2 >= a3)
    ok = ok and not (a2 == a3)
    # ---------------------------------------------------------------------
    ok = ok and a2   <  3.0
    ok = ok and a2   <= 3.0
    ok = ok and a3 >  2.0
    ok = ok and a3 >= 2.0
    ok = ok and a3 != 2.0
    ok = ok and a3 == 3.0
```

(continues on next page)

(continued from previous page)

```
    #
    ok = ok and not (a2 >  3.0)
    ok = ok and not (a2 >= 3.0)
    ok = ok and not (a2 == 3.0)
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/a_double_compare_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_assign

3.2.1.6 a_double Assignment Operators

- *Syntax*
- *op*
- *ax*
- *ay*
- *y*
- *Example*

Syntax

ax op ay
aw op y

op

The assignment operator *op* is one of the following:

op	Meaning
=	simple assignment
+=	*ax* = *ax* + (*ay* `or` *y*)
−=	*ax* = *ax* - (*ay* `or` *y*)
⋆=	*ax* = *ax* ⋆ *ay* `or` *y*)
/=	*ax* = *ax* / *ay* `or` *y*)

ax

This object has prototype

```
const a_double& ax
```

ay

This object has prototype

```
a_double& ay
```

y

This object has prototype

```
const double& y
```

a_double_assign_xam_cpp

3.2.1.6.1 C++: a_double Assignment Operators: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_assign_xam(void) {
    using cppad_py::a_double;
    //
    // initialize return variable
    bool ok = true;
    //----------------------------------------------------------------------
    a_double ax(2.0);
    //
    ax = 3.0;
    ok = ok && ax == 3.0;
    //
    ax += a_double(2.0);
    ok = ok && ax == 5.0;
    //
    ax -= 1.0;
    ok = ok && ax == 4.0;
    //
    ax *= a_double(3.0);
    ok = ok && ax == 12.0;
    //
```

(continues on next page)

(continued from previous page)

```
    ax /= 4.0;
    ok = ok && ax == a_double(3.0);
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/a_double_assign_xam.cpp`

a_double_assign_xam_py

3.2.1.6.2 Python: a_double Assignment Operators: Example and Test

```
def a_double_assign_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    #
    ax = cppad_py.a_double(3.0);
    ok = ok and ax == 3.0
    #
    ax += cppad_py.a_double(2.0);
    ok = ok and ax == 5.0
    #
    ax -= 1.0;
    ok = ok and ax == cppad_py.a_double(4.0)
    #
    ax *= cppad_py.a_double(3.0);
    ok = ok and ax == 12.0
    #
    ax /= 4.0;
    ok = ok and ax == cppad_py.a_double(3.0)
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/a_double_assign_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_unary_fun

3.2.1.7 Unary Functions with AD Result

- *Syntax*
- *ax*
- *fun*
- *ay*
- *Example*

Syntax

ay = *ax* . *fun* ()

ax

This object has prototype

`const a_double&` *ax*

This is the argument for the function evaluation.

fun

This specifies which function is being evaluated and is one of following value: `abs` , `acos` , `asin` , `atan` , `cos` , `cosh` , `erf` , `exp` , `fabs` , `log` , `sin` , `sinh` , `sqrt` , `tan` , `tanh` . 2DO: Add the C++11 functions asinh, acosh, atanh, expm1, and log1p to this list.

ay

The result object has prototype

`a_double` *ay*

and is the specified function evaluated at the specified argument; i.e.,

ay = *fun* (*ax*)

a_double_unary_fun_xam_cpp

3.2.1.7.1 C++: a_double Unary Functions with AD Result: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_unary_fun_xam(void) {
    using cppad_py::a_double;
    //
    // initialize return variable
    bool ok = true;
    //------------------------------------------------------------------------
    //
    a_double a1(-1.0);
    a_double abs1 = a1.abs();
    ok  = ok && abs1 == 1.0;
    //
    a1   = 1.0;
    abs1 = a1.fabs();
    ok   = ok && abs1 == 1.0;
    //
    // pi/4
    a_double pi_4 = a1.atan();
    //
    // sqrt(2)
    a_double atmp(2.0);
    a_double r2 = atmp.sqrt();
    //
    // sin(pi/4)  * sqrt(2) = 1.0;
    atmp = r2 * pi_4.sin() ;
    ok = ok && atmp.near_equal(a1) ;
    //
    // cos(pi/4)  * sqrt(2) = 1.0;
    atmp = r2 * pi_4.cos() ;
    ok = ok && atmp.near_equal(a1) ;
    //
    // tan(pi/4)  = 1.0;
    atmp = pi_4.tan() ;
    ok = ok && atmp.near_equal(a1) ;
    //
    // erf(0.5) = 0.5204998778130465
    a_double acheck(0.5204998778130465);
    atmp =  a_double(0.5);
    atmp = atmp.erf();
    ok = ok && atmp.near_equal(acheck);
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/a_double_unary_fun_xam.cpp`

a_double_unary_fun_xam_py

3.2.1.7.2 Python: a_double Unary Functions with AD Result: Example and Test

```
def a_double_unary_fun_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    #
    # abs
    a1   = cppad_py.a_double(-1.0)
    abs1 = a1.abs()
    ok   = ok and abs1 == 1.0
    # fabs
    a1   = cppad_py.a_double(1.0)
    abs1 = a1.fabs()
    ok   = ok and abs1 == 1.0
    #
    # pi/4
    pi_4 = a1.atan()
    #
    # sqrt(2)
    atmp = cppad_py.a_double(2.0)
    r2 = atmp.sqrt()
    #
    # sin(pi/4)  * sqrt(2) = 1.0
    atmp = r2 * pi_4.sin()
    ok = ok and atmp.near_equal(a1)
    #
    # cos(pi/4)  * sqrt(2) = 1.0
    atmp = r2 * pi_4.cos()
    ok = ok and atmp.near_equal(a1)
    #
    # tan(pi/4)  = 1.0
    atmp = pi_4.tan()
    ok = ok and atmp.near_equal(a1)
    #
    # erf(0.5) = 0.5204998778130465
    acheck = cppad_py.a_double(0.5204998778130465)
    atmp   = cppad_py.a_double(0.5)
    atmp   = atmp.erf()
    ok     = ok and atmp.near_equal(acheck)
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/a_double_unary_fun_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

a_double_cond_assign

3.2.1.8 AD Conditional Assignment

- *Syntax*
- *Purpose*
- *target*
- *cop*
- *left*
- *right*
- *if_true*
- *if_false*
- *Example*

Syntax

target . `cond_assign` (*cop* , *left* , *right* , *if_true* , *if_false*)

Purpose

The code

```
if ( left cop right )
    target = if_true
else
    target = if_false
```

records either the true or false case depending on the value of *left* and *right* ; see *cpp_fun_ctor*. If *left* or *right* is a *variable*, it may be desirable to switch between *if_true* and *if_false* depending of the value of the independent variable during calls to order zero *cpp_fun_forward*. The `cond_assign` does this.

target

This object has prototype

`a_double&` *target*

cop

This argument has prototype

```
const char *cop
```

The comparison is

left cop right

where *cop* is one of the following:

cop	
<	less than
<=	less than or equal
==	equal
>=	greater than or equal
>	greater than

left

This argument has prototype

`const a_double&` *left*

It specifies the left operand in the comparison.

right

This argument has prototype

`const a_double&` *right*

It specifies the right operand in the comparison.

if_true

This argument has prototype

`const a_double&` *if_true*

It specifies the value assigned to *ad* if the result of the comparison is true.

if_false

This argument has prototype

`const a_double&` *if_false*

It specifies the value assigned to *ad* if the result of the comparison is false.

a_double_cond_assign_xam_cpp

3.2.1.8.1 C++: a_double Conditional Assignment: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool a_double_cond_assign_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    int n_ind = 4;
    int n_dep = 1;
    //
    // create ax (value of independent variables does not matter)
    vec_double x(n_ind);
```

(continues on next page)

(continued from previous page)

```
    x[0] = 0.0;
    x[1] = 1.0;
    x[2] = 2.0;
    x[3] = 3.0;
    vec_a_double ax = cppad_py::independent(x);
    //
    // arguments to conditional assignment
    a_double left = ax[0];
    a_double right = ax[1];
    a_double if_true = ax[2];
    a_double if_false = ax[3];
    //
    // assignment
    a_double target;
    target.cond_assign(
        "<",
        left,
        right,
        if_true,
        if_false
    );
    //
    // f(x) = taget
    vec_a_double ay(n_dep);
    ay[0] = target;
    d_fun f(ax, ay);
    //
    // assignment with different independent variable values
    x[0] = 9.0; // left
    x[1] = 8.0; // right
    x[2] = 7.0; // if_true
    x[3] = 6.0; // if_false
    int p = 0;
    vec_double y = f.forward(p, x);
    ok = ok && y[0] == 6.0;
    //
    return( ok  );
}
```

xsrst input file: `example/cplusplus/a_double_cond_assign_xam.cpp`

a_double_cond_assign_xam_py

3.2.1.8.2 Python: a_double Conditional Assignment: Example and Test

```
def a_double_cond_assign_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
    n_ind = 4
    n_dep = 1
    #
    # create ax (value of independent variables does not matter)
    x = numpy.empty(n_ind, dtype=float)
    x[0] = 0.0
    x[1] = 1.0
    x[2] = 2.0
    x[3] = 3.0
    ax = cppad_py.independent(x)
    #
    # arguments to conditional assignment
    left = ax[0]
    right = ax[1]
    if_true = ax[2]
    if_false = ax[3]
    #
    # assignment
    target = cppad_py.a_double()
    target.cond_assign(
        "<",
        left,
        right,
        if_true,
        if_false
    )
    #
    # f(x) = taget
    ay = numpy.empty(n_dep, dtype=cppad_py.a_double)
    ay[0] = target
    f  = cppad_py.d_fun(ax, ay)
    #
    # assignment with different independent variable values
    x[0] = 9.0 # left
    x[1] = 8.0 # right
    x[2] = 7.0 # if_true
    x[3] = 6.0 # if_false
    p = 0
    y = f.forward(p, x)
    ok = ok and y[0] == 6.0
    #
    return( ok  )
#
```

xsrst input file: `example/python/core/a_double_cond_assign_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/a_double.cpp`

1. a_double_ctor: *3.2.1.1 The a_double Constructor*
2. a_double_unary_op: *3.2.1.2 a_double Unary Plus and Minus*
3. a_double_property: *3.2.1.3 Properties of an a_double Object*
4. a_double_binary: *3.2.1.4 a_double Binary Operators with an AD Result*
5. a_double_compare: *3.2.1.5 a_double Comparison Operators*
6. a_double_assign: *3.2.1.6 a_double Assignment Operators*
7. a_double_unary_fun: *3.2.1.7 Unary Functions with AD Result*
8. a_double_cond_assign: *3.2.1.8 AD Conditional Assignment*

---

xsrst input file: `lib/cplusplus/cplusplus.xsrst`

vector

3.2.2 Cppad Py Vectors

- *Children*

Children

vector_ctor

3.2.2.1 Cppad Py Vector Constructors

- *Syntax*
- *Purpose*
- *n*
- *vec_bool*
- *vec_int*
- *vec_double*
- *vec_a_double*
- *Example*

Syntax

bv = `cppad_py.vec_bool` (*n*)
iv = `cppad_py.vec_int` (*n*)
dv = `cppad_py.vec_double` (*n*)
av = `cppad_py.vec_a_double` (*n*)

Purpose

Creates a vector with *n* elements.

n

The argument *n* is a non-negative integer with default value zero; i.e., if it is not present, zero is used.

vec_bool

This result *bv* is a vector with elements of type `bool`

vec_int

This result *bv* is a vector with elements of type `int`

vec_double

This result *bv* is a vector with elements of type `double`

vec_a_double

This result `av` is a vector with elements of type *a_double*.

Example

All of the other vector examples use the vector constructors.

---

xsrst input file: `lib/cplusplus/vector.xsrst`

vector_size

3.2.2.2 Size of a Vector

- *Syntax*
- *v*
- *n*
- *Example*

Syntax

n = v . `size` ()

v

The object v has one of the following prototypes:

```
const vec_bool& v
const vec_int& v
const vec_double& v
const vec_a_double& v
```

n

The result has prototype

```
size_t n
```

i.e., it is a positive integer. Its value is the number of elements in the vector v .

vector_size_xam_cpp

3.2.2.2.1 C++: Size of Vectors: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool vector_size_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_bool;
    using cppad_py::vec_int;
    using cppad_py::vec_double;
```

(continues on next page)

(continued from previous page)

```
    using cppad_py::vec_a_double;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    // create vectors
    vec_bool bv = vec_bool();
    vec_int iv = vec_int(1);
    vec_double dv(2);
    vec_a_double av(3);
    //
    // check size of vectors
    ok = ok && bv.size() == 0 ;
    ok = ok && iv.size() == 1 ;
    ok = ok && dv.size() == 2 ;
    ok = ok && av.size() == 3 ;
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/vector_size_xam.cpp`

vector_size_xam_py

3.2.2.2.2 Python: Size of Vectors: Example and Test

```
def vector_size_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ---------------------------------------------------------------------
    # create vectors
    bv = cppad_py.vec_bool()
    iv = cppad_py.vec_int(1)
    dv = cppad_py.vec_double(2)
    av = cppad_py.vec_a_double(3)
    #
    # check size of vectors
    ok = ok and bv.size() == 0
    ok = ok and iv.size() == 1
    ok = ok and dv.size() == 2
    ok = ok and av.size() == 3
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/vector_size_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/vector.xsrst`

vector_set_get

3.2.2.3 Setting and Getting Vector Elements

- *Syntax*
- *element_type*
- *i*
- *u*
- *x*
- *v*
- *y*
- *Example*

Syntax

u [i] = x
y = v [i]

element_type

We use *element_type* to denote the type of elements in the vector. It must be one of the following types: `bool` , `int` , `double` , `a_double` .

i

This argument has prototype

```
size_t i
```

It must be between zero and the size of the vector minus one.

u

The object *u* has prototype

`vec_` *element_type* `&` *u*

x

The argument *x* has prototype

`const` *element_type* `&` *x*

v

The object *v* has the following prototype

`const vec_` *element_type* `&` *v*

y

The result *y* has prototype

element_type y

vector_set_get_xam_cpp

3.2.2.3.1 C++: Setting and Getting Vector Elements: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool vector_set_get_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_bool;
    using cppad_py::vec_int;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    //
    // initialize return variable
    bool ok = true;
```

(continues on next page)

(continued from previous page)

```
    //------------------------------------------------------------------------
    int n = 4;
    vec_bool bv = vec_bool(n);
    vec_int iv = vec_int(n);
    vec_double dv(n);
    vec_a_double av(n);
    //
    // setting elements
    for(int i = 0; i < n ; i++) {
        bv[i] = i > n / 2;
        iv[i] = 2 * i;
        dv[i] = 3.0 * i;
        av[i] = 4.0 * i;
    }
    //
    for(int i = 0; i < n ; i++) {
        bool be = bv[i];
        ok = ok && be == (i > n / 2) ;
        //
        int ie = iv[i];
        ok = ok && ie == 2 * i ;
        //
        double de = dv[i];
        ok = ok && de == 3.0 * i ;
        //
        a_double ae = av[i];
        ok = ok && ae == 4.0 * i ;
    }
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/vector_set_get_xam.cpp`

vector_set_get_xam_py

3.2.2.3.2 Python: Setting and Getting Vector Elements: Example and Test

```
def vector_set_get_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ----------------------------------------------------------------------
    n = 4
    bv = cppad_py.vec_bool(n)
    iv = cppad_py.vec_int(n)
    dv = numpy.empty(n, dtype=float)
    av = numpy.empty(n, dtype=cppad_py.a_double)
    #
    # setting elements
```

(continues on next page)

(continued from previous page)

```
    for i in range( n  ) :
        bv[i] = i > n / 2
        iv[i] = 2 * i
        dv[i] = 3.0 * i
        av[i] = cppad_py.a_double(4.0 * i)
    #
    #
    for i in range( n  ) :
        be = bv[i]
        ok = ok and be == (i > n / 2)
        #
        ie = iv[i]
        ok = ok and ie == 2 * i
        #
        de = dv[i]
        ok = ok and de == 3.0 * i
        #
        ae = av[i]
        ok = ok and ae == 4.0 * i
    #
    #
    return( ok )
#
```

---

xsrst input file: `example/python/core/vector_set_get_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/vector.xsrst`

1. vector_ctor: *3.2.2.1 Cppad Py Vector Constructors*
2. vector_size: *3.2.2.2 Size of a Vector*
3. vector_set_get: *3.2.2.3 Setting and Getting Vector Elements*

---

xsrst input file: `lib/cplusplus/cplusplus.xsrst`

cpp_fun

3.2.3 Cppad Py AD Functions

- *Children*

Children

cpp_independent

3.2.3.1 Declare Independent Variables and Start Recording

- *Syntax*
- *Purpose*
- *x*
- *dynamic*
- *ax*
- *a_both*
- *Example*

Syntax

ax = `cppad_py::independent` (*x*)
a_both = `cppad_py::independent` (*x* , *dynamic*)

Purpose

This starts recording *a_double* operations. This recording is terminated, and the information is stored, by calling the *d_fun_constructor*. It can be terminated, and the information is lost, by calling *abort_recording*.

x

This argument has prototype

```
const vec_double& x
```

Its specifies the number of independent variables and their values during the recording. We use the notation *nx* = *x* . `size` () to denote the number of independent variables.

dynamic

This argument has prototype

`const vec_double&` *dynamic*

Its specifies the number of independent dynamic parameters and their values during the recording. We use the notation *nd* = *dynamic* . `size` () to denote the number of independent variables.

ax

This result has prototype

`vec_a_double&` *ax*

and is the vector of independent variables. It has size *nx* and for *i* = 0 to *n* -1

ax [*i*] == *x* [*i*]

a_both

this result has prototype

`vec_a_double&` *a_both*

and is the vector of both the independent variables and independent dynamic parameters. It has size *nx* + *nd* . For *i* = 0 to *nx* -1

a_both [*i*] == *x* [*i*]

is the *i*-th independent variable. For *i* = 0 to *nd* -1

a_both [*nx* + *i*] == *dynamic* [*i*]

is the *i*-th independent dynamic parameter.

fun_dynamic_xam_cpp

3.2.3.1.1 C++: Using Dynamic Parameters: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_dynamic_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    int nx = 2;
    int nd = 2;
    //
    // value of independent variables during recording
    vec_double x(nx);
    for(int i = 0; i < nx ; i++) {
        x[i] = i + 1.0;
    }
    //
    // value of independent dynamic parameters during recording
    vec_double dynamic(nd);
    for(int i = 0; i < nd ; i++) {
        dynamic[i] = i + x[nx-1] + 1.0;
    }
    vec_a_double a_both = cppad_py::independent(x, dynamic);
    //
    // extract vector of independent variables and dynamic parameters
    vec_a_double ax(nx), adynamic(nd);
    for(int i = 0; i < nx ; i++) {
        ax[i] = a_both[i];
    }
    for(int i = 0; i < nd ; i++) {
        adynamic[i] = a_both[nx + i];
    }
    //
    // create another dynamic paramerer
    a_double adyn  = adynamic[0] + adynamic[1];
    //
    // create another variable
    a_double avar  = ax[0] + ax[1] + adyn;
    //
    // create f(x) = x[0] + x[1] + dynamic[0] + dynamic[1]
    vec_a_double ay(1);
    ay[0] = avar;
    d_fun f(ax, ay);
    //
    // check some properties of f
    ok &= f.size_domain() == nx;
    ok &= f.size_order()  == 0;
    //
    // zero order forward mode using same values as during the recording
```

(continues on next page)

(continued from previous page)

```
    vec_double y(1);
    y   = f.forward(0, x);
    ok &= y[0] == (x[0] + x[1] + dynamic[0] + dynamic[1]);
    //
    // zero order forward mode using different value for dynamic parameters
    dynamic[0] = dynamic[0] + 1.0;
    dynamic[1] = dynamic[1] + 1.0;
    f.new_dynamic(dynamic);
    y   = f.forward(0, x);
    ok &= y[0] == (x[0] + x[1] + dynamic[0] + dynamic[1]);
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/fun_dynamic_xam.cpp`

Example

Most of the c++ `d_fun` examples use the *ax* return syntax. The *fun_dynamic_xam_cpp* example uses the *a_both* return syntax.

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_abort_recording

3.2.3.2 Abort Recording

- *Syntax*
- *Purpose*
- *Example*

Syntax

`cppad_py::abort_recording()`

Purpose

This aborts the current recording (if it exists) started by the most recent call to *independent*.

fun_abort_xam_cpp

3.2.3.2.1 C++: Abort Recording a_double Operations: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_abort_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    //
    // initialize return variable
    bool ok = true;
    //------------------------------------------------------------------------
    int n_ind = 2;
    //
    // create ax
    vec_double x(n_ind);
    for(int i = 0; i < n_ind ; i++) {
        x[i] = i + 1.0;
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // preform some a_double operations
    a_double ax0 = ax[0];
    a_double ax1 = ax[1];
    a_double ay = ax0 + ax1;
    //
    // check that ay is a variable; its value depends on the value of ax
    ok = ok && ay.variable();
    //
    // abort this recording
    cppad_py::abort_recording();
    //
    // check that ay is now a parameter, no longer a variable.
    ok = ok && ay.parameter();
    //
    // since it is a parameter, we can retrieve its value
    double y = ay.value();
    //
    // its value should be x0 + x1
    ok = ok && y  == x[0] + x[1];
    //
    // an abort when not recording has no effect
    cppad_py::abort_recording();
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/fun_abort_xam.cpp`

Example

c++.

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_ctor

3.2.3.3 Stop Current Recording and Store Function Object

- ***Syntax***
  - *d_fun*
  - *a_fun*
- *ax*
- *ay*
- ***f***
  - *Empty Function*
- *af*
- *Example*

Syntax

d_fun

f = `cppad_py::d_fun` (*ax* , *ay*)

a_fun

af = `cppad_py::a_fun` (*f*)

ax

This argument has prototype

```
const vec_a_double& ax
```

and must be the same as *ax* returned by the previous call to `independent` ; i.e., it must be the independent variable vector. We use the notation *n* = *ax* . `size` () to denote the number of independent variables.

ay

This argument has prototype

```
const vec_a_double& ax
```

It specifies the dependent variables. We use the notation $m = ay$. `size` () to denote the number of dependent variables.

f

This result has prototype

```
cppad_py::d_fun f
```

It has a representation for the floating point operations that mapped the independent variables ax to the dependent variables ay . This object computes function and derivative values using `double` .

Empty Function

In the case where ax and ay have size zero, the function is 'empty' and all its sizes are zero; see *cpp_fun_property*.

af

This result has prototype

```
cppad_py::a_fun af
```

It has a representation of the same function as f . This object computes function and derivative values using `a_double` . Initially, there are not Taylor coefficient stored in af ; i.e., *af_size_order()* is zero.

Example

All of the examples use these constructors.

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_property

3.2.3.4 Properties of a Function Object

- *Syntax*
- *f*
- *size_domain*
- *size_range*
- *size_var*
- *size_op*
- *size_order*
- *Example*

Syntax

`a_fun` *f* (*f*)
n = *f* . `size_domain` ()
m = *f* . `size_range` ()
v = *f* . `size_var` ()
p = *f* . `size_op` ()
q = *f* . `size_order` ()

f

This is either a *d_fun* or *a_fun* function object and is `const` .

size_domain

The return value has prototype

```
int n
```

and is the size of the vector *ax* in the function constructor; i.e., the number of independent variables.

size_range

The return value has prototype

```
int m
```

and is the size of the vector *ay* in the function constructor; i.e., the number of dependent variables.

size_var

The return value has prototype

```
int v
```

and is the number of variables in the function. This includes the independent variables, dependent variables, and any variables that are used to compute the dependent variables from the independent variables.

size_op

The return value has prototype

```
int p
```

and is the number of atomic operations that are used to express the dependent variables as a function of the independent variables.

size_order

The return value has prototype

```
int q
```

and is the number of Taylor coefficients currently stored in *f* , for every variable in the operation sequence corresponding to *f* . These coefficients are computed by *cpp_fun_forward*. This is different from the other function properties in that it can change after each call to *f* . `forward` ; see *size_order* in the forward mode section. The initial value for this property, when the object *f* or *af* is created, is zero.

fun_property_xam_cpp

3.2.3.4.1 C++: function Properties: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_property_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
    // initialize return variable
    bool ok = true;
    // ----------------------------------------------------------------------
    int n_ind = 1; // number of independent variables
    int n_dep = 2; // number of dependent variables
    int n_var = 1; // phantom variable at address 0
    int n_op  = 1; // special operator at beginning
    //
    // dimension some vectors
    vec_double x(n_ind);
    vec_a_double ay(n_dep);
    //
    // independent variables
    x[0]             = 1.0;
    vec_a_double ax = cppad_py::independent(x);
    n_var            = n_var + n_ind; // one for each indpendent
    n_op             = n_op + n_ind;
    //
    // first dependent variable
    ay[0] = ax[0] + ax[0];
    n_var = n_var + 1; // one variable and operator
    n_op = n_op + 1;
    //
    // second dependent variable
    a_double ax0 = ax[0];
    ay[1]        = ax0.sin();
    n_var        = n_var + 2; // two varialbes, one operator
    n_op         = n_op + 1;
    //
    // define f(x) = y
    d_fun f(ax, ay);
    n_op     = n_op + 1; // speical operator at end
    //
    // check f properties
    ok = ok && f.size_domain() == n_ind;
    ok = ok && f.size_range()  == n_dep;
    ok = ok && f.size_var()    == n_var;
    ok = ok && f.size_op()     == n_op;
    ok = ok && f.size_order()  == 0;
    //
    // compute zero order Taylor coefficients
    vec_double y  = f.forward(0, x);
    ok = ok && f.size_order() == 1;
```

(continues on next page)

(continued from previous page)

```
    //
    // create an a_fun object
    a_fun af(f);
    //
    // ----------------------------------------------------------------------
    // check af properties
    ok = ok && af.size_domain() == n_ind;
    ok = ok && af.size_range()  == n_dep;
    ok = ok && af.size_var()    == n_var;
    ok = ok && af.size_op()     == n_op;
    ok = ok && af.size_order()  == 0;
    // ----------------------------------------------------------------------
    // The empty function
    ax.resize(0);
    ay.resize(0);
    f = d_fun(ax, ay);
    ok = ok && f.size_domain() == 0;
    ok = ok && f.size_range()  == 0;
    ok = ok && f.size_var()    == 0;
    ok = ok && f.size_op()     == 0;
    ok = ok && f.size_order()  == 0;
    //
    return( ok  );
}
```

---

xsrst input file: `example/cplusplus/fun_property_xam.cpp`

Example

fun_property_xam_cpp

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_new_dynamic

3.2.3.5 Change The Dynamic Parameters

- *Syntax*
- *f*
- *dynamic*
- *size_order*
- *Example*

Syntax

f . `new_dynamic` (*dynamic*)

f

This is either a *d_fun* or *a_fun* function object.

dynamic

If *f* is a `d_fun` or `a_fun` , this argument has prototype

```
const vec_double& dynamic
const vec_a_double& dynamic
```

and its size must be the same as the size of *dynamic* in the corresponding call to `independent` . It specifies new values for the dynamic parameters in *f* .

size_order

After this call *f_size_order()* is zero.

Example

See *fun_dynamic_xam_cpp*.

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_jacobian

3.2.3.6 Jacobian of an AD Function

- *Syntax*
- *f*
- *f(x)*
- *x*
- *J*
- *Example*

Syntax

J = f . `jacobian` (x)

f

This is either a *d_fun* or *a_fun* function object. Upon return, the zero order *taylor_coefficients* in f correspond to the value of x . The other Taylor coefficients in f are unspecified.

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to f . Note that n is the size of *ax* and m is the size of *ay* in to the constructor for f .

x

If f is a `d_fun` or `a_fun` , this argument has prototype

```
const vec_double& x
const vec_a_double& x
```

and its size must be n . It specifies the argument value at we are computing the Jacobian $f'(x)$.

J

If f is a `d_fun` or `a_fun` , the result has prototype

```
vec_double J
vec_a_double J
```

respectively and its size is $m * n$. For i between zero and m -1 and j between zero and n -1 ,

$$J[i * n + j] = \frac{\partial f_i}{\partial x_j}(x)$$

fun_jacobian_xam_cpp

3.2.3.6.1 C++: Dense Jacobian Using AD: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_jacobian_xam(void) {
```

(continues on next page)

(continued from previous page)

```
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
    // initialize return variable
    bool ok = true;
    //------------------------------------------------------------------------
    // number of dependent and independent variables
    int n_dep = 1;
    int n_ind = 3;
    //
    // create the independent variables ax
    vec_double x(n_ind);
    for(int i = 0; i < n_ind ; i++) {
        x[i] = i + 2.0;
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // create dependent variables ay with ay0 = ax_0 * ax_1 * ax_2
    a_double ax_0 = ax[0];
    a_double ax_1 = ax[1];
    a_double ax_2 = ax[2];
    vec_a_double ay(n_dep);
    ay[0] = ax_0 * ax_1 * ax_2;
    //
    // define af corresponding to f(x) = x_0 * x_1 * x_2
    d_fun f(ax, ay);
    //
    // compute the Jacobian f'(x) = ( x_1*x_2, x_0*x_2, x_0*x_1 )
    vec_double fp = f.jacobian(x);
    //
    // check Jacobian
    double x_0 = x[0];
    double x_1 = x[1];
    double x_2 = x[2];
    ok = ok && fp[0 * n_ind + 0] == x_1 * x_2 ;
    ok = ok && fp[0 * n_ind + 1] == x_0 * x_2 ;
    ok = ok && fp[0 * n_ind + 2] == x_0 * x_1 ;
    //------------------------------------------------------------------------
    a_fun af(f);
    //
    // compute the Jacobian f'(x) = ( x_1*x_2, x_0*x_2, x_0*x_1 )
    vec_a_double afp = af.jacobian(ax);
    //
    // check Jacobian
    ok = ok && afp[0 * n_ind + 0] == x_1 * x_2 ;
    ok = ok && afp[0 * n_ind + 1] == x_0 * x_2 ;
    ok = ok && afp[0 * n_ind + 2] == x_0 * x_1 ;
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/fun_jacobian_xam.cpp`

Example

fun_jacobian_xam_cpp

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_hessian

3.2.3.7 Hessian of an AD Function

- *Syntax*
- *f*
- *f(x)*
- *g(x)*
- *x*
- *w*
- *H*
- *Example*

Syntax

H = *f* . `hessian` (*x* , *w*)

f

This is either a *d_fun* or *a_fun* function object. Upon return, the zero order *taylor_coefficients* in *f* correspond to the value of *x* . The other Taylor coefficients in *f* are unspecified.

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

g(x)

We use the notation $g : \mathbf{R}^n \to \mathbf{R}$ for the function defined by

$$g(x) = \sum_{i=0}^{n-1} w_i f_i(x)$$

x

If *f* is a `d_fun` or `a_fun` , this argument has prototype

```
const vec_double& x
const vec_a_double& x
```

and its size must be *n* . It specifies the argument value at we are computing the Hessian $g^{(2)}(x)$.

w

If *f* is a `d_fun` or `a_fun` , this argument has prototype

```
const vec_double& w
const vec_a_double& w
```

and its size must be *m* . It specifies the vector *w* in the definition of $g(x)$ above.

H

If *f* is a `d_fun` or `a_fun` , the result has prototype

```
vec_double H
vec_a_double H
```

and its size is $n * n$. For *i* between zero and *n* -1 and *j* between zero and *n* -1 ,

$$H[i * n + j] = \frac{\partial^2 g}{\partial x_i \partial x_j}(x)$$

fun_hessian_xam_cpp

3.2.3.7.1 C++: Dense Hessian Using AD: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_hessian_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
```

(continues on next page)

(continued from previous page)

```
    // initialize return variable
    bool ok = true;
    // ----------------------------------------------------------------------
    // number of dependent and independent variables
    int n_dep = 1;
    int n_ind = 3;
    //
    // create the independent variables ax
    vec_double x(n_ind);
    for(int i = 0; i < n_ind ; i++) {
        x[i] = i + 2.0;
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // create dependent variables ay with ay0 = ax_0 * ax_1 * ax_2
    a_double ax_0 = ax[0];
    a_double ax_1 = ax[1];
    a_double ax_2 = ax[2];
    vec_a_double ay(n_dep);
    ay[0] = ax_0 * ax_1 * ax_2;
    //
    // define af corresponding to f(x) = x_0 * x_1 * x_2
    d_fun f(ax, ay);
    //
    // g(x) = w_0 * f_0 (x) = f(x)
    vec_double w(n_dep);
    w[0] = 1.0;
    //
    // compute Hessian
    vec_double fpp = f.hessian(x, w);
    //
    //          [ 0.0 , x_2 , x_1 ]
    // f''(x) = [ x_2 , 0.0 , x_0 ]
    //          [ x_1 , x_0 , 0.0 ]
    ok = ok && fpp[0 * n_ind + 0] == 0.0 ;
    ok = ok && fpp[0 * n_ind + 1] == x[2] ;
    ok = ok && fpp[0 * n_ind + 2] == x[1] ;
    //
    ok = ok && fpp[1 * n_ind + 0] == x[2] ;
    ok = ok && fpp[1 * n_ind + 1] == 0.0 ;
    ok = ok && fpp[1 * n_ind + 2] == x[0] ;
    //
    ok = ok && fpp[2 * n_ind + 0] == x[1] ;
    ok = ok && fpp[2 * n_ind + 1] == x[0] ;
    ok = ok && fpp[2 * n_ind + 2] == 0.0 ;
    // ----------------------------------------------------------------------
    a_fun af(f);
    //
    // compute and check Hessian
    vec_a_double aw(n_dep);
    aw[0] = w[0];
    vec_a_double afpp = af.hessian(ax, aw);
    ok = ok && afpp.size() == fpp.size();
    for(size_t i = 0; i < fpp.size(); ++i)
        ok = ok && afpp[i] == fpp[i];
    //
    return( ok );
```

(continues on next page)

(continued from previous page)

```
}
```

---

xsrst input file: `example/cplusplus/fun_hessian_xam.cpp`

Example

fun_hessian_xam_cpp

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_forward

3.2.3.8 Forward Mode AD

- *Syntax*
- *Taylor Coefficient*
- *f*
- *f(x)*
- *X(t)*
- *Y(t)*
- ***p***
  - *size_order*
- *xp*
- *yp*
- *Example*

Syntax

yp = *f* . `forward` (*p* , *xp*)

Taylor Coefficient

For a function $g(t)$ of a scalar argument $t \in \mathbf{R}$, the p-th order Taylor coefficient is its p -th order derivative divided by p factorial and evaluated at $t = 0$; i.e.,

$$g^{(p)}(0)/p!$$

f

This is either a *d_fun* or *a_fun* function object. Note that its state is changed by this operation because all the Taylor coefficient that it calculates for every variable in recording are stored. See more discussion of this fact under the heading *p* below.

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

X(t)

We use the notation $X : \mathbf{R} \to \mathbf{R}^n$ for a function that the calling routine chooses.

Y(t)

We define the function $Y : \mathbf{R} \to \mathbf{R}^n$ by $Y(t) = f(X(t))$.

p

This argument has prototype

```
int p
```

and is non-negative. It is the order of the Taylor coefficient being calculated. If there was no call to `forward` for this *f* , the value of *p* must be zero. Otherwise, it must be between zero and one greater that its value for the previous call using this *f* . After this call, the Taylor coefficients for orders zero though *p* , and for every variable in the recording, will be stored in *f* .

size_order

After this call, *f_size_order()* is *p* +1 .

xp

If *f* is a `d_fun` or `a_fun` , this argument has prototype

```
const vec_double& xp
const vec_a_double& xp
```

respectively and its size must be *n* . It specifies the *p*-th order Taylor coefficients for *X(t*) .

yp

If f is a `d_fun` or `a_fun` , the result has prototype

```
vec_double& yp
vec_a_double& yp
```

respectively and its size is m . It is the p-th order Taylor coefficients for $Y(t)$.

fun_forward_xam_cpp

3.2.3.8.1 C++: Forward Mode AD: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_forward_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
    // initialize return variable
    bool ok = true;
    // ----------------------------------------------------------------------
    // number of dependent and independent variables
    int n_dep = 1;
    int n_ind = 2;
    //
    // create the independent variables ax
    vec_double xp(n_ind);
    for(int i = 0; i < n_ind ; i++) {
        xp[i] = i + 1.0;
    }
    vec_a_double ax = cppad_py::independent(xp);
    //
    // create dependent varialbes ay with ay0 = ax0 * ax1
    a_double ax0 = ax[0];
    a_double ax1 = ax[1];
    vec_a_double ay(n_dep);
    ay[0] = ax0 * ax1;
    //
    // define af corresponding to f(x) = x0 * x1
    d_fun f(ax, ay);
    ok &= f.size_order() == 0;
    //
    // define X(t) = (3 + t, 2 + t)
    // it follows that Y(t) = f(X(t)) = (3 + t) * (2 + t)
    //
    // Y(0) = 6 and p ! = 1
    int p = 0;
```

(continues on next page)

(continued from previous page)

```
    xp[0] = 3.0;
    xp[1] = 2.0;
    vec_double yp = f.forward(p, xp);
    ok  = ok && yp[0] == 6.0;
    ok &= f.size_order() == 1;
    //
    // first order Taylor coefficients for X(t)
    p = 1;
    xp[0] = 1.0;
    xp[1] = 1.0;
    //
    // first order Taylor coefficient for Y(t)
    // Y'(0) = 3 + 2 = 5 and p ! = 1
    yp = f.forward(p, xp);
    ok = ok && yp[0] == 5.0;
    ok &= f.size_order() == 2;
    //
    // second order Taylor coefficients for X(t)
    p = 2;
    xp[0] = 0.0;
    xp[1] = 0.0;
    //
    // second order Taylor coefficient for Y(t)
    // Y''(0) = 2.0 and p ! = 2
    yp = f.forward(p, xp);
    ok = ok && yp[0] == 1.0;
    ok &= f.size_order() == 3;
    // ------------------------------------------------------------------------
    a_fun af(f);
    ok &= af.size_order() == 0;
    //
    // zero order forward
    vec_a_double axp(n_ind), ayp(n_dep);
    p      = 0;
    axp[0] = 3.0;
    axp[1] = 2.0;
    ayp    = af.forward(p, axp);
    ok     = ok && ayp[0] == 6.0;
    ok    &= af.size_order() == 1;
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/fun_forward_xam.cpp`

Example

fun_forward_xam_cpp

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_reverse

3.2.3.9 Reverse Mode AD

- *Syntax*
- *f*
- ***Notation***
  - *f(x)*
  - *X(t), S*
  - *Y(t), T*
  - *G(T)*
- *q*
- *yq*
- *xq*
- *Example*

Syntax

xq = *f* . `reverse` (*q* , *yq*)

f

This is either a *d_fun* or *a_fun* function object and is effectively `const` . (Some details that are not visible to the user may change.)

Notation

f(x)

We use the notation $f : \mathbf{R}^n \to \mathbf{R}^m$ for the function corresponding to *f* . Note that *n* is the size of *ax* and *m* is the size of *ay* in to the constructor for *f* .

X(t), S

This is the same function as *x(t)* in the previous call to *f* . `forward` . We use $S \in \mathbf{R}^{n \times q}$ to denote the Taylor coefficients of $X(t)$.

Y(t), T

This is the same function as *y(t)* in the previous call to *f* . `forward` . We use $T \in \mathbf{R}^{m \times q}$ to denote the Taylor coefficients of $Y(t)$. We also use the notation $T(S)$ to express the fact that the Taylor coefficients for $Y(t)$ are a function of the Taylor coefficients of $X(t)$.

G(T)

We use the notation $G : \mathbf{R}^{m \times p} \to \mathbf{R}$ for a function that the calling routine chooses.

q

This argument has prototype

```
int q
```

and is positive. It is the number of the Taylor coefficient (for each variable) that we are computing the derivative with respect to. It must be greater than zero, and less than or equal the number of Taylor coefficient stored in *f* ; i.e., *f_size_order()*.

yq

If *f* is a `d_fun` or `a_fun` , this argument has prototype

```
const vec_double& yq
const vec_a_double& yq
```

and its size must be *m* ⋆ *q* . For 0 <= *i* < *m* and 0 <= *k* < *q* , *yq* [*i* ⋆ *q* + *k*] is the partial derivative of $G(T)$ with respect to the *k*-th order Taylor coefficient for the *i*-th component function; i.e., the partial derivative of $G(T)$ w.r.t. $Y_i^{(k)}(t)/k!$.

xq

If f is a `d_fun` or `a_fun` , the result has prototype

```
const vec_double& xq
const vec_a_double& xq
```

respectively and its size is $n * q$. For 0 <= j < n and 0 <= k < q , xq [$j * q + k$] is the partial derivative of $G(T(S))$ with respect to the k-th order Taylor coefficient for the j-th component function; i.e., the partial derivative of $G(T(S))$ w.r.t. $S_j^{(k)}(t)/k!$.

fun_reverse_xam_cpp

3.2.3.9.1 C++: Reverse Mode AD: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_reverse_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    // number of dependent and independent variables
    int n_dep = 1;
    int n_ind = 3;
    //
    // create the independent variables ax
    vec_double xp(n_ind);
    for(int i = 0; i < n_ind ; i++) {
        xp[i] = i;
    }
    vec_a_double ax = cppad_py::independent(xp);
    //
    // create dependent variables ay with ay0 = ax_0 * ax_1 * ax_2
    a_double ax_0 = ax[0];
    a_double ax_1 = ax[1];
    a_double ax_2 = ax[2];
    vec_a_double ay(n_dep);
    ay[0] = ax_0 * ax_1 * ax_2;
    //
    // define af corresponding to f(x) = x_0 * x_1 * x_2
    d_fun f(ax, ay);
    // -----------------------------------------------------------------------
    // define          X(t) = (x_0 + t, x_1 + t, x_2 + t)
    // it follows that Y(t) = f(X(t)) = (x_0 + t) * (x_1 + t) * (x_2 + t)
    // and that       Y'(0) = x_1 * x_2 + x_0 * x_2 + x_0 * x_1
```

(continues on next page)

(continued from previous page)

```
// ----------------------------------------------------------------------
// zero order forward mode
int p = 0;
xp[0] = 2.0;
xp[1] = 3.0;
xp[2] = 4.0;
vec_double yp = f.forward(p, xp);
ok = ok && yp[0] == 24.0;
// ----------------------------------------------------------------------
// first order reverse (derivative of zero order forward)
// define G( Y ) = y_0 = x_0 * x_1 * x_2
int q = 1;
vec_double yq1 = vec_double(n_dep);
yq1[0] = 1.0;
vec_double xq1 = f.reverse(q, yq1);
// partial G w.r.t x_0
ok = ok && xq1[0] == 3.0 * 4.0 ;
// partial G w.r.t x_1
ok = ok && xq1[1] == 2.0 * 4.0 ;
// partial G w.r.t x_2
ok = ok && xq1[2] == 2.0 * 3.0 ;
// ----------------------------------------------------------------------
// first order forward mode
p = 1;
xp[0] = 1.0;
xp[1] = 1.0;
xp[2] = 1.0;
yp = f.forward(p, xp);
ok = ok && yp[0] == 3.0*4.0 + 2.0*4.0 + 2.0*3.0;
// ----------------------------------------------------------------------
// second order reverse (derivative of first order forward)
// define G( y_0^0 , y_0^1 ) = y_0^1
// = x_1^0 * x_2^0  +  x_0^0 * x_2^0  +  x_0^0  *  x_1^0
q = 2;
vec_double yq2 = vec_double(n_dep * q);
yq2[0 * q + 0] = 0.0; // partial of G w.r.t y_0^0
yq2[0 * q + 1] = 1.0; // partial of G w.r.t y_0^1
vec_double xq2 = f.reverse(q, yq2);
// partial G w.r.t x_0^0
ok = ok && xq2[0 * q + 0] == 3.0 + 4.0;
// partial G w.r.t x_1^0
ok = ok && xq2[1 * q + 0] == 2.0 + 4.0;
// partial G w.r.t x_2^0
ok = ok && xq2[2 * q + 0] == 2.0 + 3.0;
// ----------------------------------------------------------------------
a_fun af(f);
ok &= af.size_order() == 0;
//
// zero order forward
vec_a_double axp(n_ind), ayp(n_dep);
p      = 0;
axp[0] = 2.0;
axp[1] = 3.0;
axp[2] = 4.0;
ayp    = af.forward(p, axp);
ok     = ok && ayp[0] == 24.0;
ok    &= af.size_order() == 1;
```

(continues on next page)

(continued from previous page)

```
    //
    // first order reverse
    q = 1;
    vec_a_double ayq1 = vec_a_double(n_dep);
    ayq1[0]           = 1.0;
    vec_a_double axq1 = af.reverse(q, ayq1);
    // partial G w.r.t x_0
    ok = ok && axq1[0] == 3.0 * 4.0;
    // partial G w.r.t x_1
    ok = ok && axq1[1] == 2.0 * 4.0;
    // partial G w.r.t x_2
    ok = ok && axq1[2] == 2.0 * 3.0;
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/fun_reverse_xam.cpp`

Example

fun_reverse_xam_cpp

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_optimize

3.2.3.10 Optimize an AD Function

- *Syntax*
- *Purpose*
- *f*
- *Example*

Syntax

f `.optimize()`

Purpose

This reduces the number of operations (hence to time and memory) used to compute the function stored in f On the other hand, the optimization may take a significant amount of time and memory.

f

This object is a *d_fun*. Optimizing this *f* also optimizes the corresponding *a_fun*.

fun_optimize_xam_cpp

3.2.3.10.1 C++: Optimize an d_fun: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool fun_optimize_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    int n_ind = 1; // number of independent variables
    int n_dep = 1; // number of dependent variables
    int n_var = 1; // phantom variable at address 0
    int n_op = 1;  // special operator at beginning
    //
    // dimension some vectors
    vec_double x(n_ind);
    vec_a_double ay(n_dep);
    //
    // independent variables
    x[0] = 1.0;
    vec_a_double ax = cppad_py::independent(x);
    n_var = n_var + n_ind; // one for each indpendent
    n_op = n_op + n_ind;
    //
    // accumulate summation
    a_double ax0 = ax[0];
    a_double csum(0.0);
    csum = ax0 + ax0 + ax0 + ax0;
    n_var = n_var + 3; // one per + operator
    n_op = n_op + 3;
    //
    // define f(x) = y_0 = csum
    ay[0] = csum;
    d_fun f(ax, ay);
    n_op = n_op + 1; // speical operator at end
    //
    // check number of variables and operators
    ok = ok && f.size_var() == n_var;
    ok = ok && f.size_op() == n_op;
    //
    // optimize
    f.optimize();
    //
    // number of variables and operators has decreased by two
```

(continues on next page)

(continued from previous page)

```
    ok = ok && f.size_var() == n_var-2;
    ok = ok && f.size_op() == n_op-2;
    //
    return( ok  );
}
```

---

xsrst input file: `example/cplusplus/fun_optimize_xam.cpp`

Example

fun_optimize_xam_cpp

---

xsrst input file: `lib/cplusplus/fun.cpp`

cpp_fun_json

3.2.3.11 Json Representation of AD Computational Graph

- *Syntax*
- *f*
- *json*
- *to_json*
- *from_json*
- *Examples*

Syntax

json = *f* . `to_json` ()
f . `from_json` ()

f

This is a *d_fun* object.

json

is a Json representation of the computation graph corresponding to f ; see the CppAD documentation for json_ad_graph.

to_json

In this case, the function object f is `const` and the return value *json* has prototype

`std::string` *json*

from_json

In this case, *json* has prototype

`const std::string&` *json*

and the function f so it corresponds to *json* .

fun_to_json_xam_cpp

3.2.3.11.1 C++: to_json: Example and Test

```
bool to_json_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
    // initialize return variable
    bool ok = true;
    // ----------------------------------------------------------------------
    int n = 1; // number of independent variables
    int m = 2; // number of dependent variables
    //
    // dimension some vectors
    vec_double x(n);
    vec_a_double ay(m);
    //
    // independent variables
    x[0]            = 1.0;
    vec_a_double ax = cppad_py::independent(x);
    //
    // f(x) = [ x0 + x0, sin(x0) ]
    ay[0] = ax[0] + ax[0];
    ay[1] = ax[0].sin();
```

(continues on next page)

(continued from previous page)

```
    d_fun f(ax, ay);
    //
    std::string json = f.to_json();
    size_t pos       = json.find("\"op_code\"");
    size_t start     = json.find(":", pos) + 1;
    size_t end       = json.find(",", pos);
    int op_code      = std::atoi( json.substr(start, end - start).c_str() );
    ok              &= op_code == 1;
    pos              = json.find("\"name\"", pos);
    start            = json.find(":", pos);
    start            = json.find("\"", start + 1) + 1;
    end              = json.find("\"", start + 1);
    std::string name = json.substr(start, end - start);
    ok              &= name == "add" || name == "sin";
    //
    return( ok  );
}
```

---

xsrst input file: `example/cplusplus/fun_json_xam.cpp`

fun_from_json_xam_cpp

3.2.3.11.2 C++: from_json: Example and Test

```
bool from_json_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::a_fun;
    //
    // initialize return variable
    bool ok = true;
    // ----------------------------------------------------------------------
    // AD graph repersentation of f(x) = sin(x) / cos(x)
    //
    // node_1 : x[0]
    // node_2 : sin( x[0] )
    // node_3 : cos( x[0] )
    // node_4 : sin( x[0] ) / cos( x[0] )
    // y[0]   = sin( x[0] ) / cos( x[0] )
    // use single quote to avodi having to escape double quote
    std::string json =
        "{\n"
        "   'function_name' : 'tangent function',\n"
        "   'op_define_vec' : [ 3, [\n"
        "       { 'op_code':1, 'name':'sin', 'n_arg':1 } ,\n"
        "       { 'op_code':2, 'name':'cos', 'n_arg':1 } ,\n"
        "       { 'op_code':3, 'name':'div', 'n_arg':2 } ]\n"
        "   ],\n"
        "   'n_dynamic_ind'  : 0,\n"
        "   'n_variable_ind' : 1,\n"
        "   'constant_vec'   : [ 0, [ ] ],\n"
```

(continues on next page)

(continued from previous page)

```
        "   'op_usage_vec'   : [ 3, [\n"
        "       [ 1, 1 ]   ,\n"  // node_2: sin(x[0])
        "       [ 2, 1 ]   ,\n"  // node_3: cos(x[0])
        "       [ 3, 2, 3] ]\n"  // node_4: sin(x[0]) / cos(x[0])
        "   ],\n"
        "   'dependent_vec' : [ 1, [4] ]\n"
        "}\n";
    // convert the single quote to double quote
    for(size_t i = 0; i < json.size(); ++i)
        if( json[i] == '\'' )
            json[i] = '"';
    //
    // convert json to a fucntion object
    vec_a_double ax(0), ay(0); // construct an empty function
    d_fun f(ax, ay);
    f.from_json(json);
    //
    // compute y = f(x)
    vec_double x(1), y(1);
    x[0]  = 1.0;
    y     = f.forward(0, x);
    //
    // check the function value
    double eps99     = std::numeric_limits<double>::epsilon();
    double check     = std::tan(x[0]);
    double rel_error = y[0] / check - 1.0;
    ok              &= std::fabs( rel_error ) < eps99;
    //
    return ok;
}
```

---

xsrst input file: `example/cplusplus/fun_json_xam.cpp`

Examples

fun_to_json_xam_cpp, *fun_from_json_xam_cpp*.

---

xsrst input file: `lib/cplusplus/fun.cpp`

1. cpp_independent: *3.2.3.1 Declare Independent Variables and Start Recording*
2. cpp_abort_recording: *3.2.3.2 Abort Recording*
3. cpp_fun_ctor: *3.2.3.3 Stop Current Recording and Store Function Object*
4. cpp_fun_property: *3.2.3.4 Properties of a Function Object*
5. cpp_fun_new_dynamic: *3.2.3.5 Change The Dynamic Parameters*
6. cpp_fun_jacobian: *3.2.3.6 Jacobian of an AD Function*
7. cpp_fun_hessian: *3.2.3.7 Hessian of an AD Function*
8. cpp_fun_forward: *3.2.3.8 Forward Mode AD*
9. cpp_fun_reverse: *3.2.3.9 Reverse Mode AD*

10. cpp_fun_optimize: *3.2.3.10 Optimize an AD Function*
11. cpp_fun_json: *3.2.3.11 Json Representation of AD Computational Graph*

---

xsrst input file: `lib/cplusplus/cplusplus.xsrst`

sparse

3.2.4 Cppad Py Sparse Calculation

- *Children*

Children

cpp_sparse_rc

3.2.4.1 Sparsity Patterns

- *Syntax*
- *pattern*
- *nr*
- *nc*
- *nnz*
- *resize*
- ***put***
  - *k*
  - *r*
  - *c*
- *row*
- *col*
- *row_major*
- *col_major*
- *Example*

Syntax

pattern = `cppad_py::sparse_rc` ()
pattern . `resize` (*nr* , *nc* , *nnz*)
nr = *pattern* . `nr` ()
nc = *pattern* . `nc` ()
nnz = *pattern* . `nnz` ()
pattern . `put` (*k* , *r* , *c*)
row = *pattern* . `row` ()
col = *pattern* . `col` ()
row_major = *pattern* . `row_major` ()
col_major = *pattern* . `col_major` ()

pattern

This result has prototype

`sparse_rc` *pattern*

It is used to hold a sparsity pattern for a matrix. The sparsity *pattern* is `const` except during the `resize` and `put` operations.

nr

This argument has prototype

`int` *nr*

is non-negative, and specifies the number of rows in the sparsity pattern. The function `nr()` returns the value of *nr* in the previous `resize` operation.

nc

This argument has prototype

`int` *nc*

is non-negative, and specifies the number of columns in the sparsity pattern. The function `nc()` returns the value of *nc* in the previous `resize` operation.

nnz

This argument and result has prototype

`int` *nnz*

It is the number of possibly non-zero index pairs in the sparsity pattern. The function `nnz()` returns the value of *nnz* in the previous `resize` operation.

resize

The current sparsity pattern is lost and a new one is started with the specified parameters. After each `resize` , the elements in the *row* and *col* vectors should be assigned using `put` .

put

This function sets the values

row [*k*] = *r*
col [*k*] = *c*

(The name `set` is used by Cppad, but not used here, because `set` it is a built-in name in Python.)

k

This argument has type

`int` *k*

is non-negative, and must be less than *nnz* .

r

This argument has type

`int` *r*

is non-negative, and must be less than *nr* . It specifies the value assigned to *row* [*k*] and must

c

This argument has type

```
int c
```

It specifies the value assigned to *col* [*k*] and must be less than *nc* .

row

This result has type

```
vec_int row
```

and its size is *nnz* . For *k* = 0, . . . , *nnz* -1 , *row* [*k*] is the row index for the *k*-th possibly non-zero entry in the matrix.

col

This result has type

```
vec_int col
```

and its size is *nnz* . For *k* = 0, . . . , *nnz* -1 , *col* [*k*] is the column index for the *k*-th possibly non-zero entry in the matrix.

row_major

This vector has prototype

```
vec_int row_major
```

and its size *nnz* . It sorts the sparsity pattern in row-major order. To be specific,

col [*row_major* [*k*]] <= *col* [*row_major* [*k* +1]]

and if *col* [*row_major* [*k*]] == *col* [*row_major* [*k* +1]] ,

row [*row_major* [*k*]] < *row* [*row_major* [*k* +1]]

This routine generates an assert if there are two entries with the same row and column values (if NDEBUG is not defined).

col_major

This vector has prototype

vec_int *col_major*

and its size *nnz* . It sorts the sparsity pattern in column-major order. To be specific,

row [*col_major* [*k*]] <= *row* [*col_major* [*k* +1]]

and if *row* [*col_major* [*k*]] == *row* [*col_major* [*k* +1]] ,

col [*col_major* [*k*]] < *col* [*col_major* [*k* +1]]

This routine generates an assert if there are two entries with the same row and column values (if NDEBUG is not defined).

sparse_rc_xam_cpp

3.2.4.1.1 C++: Sparsity Patterns: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool sparse_rc_xam(void) {
    using cppad_py::vec_int;
    using cppad_py::sparse_rc;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    //
    // create a sparsity pattern
    sparse_rc pattern = sparse_rc();
    //
    int nr = 6;
    int nc = 5;
    int nnz = 4;
    //
    // resize
    pattern.resize(nr, nc, nnz);
    //
    ok = ok && pattern.nr()  == nr ;
```

(continues on next page)

(continued from previous page)

```
    ok = ok && pattern.nc()  == nc ;
    ok = ok && pattern.nnz() == nnz ;
    //
    // indices corresponding to upper-diagonal
    for(int k = 0; k < nnz; k++) {
        pattern.put(k, k, k+1);
    }
    //
    // row and column indices
    vec_int row = pattern.row();
    vec_int col = pattern.col();
    //
    // check row and column indices
    for(int k = 0; k < nnz; k++) {
        ok = ok && row[k] == k;
        ok = ok && col[k] == k+1;
    }
    //
    // For this case, row_major and col_major order are same as original order
    vec_int row_major = pattern.row_major();
    vec_int col_major = pattern.col_major();
    for(int k = 0; k < nnz; k++) {
        ok = ok && row_major[k] == k;
        ok = ok && col_major[k] == k;
    }
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/sparse_rc_xam.cpp`

Example

sparse_rc_xam_cpp

---

xsrst input file: `lib/cplusplus/sparse.cpp`

cpp_sparse_rcv

3.2.4.2 Sparse Matrices

- *Syntax*
- *pattern*
- *matrix*
- *nr*
- *nc*
- *nnz*

- ***put***
  - *k*
  - *v*
- *row*
- *col*
- *val*
- *row_major*
- *col_major*
- *Example*

Syntax

matrix = `cppad_py::sparse_rcv` (*pattern*)
nr = *matrix* . `nr` ()
nc = *matrix* . `nc` ()
nnz = `matrix` . *nnz* ()
matrix . `put` (*k* , *v*)
row = *matrix* . `row` ()
col = *matrix* . `col` ()
val = *matrix* . `val` ()
row_major = *matrix* . `row_major` ()
col_major = *matrix* . `col_major` ()

pattern

This argument has prototype

```
const sparse_rc& pattern
```

It specifies the number of rows, number of columns and the possibly non-zero entries in the *matrix* .

matrix

This is a sparse matrix object with the sparsity specified by *pattern* . Only the *val* vector can be changed. All other values returned by *matrix* are fixed during the constructor and constant there after. The *val* vector is only changed by the constructor and the `set` function.

nr

This return value has prototype

`int` *nr*

and is the number of rows in the matrix.

nc

This return value has prototype

`int` *nc*

and is the number of columns in the matrix.

nnz

This return value has prototype

`int` *nnz*

and is the number of possibly non-zero values in the matrix.

put

This function sets the value

val [*k*] = *v*

(The name `set` is used by Cppad, but not used here, because `set` it is a built-in name in Python.)

k

This argument has type

`int` *k*

and must be non-negative and less than *nnz* .

v

This argument has type

```
double v
```

It specifies the value assigned to *val* [*k*] .

row

This result has type

```
vec_int row
```

and its size is *nnz* . For *k* = 0, . . . , *nnz* -1 , *row* [*k*] is the row index for the *k*-th possibly non-zero entry in the matrix.

col

This result has type

```
vec_int col
```

and its size is *nnz* . For *k* = 0, . . . , *nnz* -1 , *col* [*k*] is the column index for the *k*-th possibly non-zero entry in the matrix.

val

This result has type

```
vec_double val
```

and its size is *nnz* . For *k* = 0, . . . , *nnz* -1 , *val* [*k*] is the value of the *k*-th possibly non-zero entry in the matrix (the value may be zero).

row_major

This vector has prototype

`vec_int` *row_major*

and its size *nnz* . It sorts the sparsity pattern in row-major order. To be specific,

col [*row_major* [*k*]] <= *col* [*row_major* [*k* +1]]

and if *col* [*row_major* [*k*]] == *col* [*row_major* [*k* +1]] ,

row [*row_major* [*k*]] < *row* [*row_major* [*k* +1]]

This routine generates an assert if there are two entries with the same row and column values (if `NDEBUG` is not defined).

col_major

This vector has prototype

`vec_int` *col_major*

and its size *nnz* . It sorts the sparsity pattern in column-major order. To be specific,

row [*col_major* [*k*]] <= *row* [*col_major* [*k* +1]]

and if *row* [*col_major* [*k*]] == *row* [*col_major* [*k* +1]] ,

col [*col_major* [*k*]] < *col* [*col_major* [*k* +1]]

This routine generates an assert if there are two entries with the same row and column values (if `NDEBUG` is not defined).

sparse_rcv_xam_cpp

3.2.4.2.1 C++: Sparsity Patterns: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool sparse_rcv_xam(void) {
    using cppad_py::vec_int;
    using cppad_py::vec_double;
    using cppad_py::sparse_rc;
    using cppad_py::sparse_rcv;
    //
    // initialize return variable
    bool ok = true;
    //----------------------------------------------------------------------
    //
    // create sparsity pattern for n by n identity matrix
    sparse_rc pattern = sparse_rc();
    int n = 5;
    pattern.resize(n, n, n);
    for(int k = 0; k < n; k++) {
        pattern.put(k, k, k);
    }
    //
    // create n by n sparse representation of identity matrix
    sparse_rcv matrix = sparse_rcv(pattern);
    for(int k = 0; k < n; k++) {
        matrix.put(k, 1.0);
    }
    //
    // row, column indices
    vec_int row = matrix.row();
    vec_int col = matrix.col();
    vec_double val = matrix.val();
    //
    // check results
    for(int k = 0; k < n; k++) {
        ok = ok && row[k] == k;
        ok = ok && col[k] == k;
        ok = ok && val[k] == 1.0;
    }
    //
    // For this case, row_major and col_major order are same as original order
    vec_int row_major = matrix.row_major();
    vec_int col_major = matrix.col_major();
    for(int k = 0; k < n; k++) {
        ok = ok && row_major[k] == k;
        ok = ok && col_major[k] == k;
    }
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/sparse_rcv_xam.cpp`

Example

sparse_rcv_xam_cpp

---

xsrst input file: `lib/cplusplus/sparse.cpp`

cpp_jac_sparsity

3.2.4.3 Jacobian Sparsity Patterns

- *Syntax*
- ***Purpose***
  - *for_jac_sparsity*
  - *rev_jac_sparsity*
- *x*
- *f*
- *pattern_in*
- *pattern_out*
- *Sparsity for Entire Jacobian*
- *Example*

Syntax

f . `for_jac_sparsity` (*pattern_in* , *pattern_out*)
f . `rev_jac_sparsity` (*pattern_in* , *pattern_out*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to the operation sequence stored in *f* .

for_jac_sparsity

Fix $R \in \mathbf{R}^{n \times \ell}$ and define the function

$$J(x) = F^{(1)}(x) * R$$

Given a sparsity pattern for R, `for_jac_sparsity` computes a sparsity pattern for $J(x)$.

rev_jac_sparsity

Fix $R \in \mathbf{R}^{\ell \times m}$ and define the function

$$J(x) = R * F^{(1)}(x)$$

Given a sparsity pattern for R, `rev_jac_sparsity` computes a sparsity pattern for $J(x)$.

x

Note that a sparsity pattern for $J(x)$ corresponds to the operation sequence stored in f and does not depend on the argument x .

f

The object f has prototype

`d_fun` f

The object f is not `const` when using `for_jac_sparsity` . After a call to `for_jac_sparsity` , a sparsity pattern for each of the variables in the operation sequence is held in f for possible later use during reverse Hessian sparsity calculations.

pattern_in

The argument *pattern_in* has prototype

`const sparse_rc&` *pattern_in*

see *cpp_sparse_rc*. This is a sparsity pattern for R.

pattern_out

This argument has prototype

`sparse_rc` < *SizeVector* > `&` *pattern_out*

This input value of *pattern_out* does not matter. Upon return *pattern_out* is a sparsity pattern for $J(x)$.

Sparsity for Entire Jacobian

Suppose that R is the identity matrix. In this case, *pattern_out* is a sparsity pattern for $F^{(1)}(x)$.

sparse_jac_pattern_xam_cpp

3.2.4.3.1 C++: Jacobian Sparsity Patterns: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool sparse_jac_pattern_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_int;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::sparse_rc;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    // number of dependent and independent variables
    int n = 3;
    //
    // create the independent variables ax
    vec_double x(n);
    for(int i = 0; i < n ; i++) {
        x[i] = i + 2.0;
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // create dependent variables ay with ay[i] = ax[j]
    // where i = mod(j + 1, n)
    vec_a_double ay(n);
    for(int j = 0; j < n ; j++) {
        int i = j+1;
        if( i >= n  ) {
            i = i - n;
        }
        a_double ay_i = ax[j];
        ay[i] = ay_i;
    }
    //
    // define af corresponding to f(x)
    d_fun f(ax, ay);
    //
    // sparsity pattern for identity matrix
    sparse_rc pat_in = sparse_rc();
    pat_in.resize(n, n, n);
    for(int k = 0; k < n; k++) {
        pat_in.put(k, k, k);
    }
    //
    // loop over forward and reverse mode
```

(continues on next page)

(continued from previous page)

```
    for(int mode = 0; mode < 2; mode++) {
        sparse_rc pat_out = sparse_rc();
        if( mode == 0  ) {
            f.for_jac_sparsity(pat_in, pat_out);
        }
        if( mode == 1  ) {
            f.rev_jac_sparsity(pat_in, pat_out);
        }
        //
        // check that result is sparsity pattern for Jacobian
        ok = ok && n == pat_out.nnz();
        vec_int col_major = pat_out.col_major();
        vec_int row = pat_out.row();
        vec_int col = pat_out.col();
        for(int k = 0; k < n; k++) {
            int ell = col_major[k];
            int r = row[ell];
            int c = col[ell];
            int i = c+1;
            if( i >=  n  ) {
                i = i - n;
            }
            ok = ok && c == k;
            ok = ok && r == i;
        }
    }
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/sparse_jac_pattern_xam.cpp`

Example

c++

---

xsrst input file: `lib/cplusplus/sparse.cpp`

cpp_sparsity

3.2.4.4 Hessian Sparsity Patterns

- *Syntax*
- *Purpose*
- *x*
- *f*
- *select_domain*
- *select_range*

- *pattern_out*
- *Sparsity for Component Wise Hessian*
- *Example*

Syntax

f . `for_hes_sparsity` (*select_domain* , *select_range* , *pattern_out*)
f . `rev_hes_sparsity` (*select_domain* , *select_range* , *pattern_out*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to the operation sequence stored in *f* . Fix a diagonal matrix $D \in \mathbf{R}^{n \times n}$, fix a vector $r \in \mathbf{R}^m$, and define

$$H(x) = D(r^{\mathrm{T}} F)^{(2)}(x) D$$

Given a sparsity pattern for D and r, these routines compute a sparsity pattern for $H(x)$.

x

Note that a sparsity pattern for $H(x)$ corresponds to the operation sequence stored in *f* and does not depend on the argument *x* .

f

The object *f* has prototype

`d_fun` *f*

select_domain

The argument *select_domain* has prototype

`const vec_bool&` *select_domain*

It has size *n* and is a sparsity pattern for the diagonal of D; i.e., *select_domain* [*j*] is true if and only if $D_{j,j}$ is possibly non-zero.

select_range

The argument *select_range* has prototype

`const vec_bool&` *select_range*

It has size *m* and is a sparsity pattern for the vector r; i.e., *select_range* [*i*] is true if and only if r_i is possibly non-zero.

pattern_out

This argument has prototype

`sparse_rc` < *SizeVector* > `&` *pattern_out*

This input value of *pattern_out* does not matter. Upon return *pattern_out* is a sparsity pattern for $J(x)$.

Sparsity for Component Wise Hessian

Suppose that D is the identity matrix, and only the *i*-th component of *r* is possibly non-zero. In this case, *pattern_out* is a sparsity pattern for $F_i^{(2)}(x)$.

sparse_hes_pattern_xam_cpp

3.2.4.4.1 C++: Hessian Sparsity Patterns: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool sparse_hes_pattern_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_bool;
    using cppad_py::vec_int;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::sparse_rc;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    // number of dependent and independent variables
    int n = 3;
    //
    // create the independent variables ax
    vec_double x(n);
    for(int i = 0; i < n ; i++) {
        x[i] = i + 2.0;
```

(continues on next page)

(continued from previous page)

```
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // create dependent variables ay with ay[i] = ax[j] * ax[i]
    // where i = mod(j + 1, n)
    vec_a_double ay(n);
    for(int j = 0; j < n ; j++) {
        int i = j+1;
        if( i >= n  ) {
            i = i - n;
        }
        a_double ay_i = ax[i] * ax[j];
        ay[i] = ay_i;
    }
    //
    // define af corresponding to f(x)
    d_fun f(ax, ay);
    //
    // Set select_d (domain) to all true, initial select_r (range) to all false
    vec_bool select_d = vec_bool(n);
    vec_bool select_r = vec_bool(n);
    for(int i = 0; i < n; i++) {
        select_d[i] = true;
        select_r[i] = false;
    }
    //
    // only select component 0 of the range function
    // f_0 (x) = x_0 * x_{n-1}
    select_r[0] = true;
    //
    // loop over forward and reverse mode
    for(int mode = 0; mode < 2; mode++) {
        sparse_rc pat_out = sparse_rc();
        if( mode == 0  ) {
            f.for_hes_sparsity(select_d, select_r, pat_out);
        }
        if( mode == 1  ) {
            f.rev_hes_sparsity(select_d, select_r, pat_out);
        }
        //
        // check that result is sparsity pattern for Hessian of f_0 (x)
        ok = ok && pat_out.nnz() == 2 ;
        vec_int row = pat_out.row();
        vec_int col = pat_out.col();
        for(int k = 0; k < 2; k++) {
            int r = row[k];
            int c = col[k];
            if( r <= c  ) {
                ok = ok && r == 0;
                ok = ok && c == n-1;
            }
            if( r >= c  ) {
                ok = ok && r == n-1;
                ok = ok && c == 0;
            }
        }
    }
```

(continues on next page)

(continued from previous page)

```
    //
    return( ok );
}
```

---

xsrst input file: `example/cplusplus/sparse_hes_pattern_xam.cpp`

Example

c++

---

xsrst input file: `lib/cplusplus/sparse.cpp`

cpp_sparse_jac

3.2.4.5 Computing Sparse Jacobians

- *Syntax*
- *Purpose*
- *sparse_jac_for*
- *sparse_jac_rev*
- *f*
- *subset*
- *x*
- *pattern*
- *work*
- *n_sweep*
- *Uses Forward*
- *Example*

Syntax

work = `cppad_py::sparse_jac_work` ()
n_sweep = *f* . `sparse_jac_for` (*subset* , *x* , *pattern* , *work*)
n_sweep = *f* . `sparse_jac_rev` (*subset* , *x* , *pattern* , *work*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to *f* . The syntax above takes advantage of sparsity when computing the Jacobian

$$J(x) = F^{(1)}(x)$$

In the sparse case, this should be faster and take less memory than *cpp_fun_jacobian*. We use the notation $J_{i,j}(x)$ to denote the partial of $F_i(x)$ with respect to x_j.

sparse_jac_for

This function uses first order forward mode sweeps *cpp_fun_forward* to compute multiple columns of the Jacobian at the same time.

sparse_jac_rev

This function uses first order reverse mode sweeps *cpp_fun_reverse* to compute multiple rows of the Jacobian at the same time.

f

This object has prototype

`ADFun` < *Base* > *f*

Note that the Taylor coefficients stored in *f* are affected by this operation; see *uses_forward* below.

subset

This argument has prototype

`sparse_rcv&` *subset*

Its row size is *subset* . `nr` () == *m* , and its column size is *subset* . `nc` () == *n* . It specifies which elements of the Jacobian are computed. The input value of its value vector *subset* . `val` () does not matter. Upon return it contains the value of the corresponding elements of the Jacobian. All of the row, column pairs in *subset* must also appear in *pattern* ; i.e., they must be possibly non-zero.

x

This argument has prototype

```
const vec_double& x
```

and its size is *n* . It specifies the point at which to evaluate the Jacobian $J(x)$.

pattern

This argument has prototype

```
const sparse_rc& pattern
```

Its row size is *pattern* . `nr` () == *m* , and its column size is *pattern* . `nc` () == *n* . It is a sparsity pattern for the Jacobian $J(x)$. This argument is not used (and need not satisfy any conditions), when *work* is non-empty.

work

This argument has prototype

```
sparse_jac_work& work
```

We refer to its initial value, and its value after *work* . `clear` () , as empty. If it is empty, information is stored in *work* . This can be used to reduce computation when a future call is for the same object *f* , the same member function `sparse_jac_for` or `sparse_jac_rev` , and the same subset of the Jacobian. If any of these values change, use *work* . `clear` () to empty this structure.

n_sweep

The return value *n_sweep* has prototype

```
int n_sweep
```

If `sparse_jac_for` (`sparse_jac_rev`) is used, *n_sweep* is the number of first order forward (reverse) sweeps used to compute the requested Jacobian values. This is proportional to the total computational work, not counting the zero order forward sweep, or combining multiple columns (rows) into a single sweep.

Uses Forward

After each call to *cpp_fun_forward*, the object f contains the corresponding Taylor coefficients for all the variables in the operation sequence.. After a call to `sparse_jac_forward` or `sparse_jac_rev` , the zero order coefficients correspond to

f `.forward(0,`x`)`

All the other forward mode coefficients are unspecified.

sparse_jac_xam_cpp

3.2.4.5.1 C++: Computing Sparse Jacobians: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool sparse_jac_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_int;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::sparse_rc;
    using cppad_py::sparse_rcv;
    using cppad_py::sparse_jac_work;
    //
    // initialize return variable
    bool ok = true;
    //----------------------------------------------------------------------
    // number of dependent and independent variables
    int n = 3;
    // one
    a_double aone(1.0);
    //
    // create the independent variables ax
    vec_double x(n);
    for(int i = 0; i < n ; i++) {
        x[i] = i + 2.0;
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // create dependent variables ay with ay[i] = (j+1) * ax[j]
    // where i = mod(j + 1, n)
    vec_a_double ay(n);
    for(int j = 0; j < n ; j++) {
        int i = j+1;
        if( i >= n  ) {
            i = i - n;
        }
        a_double aj(j);
        a_double ay_i = (aj + aone) * ax[j];
```

(continues on next page)

(continued from previous page)

```
        ay[i] = ay_i;
    }
    //
    // define af corresponding to f(x)
    d_fun f(ax, ay);
    //
    // sparsity pattern for identity matrix
    sparse_rc pat_eye = sparse_rc();
    pat_eye.resize(n, n, n);
    for(int k = 0; k < n; k++) {
        pat_eye.put(k, k, k);
    }
    //
    // sparsity pattern for the Jacobian
    sparse_rc pat_jac = sparse_rc();
    f.for_jac_sparsity(pat_eye, pat_jac);
    //
    // loop over forward and reverse mode
    for(int mode = 0; mode < 2; mode++) {
        // compute all possibly non-zero entries in Jacobian
        sparse_rcv subset = sparse_rcv(pat_jac);
        // work space used to save time for multiple calls
        sparse_jac_work work = sparse_jac_work();
        if( mode == 0  ) {
            f.sparse_jac_for(subset, x, pat_jac, work);
        }
        if( mode == 1  ) {
            f.sparse_jac_rev(subset, x, pat_jac, work);
        }
        //
        // check result
        ok = ok && n == subset.nnz();
        vec_int col_major = subset.col_major();
        vec_int row = subset.row();
        vec_int col = subset.col();
        vec_double val = subset.val();
        for(int k = 0; k < n; k++) {
            int ell = col_major[k];
            int r = row[ell];
            int c = col[ell];
            double v = val[ell];
            int i = c+1;
            if( i >=  n  ) {
                i = i - n;
            }
            ok = ok && c == k;
            ok = ok && r == i;
            ok = ok && v == c + 1.0;
        }
    }
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/sparse_jac_xam.cpp`

Example

sparse_jac_xam_cpp

---

xsrst input file: `lib/cplusplus/sparse.cpp`

cpp_sparse_hes

3.2.4.6 Computing Sparse Hessians

- *Syntax*
- *Purpose*
- *f*
- *subset*
- *x*
- *r*
- *pattern*
- *work*
- *n_sweep*
- *Uses Forward*
- *Example*

Syntax

work = `cppad_py::sparse_hes_work` ()
n_sweep = *f* . `sparse_hes` (*subset* , *x* , *r* , *pattern* , *work*)

Purpose

We use $F : \mathbf{R}^n \to \mathbf{R}^m$ to denote the function corresponding to *f* . Given a vector $r \in \mathbf{R}^m$, define

$$H(x) = (r^{\mathrm{T}} F)^{(2)}(x)$$

This routine takes advantage of sparsity when computing elements of the Hessian $H(x)$.

f

This object has prototype

`ADFun` < *Base* > *f*

Note that the Taylor coefficients stored in *f* are affected by this operation; see *uses_forward* below.

subset

This argument has prototype

`sparse_rcv&` *subset*

Its row size and column size is *n* ; i.e., *subset* . `nr` () == *n* and *subset* . `nc` () == *n* . It specifies which elements of the Hessian are computed. The input value of its value vector *subset* . `val` () does not matter. Upon return it contains the value of the corresponding elements of the Jacobian. All of the row, column pairs in *subset* must also appear in *pattern* ; i.e., they must be possibly non-zero.

x

This argument has prototype

`const vec_double&` *x*

and its size is *n* . It specifies the point at which to evaluate the Hessian $H(x)$.

r

This argument has prototype

`const vec_double&` *r*

and its size is *m* . It specifies the multiplier for each component of $F(x)$; i.e., r_i is the multiplier for $F_i(x)$.

pattern

This argument has prototype

```
const sparse_rc& pattern
```

Its row size and column sizes are *n* ; i.e., *pattern* . `nr` () == *n* and *pattern* . `nc` () == *n* . It is a sparsity pattern for the Hessian $H(x)$. This argument is not used (and need not satisfy any conditions), when *work* is non-empty.

work

This argument has prototype

```
sparse_hes_work& work
```

We refer to its initial value, and its value after *work* . `clear` () , as empty. If it is empty, information is stored in *work* . This can be used to reduce computation when a future call is for the same object *f* , and the same subset of the Hessian. If either of these values change, use *work* . `clear` () to empty this structure.

n_sweep

The return value *n_sweep* has prototype

```
int n_sweep
```

It is the number of first order forward sweeps used to compute the requested Hessian values. Each first forward sweep is followed by a second order reverse sweep so it is also the number of reverse sweeps. This is proportional to the total computational work, not counting the zero order forward sweep, or combining multiple columns and rows into a single sweep.

Uses Forward

After each call to *cpp_fun_forward*, the object *f* contains the corresponding Taylor coefficients for all the variables in the operation sequence.. After a call to `sparse_hes` the zero order coefficients correspond to

```
f.forward(0,x)
```

All the other forward mode coefficients are unspecified.

sparse_hes_xam_cpp

3.2.4.6.1 C++: Hessian Sparsity Patterns: Example and Test

```
# include <cstdio>
# include <cppad/py/cppad_py.hpp>

bool sparse_hes_xam(void) {
    using cppad_py::a_double;
    using cppad_py::vec_bool;
    using cppad_py::vec_int;
    using cppad_py::vec_double;
    using cppad_py::vec_a_double;
    using cppad_py::d_fun;
    using cppad_py::sparse_rc;
    using cppad_py::sparse_rcv;
    using cppad_py::sparse_hes_work;
    //
    // initialize return variable
    bool ok = true;
    //-----------------------------------------------------------------------
    // number of dependent and independent variables
    int n = 3;
    //
    // create the independent variables ax
    vec_double x(n);
    for(int i = 0; i < n ; i++) {
        x[i] = i + 2.0;
    }
    vec_a_double ax = cppad_py::independent(x);
    //
    // ay[i] = j * ax[j] * ax[i]
    // where i = mod(j + 1, n)
    vec_a_double ay(n);
    for(int j = 0; j < n ; j++) {
        int i = j+1;
        if( i >= n  ) {
            i = i - n;
        }
        a_double aj(j);
        a_double ax_j = ax[j];
        a_double ax_i = ax[i];
        ay[i] = aj * ax_j * ax_i;
    }
    //
    // define af corresponding to f(x)
    d_fun f(ax, ay);
    //
    // Set select_d (domain) to all true,
    // initial select_r (range) to all false
    // initialize r to all zeros
    vec_bool select_d = vec_bool(n);
    vec_bool select_r = vec_bool(n);
    vec_double r(n);
    for(int i = 0; i < n; i++) {
        select_d[i] = true;
        select_r[i] = false;
```

(continues on next page)

(continued from previous page)

```
        r[i] = 0.0;
    }
    //
    // only select component n-1 of the range function
    // f_0 (x) = (n-1) * x_{n-1} * x_0
    select_r[0] = true;
    r[0] = 1.0;
    //
    // sparisty pattern for Hessian
    sparse_rc pattern = sparse_rc();
    f.for_hes_sparsity(select_d, select_r, pattern);
    //
    // compute all possibly non-zero entries in Hessian
    // (should only compute lower triangle becuase matrix is symmetric)
    sparse_rcv subset = sparse_rcv(pattern);
    //
    // work space used to save time for multiple calls
    sparse_hes_work work = cppad_py::sparse_hes_work();
    //
    f.sparse_hes(subset, x, r, pattern, work);
    //
    // check that result is sparsity pattern for Hessian of f_0 (x)
    ok = ok && subset.nnz() == 2 ;
    vec_int row = subset.row();
    vec_int col = subset.col();
    vec_double val = subset.val();
    for(int k = 0; k < 2; k++) {
        int i = row[k];
        int j = col[k];
        double v = val[k];
        if( i <= j  ) {
            ok = ok && i == 0;
            ok = ok && j == n-1;
        }
        if( i >= j  ) {
            ok = ok && i == n-1;
            ok = ok && j == 0;
        }
        ok = ok && v == n-1;
    }
    //
    return( ok );
}
```

xsrst input file: `example/cplusplus/sparse_hes_xam.cpp`

Example

sparse_hes_xam_cpp

---

xsrst input file: `lib/cplusplus/sparse.cpp`

1. cpp_sparse_rc: *3.2.4.1 Sparsity Patterns*
2. cpp_sparse_rcv: *3.2.4.2 Sparse Matrices*
3. cpp_jac_sparsity: *3.2.4.3 Jacobian Sparsity Patterns*
4. cpp_sparsity: *3.2.4.4 Hessian Sparsity Patterns*
5. cpp_sparse_jac: *3.2.4.5 Computing Sparse Jacobians*
6. cpp_sparse_hes: *3.2.4.6 Computing Sparse Hessians*

---

xsrst input file: `lib/cplusplus/cplusplus.xsrst`

cpp_utility

3.2.5 C++ Utilities

- *Children*

Children

vec2cppad_double

3.2.5.1 Convert an a_double Vector to a CppAD::AD<double> Vector

- *Syntax*
- *Prototype*

Syntax

v_out = `cppad_py::vec2cppad_double(v_in)`

Prototype

```
std::vector< CppAD::AD<double> >
vec2cppad_double(const std::vector<a_double>& v_in )
```

---

xsrst input file: `lib/cplusplus/cppad_vec.cpp`

vec2a_double

3.2.5.2 Convert a CppAD::AD<double> Vector to an a_double Vector

- *Syntax*
- *Prototype*

Syntax

v_out = `cppad_py::vec2a_double ( v_in )`

Prototype

```
std::vector<a_double>
vec2a_double(const std::vector< CppAD::AD<double> >& v_in )
```

---

xsrst input file: `lib/cplusplus/cppad_vec.cpp`

error_message

3.2.5.3 Exception Handling

- *Syntax*
- ***In Try Block***
  - *exception*
- *In Catch Block*
- *Cppad Errors*
- *Not Thread Safe*
- *Example*

Syntax

stored_message = `error_message` (*input_message*)

In Try Block

If *input_message* is **not** the empty string, it is stored in `error_message` and an exception is thrown. This is intended to be done inside a `try` block.

exception

The type of the exception is `std::runtime_error` which is derived from `std::exception` . If the standard exception `what()` is called, the return value will be the value of *input_message* when the exception was thrown.

In Catch Block

If *input_message* is the empty string, the most recently stored message in `error_message` is returned. In addition, the message is deleted from `error_message` . If there are no more messages stored in `error_message` , the empty string is returned. This is intended to be done inside a `catch` block.

Cppad Errors

Calls to the Cppad error handler get mapped to a call to `error_message` .

Not Thread Safe

The message storage is done using static information in `error_message` and hence is not thread safe.

error_message_xam_cpp

3.2.5.3.1 C++: Cppad Py Exception Handling: Example and Test

```
# include <cstdio>
# include <string>
# include <cppad/py/cppad_py.hpp>

bool error_message_xam(void) {
    using std::string;
    //
    // initialize return variable
    bool ok = true;
    //------------------------------------------------------------------------
    ok = false;
    try {
        cppad_py::error_message("test message");
    } catch (...) {
        string stored_message = cppad_py::error_message("");
        ok = stored_message == "test message";
```

(continues on next page)

(continued from previous page)

```
    }
    return( ok  );
}
```

---

xsrst input file: `example/cplusplus/error_message_xam.cpp`

error_message_xam_py

3.2.5.3.2 Python: Cppad Py Exception Handling: Example and Test

```
def error_message_xam() :
    #
    import numpy
    import cppad_py
    #
    # initialize return variable
    ok = True
    # ------------------------------------------------------------------
    ok = False
    try :
        cppad_py.error_message("test message")
    except : # catch
        stored_message = cppad_py.error_message("")
        ok = (stored_message == "test message")
    #
    return( ok  )
#
```

---

xsrst input file: `example/python/core/error_message_xam.py`

Example

c++, *python*.

---

xsrst input file: `lib/cplusplus/error.cpp`

1. vec2cppad_double: *3.2.5.1 Convert an a_double Vector to a CppAD::AD<double> Vector*
2. vec2a_double: *3.2.5.2 Convert a CppAD::AD<double> Vector to an a_double Vector*
3. error_message: *3.2.5.3 Exception Handling*

---

xsrst input file: `lib/cplusplus/cplusplus.xsrst`

more_cpp

3.2.6 Steps To Add More C++ Functions

- *Purpose*
- ***Include File***
  - *independent*
  - *new_dynamic*
- ***Documentation***
  - *independent*
  - *new_dynamic*
  - *Example*
- ***Implementation***
  - *independent*
  - *new_dynamic*
- *Example*
- *Testing*

Purpose

This section outlines the steps for adding more CppAD functionality to cppad_py. This is done by example showing how the *cpp_fun_new_dynamic* was added. This example case was chosen because it required both changing one C++ function, *cpp_independent*, and adding a new C++ function, *cpp_fun_new_dynamic*.

Include File

independent

The include file `include/cppad/py/fun.hpp` was edited to add the following prototype:

```
CPPAD_PY_LIB_PUBLIC
std::vector<a_double> independent(
    const std::vector<double>& x, const std::vector<double>& dynamic
);
```

The `independent` function is not part of the `d_fun` or `a_fun` class, but calling it is the first step in creating these objects. This is why its prototype is in the `fun.hpp` file.

new_dynamic

The include file `include/cppad/py/fun.hpp` was edited to add the following member function to `d_fun` :

```
void new_dynamic(const std::vector<double>& dynamic);
```

The following member function was added to `a_fun` :

```
void new_dynamic(const std::vector<a_double>& adynamic);
```

Documentation

independent

The C++ file `lib/cplusplus/fun.cpp` was edited to add the following syntax documentation:

a_both = `cppad_py::independent` (*x* , *dynamic*)

and the corresponding return value was documented; see *a_both*.

new_dynamic

The *cpp_fun_new_dynamic* documentation was added.

Example

The corresponding example file was added to the documentation, below the *cpp_independent* section, using the OMhelp commands:

```
{xsrst_dollar}children%
    example/cplusplus/fun_dynamic_xam.cpp
%${xsrst_dollar}
```

In addition, a reference to this example was added under the *example* heading in the `independent` documentation.

Implementation

independent

The following function was added to the `lib/cplusplus/fun.cpp` file:

```
std::vector<a_double> independent(
    const std::vector<double>& x       ,
    const std::vector<double>& dynamic )
{   using CppAD::AD;
    size_t nx = x.size();
    size_t nd = dynamic.size();
    CppAD::vector< AD<double> > ax(nx), adynamic(nd);
    for(size_t j = 0; j < nx; j++)
        ax[j] = x[j];
    for(size_t j = 0; j < nd; j++)
        adynamic[j] = dynamic[j];
    size_t abort_op_index = 0;
    size_t record_compare = false;
    CppAD::Independent(ax, abort_op_index, record_compare, adynamic);
    std::vector<a_double> a_both(nx + nd);
    // use a_double( *AD<double> ) constructor in these assignment loops
    for(size_t j = 0; j < nx; j++)
        a_both[j] =  &ax[j] ;
    for(size_t j = 0; j < nd; j++)
        a_both[nx + j] =  &adynamic[j] ;
    return a_both;
}
```

new_dynamic

The following function was added to the `lib/cplusplus/fun.cpp` file:

```
void d_fun::new_dynamic(const std::vector<double>& dynamic)
{   if( dynamic.size() != ptr_->size_dyn_ind() )
        error_message("cppad_py::d_fun::new_dynamic dynamic.size() error");
    ptr_->new_dynamic(dynamic);
    return;
}
void a_fun::new_dynamic(const std::vector<a_double>& adynamic)
{   if( adynamic.size() != a_ptr_->size_dyn_ind() )
        error_message("cppad_py::a_fun::jacobian adynamic.size() error");
    std::vector< CppAD::AD<double> > au = vec2cppad_double(adynamic);
    a_ptr_->new_dynamic(au);
    return;
}
```

A similar function was added to the `a_fun` class.

Example

The file `example/cplusplus/fun_dynamic_xam.cpp` was added with the following contents: *fun_dynamic_xam_cpp*. In addition, in the file `example/cplusplus/check_all.cpp` :

```
extern bool fun_optimize_xam(void);
```

was added to the external declarations and

```
ok &= Run( fun_optimize_xam, "fun_optimize_xam" );
```

was added to the main program.

Testing

You must do a git add for all of the new files before running `bin/check_all.sh` . After all the C++ changes above were implemented, `bin/check_all.sh` was run and the changes were made until the warnings and errors were fixed. The command

```
grep 'fun_dynamic_xam' check_all.log
```

was used to make sure that the new C++ example / test was run. Note that if a particular step in `bin/check_all.sh` is failing, you can just re-run that step to see if a particular fix works. Once the C++ tests were working, the changes where checked into using `git` .

---

xsrst input file: `lib/cplusplus/more_cpp.xsrst`

1. a_double: *3.2.1 Cppad Py AD Scalars*
2. vector: *3.2.2 Cppad Py Vectors*
3. cpp_fun: *3.2.3 Cppad Py AD Functions*
4. sparse: *3.2.4 Cppad Py Sparse Calculation*
5. cpp_utility: *3.2.5 C++ Utilities*
6. more_cpp: *3.2.6 Steps To Add More C++ Functions*

---

xsrst input file: `lib/cplusplus/cpp_lib.xsrst`

1. py_lib: *3.1 The Python Library*
2. cpp_lib: *3.2 The C++ Library*

---

xsrst input file: `lib/library.xsrst`

xsrst_py

4 Extract Sphinx RST

- *Syntax*
- *Purpose*
- *Requirements*
- ***Notation***
  - *White Space*

    - *Beginning of a Line*
- ***Command Line Arguments***
    - ***root_file***
        * *root_section*
    - ***sphinx_dir***
        * *Example Configuration Files*
    - *spelling*
    - *keyword*
    - *line_increment*
- ***Table Of Contents***
    - *toctree*
    - *Parent Section*
- ***Heading Links***
    - *Section Level*
    - *Other Levels*
    - *Children*
    - *Example*
- ***Indentation***
    - *Example*
- ***Wish List***
    - *Tabs*
    - *Module*
- *Commands*

begin_cmd

4.1 Begin and End Commands

- *Syntax*
- *Section*
- *section_name*
- *Output File*
- *Parent Section*

Syntax

- {`xsrst_begin` *section_name* }
- {`xsrst_begin_parent` *section_name* }
- {`xsrst_end` *section_name* }

Section

The start (end) of a section of the input file is indicated by a begin (end) command at the *beginning of a line*.

section_name

The *section_name* is a non-empty sequence of the following characters: a-z, 0-9, and underbar _. It can not begin with the characters `xsrst_`. A link is included in the index under the section name to the first heading the section. The section name is also added to the html keyword meta data.

Output File

The output file corresponding to *section_name* is

sphinx_dir `/xsrst/` *section_name* `.rst`

Parent Section

There can be at most one begin parent command in an input file. In this case there must be other sections in the file and they are children of the parent section. The parent section is a child of the section that included this file using a *child command*.

If there is no parent command in an input file, all the sections in the file are children of the section that included this file using a *child command*.

---

xsrst input file: `bin/xsrst.py`

child_cmd

4.2 Children Commands

- *Syntax*
- *Purpose*
- *File Names*
- *Links*
- *Example*

Syntax

```
{xsrst_children
    file_1
    ...
    file_n
}

{xsrst_child_link
    file_1
    ...
    file_n
}

{xsrst_child_list
    file_1
    ...
    file_n
}
```

Purpose

A section can specify a set of files for which the *parent section* of each file is a child of the current section. (If there is not parent section in a file, all the sections in the file are children of the current section.) This is done using the commands above at the *beginning of a line*.

File Names

A new line character must precede and follow each of the file names *file_1* ... *file_n*. Leading and trailing white space is not included in the names The file names are relative to the directory where `xsrst.py` is executed; i.e., the top directory for this git repository. This may seem verbose, but it makes it easier to write scripts that move files and automatically change references to them.

Links

The child link and list commands also place links to all the children of the current at the location of the command. The links are displayed using the title for each section. The child list command includes the section name next to the title. You can place a heading directly before the links to make them easier to find.

Example

no_parent_exam

4.2.1 No Parent Example

All the sections in the file `children.py` are children of the section below because `children.py` does not have a *parent section*:

```
{xsrst_begin no_parent_res}

No Parent Result
################

children.py File
****************
{xsrst_file
    # BEGIN_FILE
    # END_FILE
    sphinx/test_in/children.py
}

Child Link children.py
**********************
{xsrst_child_link
        sphinx/test_in/children.py
}

Example
*******
:ref:`no_parent_exam`

{xsrst_end no_parent_res}
```

no_parent_res

4.2.1.1 No Parent Result

- *children.py File*
- *Child Link children.py*
- *Example*

children.py File

```
"""
{xsrst_begin children_exam}

Indent and Children Example
###########################

{xsrst_file
    # BEGIN_SRC
    # END_SRC
}

Result
******
:ref:`children_res`

{xsrst_end children_exam}
"""
# ---------------------------------------------------------------------------
# BEGIN_SRC
"""
{xsrst_begin children_res}

Indent and Children Result
##########################

Children
********
This example uses a children command and
the corresponding link is included.
If it had used a child link command, the same link would have been
created automatically.

{xsrst_children
    sphinx/test_in/indent.py
}

- :ref:`indent_exam`

Example
*******
:ref:`children_exam`

{xsrst_end children_res}
"""
# END_SRC
```

Child Link children.py

children_exam

4.2.1.1.1 Indent and Children Example

- *Result*

```
"""
{xsrst_begin children_res}

Indent and Children Result
##########################

Children
********
This example uses a children command and
the corresponding link is included.
If it had used a child link command, the same link would have been
created automatically.

{xsrst_children
    sphinx/test_in/indent.py
}

- :ref:`indent_exam`

Example
*******
:ref:`children_exam`

{xsrst_end children_res}
"""
```

Result

4.2.1.1.2 Indent and Children Result

---

xsrst input file: `sphinx/test_in/children.py`

children_res

4.2.1.1.2 Indent and Children Result

- *Children*
- *Example*

Children

This example uses a children command and the corresponding link is included. If it had used a child link command, the same link would have been created automatically.

indent_exam

4.2.1.1.2.1 Indent Example

```
"""
    {xsrst_begin indent_res}

    =============
    Indent Result
    =============
    {xsrst_code py}"""
    def factorial(n) :
        if n == 1 :
            return 1
        return n * factorial(n-1)
    """{xsrst_code}

    :ref:`indent_exam`

    {xsrst_end indent_res}
"""
```

indent_res

4.2.1.1.2.1.1 Indent Result

```
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
```

4.2.1.1.2.1 Indent Example

---

xsrst input file: `sphinx/test_in/indent.py`

---

xsrst input file: `sphinx/test_in/indent.py`

- *4.2.1.1.2.1 Indent Example*

Example

4.2.1.1.1 Indent and Children Example

---

xsrst input file: `sphinx/test_in/children.py`

Example

4.2.1 No Parent Example

---

xsrst input file: `sphinx/test_in/no_parent.xsrst`

---

xsrst input file: `sphinx/test_in/no_parent.xsrst`

1. no_parent_exam: *4.2.1 No Parent Example*

---

xsrst input file: `bin/xsrst.py`

spell_cmd

4.3 Spell Command

- *Syntax*
- *Purpose*
- *spelling*
- *Capital Letters*
- *Double Words*
- *Example*

Syntax

`{xsrst_spell` *word_1* ... *word_n* `}`

Here *word_1*, ..., *word_n* is the special list of words for this section. In the syntax above the list of words is all in one line, but they could be on different lines. Each word starts with an upper case letter, a lower case letter, or a back slash. The back slash is included as a possible beginning of a word so that latex commands can be included in the spelling list. The rest of the characters in a word are lower case letters.

Purpose

You can specify a special list of words (not normally considered correct spelling) for the current section using the command above at the *beginning of a line*.

spelling

The list of words in *spelling* are considered correct spellings for all sections. The latex commands corresponding to the letters in the greek alphabet are automatically added to this list.

Capital Letters

The case of the first letter does not matter when checking spelling; e.g., if `abcd` is *word_1* then `Abcd` will be considered a valid word. Each capital letter starts a new word; e.g., *CamelCase* is considered to be the two words 'camel' and 'case'. Single letter words are always correct and not included in the special word list; e.g., the word list entry `CppAD` is the same as `Cpp`.

Double Words

It is considered an error to have only white space between two occurrences of the same word. You can make an exception for this by entering the same word twice (next to each other) in the special word list.

Example

spell_exam

4.3.1 Spell Example

```
"""
{xsrst_begin spell_res}
{xsrst_spell
    iterable
    \cos
    \sin
    no no
}


Spell Result
############

Text
****
The words ``iterable`` and ``xsrst`` are not the dictionary,
so we have included them in the spelling command for this section.


Math
****
The latex commands for greek letters
```

(continues on next page)

(continued from previous page)

```
are automatically included as correct spelling.
The ``\rm`` command is included by the
:ref:`spelling<xsrst_py.command_line_arguments.spelling>`.
The other latex commands in this section, ``\cos`` and ``\sin``,
have been included in the spelling command for this section.
An alternative would be to add them to the *spelling* file
which applies to all sections:

.. math::

    z = \cos( \theta ) + {\rm i} \sin( \theta )

Double Words
************
It is consider an error to have only white space between
two occurrences of the same word; e.g.,
no no would be an error if there
were not two occurrences of :code:`no` next to each other in the
spelling command for this section.

:ref:`spell_exam`

{xsrst_end spell_res}
"""
```

spell_res

4.3.1.1 Spell Result

- *Text*
- *Math*
- *Double Words*

Text

The words `iterable` and `xsrst` are not the dictionary, so we have included them in the spelling command for this section.

Math

The latex commands for greek letters are automatically included as correct spelling. The `\rm` command is included by the *spelling*. The other latex commands in this section, `\cos` and `\sin`, have been included in the spelling command for this section. An alternative would be to add them to the *spelling* file which applies to all sections:

$$z = \cos(\theta) + \mathrm{i} \sin(\theta)$$

Double Words

It is consider an error to have only white space between two occurrences of the same word; e.g., no no would be an error if there were not two occurrences of `no` next to each other in the spelling command for this section.

4.3.1 Spell Example

---

xsrst input file: `sphinx/test_in/spell.py`

---

xsrst input file: `sphinx/test_in/spell.py`

---

xsrst input file: `bin/xsrst.py`

suspend_cmd

4.4 Suspend and Resume Commands

- *Syntax*
- *Purpose*
- *Example*

Syntax

- `{xsrst_suspend}`
- `{xsrst_resume}`

Purpose

It is possible to suspend (resume) the xsrst extraction during a section. One begins (ends) the suspension with a suspend command (resume command) at the *beginning of a line*. Note that this will also suspend all other xsrst processing; e.g., spell checking.

Example

suspend_exam

4.4.1 Suspend Example

```
"""
{xsrst_begin suspend_res}
{xsrst_spell
    iterable
}
```

(continues on next page)

(continued from previous page)

```
Suspend Result
##############

Factorial
*********
*f* = ``factorial(`` *positive_integer* ``)``
{xsrst_suspend}
"""
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
"""
{xsrst_resume}

Product
*******
*p* = ``product(`` *iterable* ``)``
{xsrst_suspend}
"""
def product(itr) :
    p = 1.0
    for v in itr :
        p *= v
    return p
"""
{xsrst_resume}

:ref:`suspend_exam`

{xsrst_end suspend_res}
"""
```

suspend_res

4.4.1.1 Suspend Result

- *Factorial*
- *Product*

Factorial

f = `factorial` (*positive_integer*)

Product

p = `product` (*iterable*)

4.4.1 Suspend Example

---

xsrst input file: `sphinx/test_in/suspend.py`

---

xsrst input file: `sphinx/test_in/suspend.py`

---

xsrst input file: `bin/xsrst.py`

code_cmd

4.5 Code Command

- *Syntax*
- *Purpose*
- *Requirements*
- *language*
- *Rest of Line*
- *Spell Checking*
- *Example*

Syntax

- `{xsrst_code` *language* `}`
- `{xsrst_code}`

Purpose

A code block, directly below in the current input file, begins with a line containing the first version (*language* included version) of the command above.

Requirements

Each code command ends with a line containing the second version of the command; i.e., `{xsrst_code}`. Hence there must be an even number of code commands. The back quote character ` can't be in the same line as the commands.

language

A *language* is a non-empty sequence of non-space the characters. It is used to determine the source code language for highlighting the code block.

Rest of Line

Other characters on the same line as a code command are not included in the xsrst output. This enables one to begin or end a comment block without having the comment characters in the xsrst output.

Spell Checking

Code blocks as usually small and spell checking is done for these code blocks. (Spell checking is not done for code blocks included using the *file command* .)

Example

code_exam

4.5.1 Code Example

```
"""
{xsrst_begin code_res}

Code Result
###########
{xsrst_code py}"""
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
"""{xsrst_code}

:ref:`code_exam`

{xsrst_end code_res}
"""
```

code_res

4.5.1.1 Code Result

```
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
```

4.5.1 Code Example

---

xsrst input file: `sphinx/test_in/code.py`

---

xsrst input file: `sphinx/test_in/code.py`

---

xsrst input file: `bin/xsrst.py`

file_cmd

4.6 File Command

- *Syntax*
- *Purpose*
- *White Space*
- *file_name*
- *start*
- *stop*
- *Spell Checking*
- *Example*

Syntax

`{xsrst_file`
    start
    stop
`}`

`{xsrst_file`
    start
    stop
    file_name
`}`

Purpose

A code block, from any where in any file, is included by the command above at the *beginning of a line*.

White Space

Leading and trailing white space is not included in *start*, *stop* or *file_name*. The new line character separates these tokens.

file_name

If *file_name* is not in the syntax, the code block is in the current input file. Otherwise, the code block is in *file_name*. This file name is relative to the directory where `xsrst.py` is executed; i.e., the top directory for this git repository. This may seem verbose, but it makes it easier to write scripts that move files and automatically change references to them.

start

The code block starts with the line following the occurence of the text *start* in *file_name*. If this is the same as the file containing the command, the text *start* will not match any text in the command. There must be one and only one occurence of *start* in *file_name*, not counting the command itself when the files are the same.

stop

The code block ends with the line before the occurence of the text *start* in *file_name*. If this is the same as the file containing the command, the text *stop* will not match any text in the command. There must be one and only one occurence of *stop* in *file_name*, not counting the command itself when the files are the same.

Spell Checking

Spell checking is **not** done for these code blocks.

Example

file_exam

4.6.1 File Example

```
// BEGIN_FACTORIAL
template<class T> factorial(const T& n)
{   if n == static_cast<T>(1)
        return n;
    return n * factorial(n - 1);
}
// END_FACTORIAL
//
// BEGIN_SQUARE
```

(continues on next page)

(continued from previous page)

```
template<class T> square(const T& x)
// END_SQUARE
{   return x * x;
}
/*
-----------------------------------------------------------------------------
{xsrst_begin file_res}

File Result
###########

factorial
*********
{xsrst_file
    // BEGIN_FACTORIAL
    // END_FACTORIAL
}

square
******
{xsrst_file
    // BEGIN_SQUARE
    // END_SQUARE
}

{xsrst_end file_res}
-----------------------------------------------------------------------------
*/
```

file_res

4.6.1.1 File Result

- *factorial*
- *square*

factorial

```
template<class T> factorial(const T& n)
{   if n == static_cast<T>(1)
        return n;
    return n * factorial(n - 1);
}
```

square

```
template<class T> square(const T& x)
```

---

xsrst input file: `sphinx/test_in/file.cpp`

---

xsrst input file: `sphinx/test_in/file.cpp`

---

xsrst input file: `bin/xsrst.py`

comment_ch_cmd

4.7 Comment Character Command

- *Syntax*
- ***Purpose***
  - *ch*
- *Beginning of a Line*
- *Input Stream*
- *Example*

Syntax

`{xsrst_comment_ch` *ch* `}`

Purpose

Some languages have a special character that indicates the rest of the line is a comment. If you embed sphinx documentation in this type of comment, you need to inform xsrst of the special character so it does not end up in your `.rst` output file.

ch

The value of *ch* must be one non white space character. There must be at least one white space character between `xsrst_comment_ch` and *ch*. Leading and trailing white space around *ch* is ignored. There can be only one occurence of this command within a file, it's effect lasts for the entire file, and it must come before the first *4.1 Begin and End Commands* in the file.

Beginning of a Line

A sequence of characters *text* is at the beginning of a line if there are only space characters between the previous new line character and *text*. In addition, the special character *ch* can be the first character after the new line and before *text*.

Input Stream

The special character (and one space if present directly after) is removed from the input stream before any xsrst processing; e.g., calculating the amount of *Indentation* for the current section. For example, if # is the special character, the following input has the heading Factorial and the `def` token indented the same amount:

```
# Factorial
# ---------
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
```

Example

comment_ch_exam

4.7.1 Comment Character Example

```
# {xsrst_begin comment_ch_res}
#
# ========================
# Comment Character Result
# ========================
# In the original source, the leading characters '#' and ' ' get removed
# and the remaining text lines up with the ``def`` below:
#
# {xsrst_code py}
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
# {xsrst_code}
#
# :ref:`comment_ch_exam`
#
# {xsrst_end comment_ch_res}
```

comment_ch_res

4.7.1.1 Comment Character Result

In the original source, the leading characters '#' and ' ' get removed and the remaining text lines up with the `def` below:

```
def factorial(n) :
    if n == 1 :
        return 1
    return n * factorial(n-1)
```

4.7.1 Comment Character Example

---

xsrst input file: `sphinx/test_in/comment_ch.py`

---

xsrst input file: `sphinx/test_in/comment_ch.py`

---

xsrst input file: `bin/xsrst.py`

heading_exam

4.8 Heading Example

- *Child Sections*

```
"""
{xsrst_begin heading_res}

Heading Result
##############
The label for this heading is the section name ``heading_res``.

Second Level
************
The label for this heading is ``heading_res.second_level``.

Third Level
===========
The label for this heading is ``heading_res.second_level.third_level``.

Another Second Level
********************
The label for this heading is ``heading_res.another_second_level``.

Third Level
===========
The label for this heading is
``heading_res.another_second_level.third_level``.
```

(continues on next page)

(continued from previous page)

```
Links
*****
These links would also work from any other section because the
:ref:`section_name<begin_cmd.section_name>`
(which is ``heading_res`` in this case)
is included at the beginning of the target for the link:

1. :ref:`heading_res`
2. :ref:`heading_res.second_level`
3. :ref:`heading_res.second_level.third_level`
4. :ref:`heading_res.another_second_level`
5. :ref:`heading_res.another_second_level.third_level`

:ref:`heading_exam`

{xsrst_end heading_res}
"""
```

Child Sections

The heading above (Child Sections) is an example heading for the *children* of a *parent section*.

heading_res

4.8.1 Heading Result

- ***Second Level***
  - *Third Level*
- ***Another Second Level***
  - *Third Level*
- *Links*

The label for this heading is the section name `heading_res`.

Second Level

The label for this heading is `heading_res.second_level`.

Third Level

The label for this heading is `heading_res.second_level.third_level`.

Another Second Level

The label for this heading is `heading_res.another_second_level`.

Third Level

The label for this heading is `heading_res.another_second_level.third_level`.

Links

These links would also work from any other section because the *section_name* (which is `heading_res` in this case) is included at the beginning of the target for the link:

1. *4.8.1 Heading Result*
2. *Second Level*
3. *Third Level*
4. *Another Second Level*
5. *Third Level*

4.8 Heading Example

---

xsrst input file: `sphinx/test_in/heading.py`

---

xsrst input file: `sphinx/test_in/heading.py`

configure

4.9 Example xsrst Configuration Files

conf_py

4.9.1 Example conf.py

```
# Configuration file for the Sphinx documentation builder.
#
# For conf.py documentation see
# http://www.sphinx-doc.org/en/master/config
#
# -- Path setup --------------------------------------------------------------
import sphinx_rtd_theme

# -- Project information -----------------------------------------------------
project   = 'cppad_py'
copyright = '2020'
author    = 'Brad Bell'
```

(continues on next page)

(continued from previous page)

```
# -- General configuration ------------------------------------------------
extensions = [
    'sphinx.ext.mathjax',
    'sphinx_rtd_theme',
]
exclude_patterns = [
    '_build', 'Thumbs.db', '.DS_Store', 'test_out', 'preamble.rst'
]

# -- Options for HTML output ----------------------------------------------
html_theme = 'sphinx_rtd_theme'
html_theme_options = {
    'navigation_depth' : -1   ,
    'titles_only'      : True ,
}

# -- These folders are copied to the documentation's HTML output ------------
# html_static_path = [ '_static' ]

# -- These paths are either relative to html_static_path --------------------
# or fully qualified paths (eg. https://...)
html_css_files = [
    'css/custom.css',
]

# -- Latex commands used by all sections ------------------------------------
latex_elements = {
    'preamble' :
        r'\newcommand{\B}[1]{{\bf #1}} '  + '\n' +
        r'\newcommand{\R}[1]{{\rm #1}} '  + '\n' +
        r'\renewcommand{\thesection}{{\hspace{-1em}}}'        + '\n' +
        r'\renewcommand{\thesubsection}{{\hspace{-1em}}}'     + '\n' +
        r'\renewcommand{\thesubsubsection}{{\hspace{-1em}}}'  + '\n' +
        ''
    ,
}
```

xsrst input file: `sphinx/conf.py`

index_rst

4.9.2 Example index.rst

```
.. toctree::
    xsrst/xsrst_automatic
    xsrst/cppad_py
```

xsrst input file: `sphinx/index.rst`

spelling

4.9.3 Example Spelling File

```
# standard mathematical abbreviations
grad
hess
sinh
#
# example words special for this projext
ctor
jac
nc
nr
op
jac
rc
var
vec
xam
#
# example words in xsrst commands
xsrst
rst
#
# example words in sphinx commands
http
https
www
html
github
csv
#
# example python commands
dict
dtype
len
numpy
py
scipy
str
sys
#
# example c++ commands
extern
const
cpp
std
#
# example latex commands
\bar
\begin
\bf
\cdot
\circ
\dot
\ell
```

(continues on next page)

(continued from previous page)

```
\end
\exp
\frac
\geq
\int
\in
\ldots
\leq
\log
\left
\mbox
\newcommand
\partial
\prod
\right
\rightarrow
\rm
\sqrt
\sum
\times
\varepsilon
```

---

xsrst input file: `sphinx/spelling`

keyword

4.9.4 Example Keyword File

```
# List of patterns to exclude from the index
a
and
of
# exclude whats new mm-dd headings used by whats new sections for each year
[0-9][0-9]-[0-9][0-9]
```

---

xsrst input file: `sphinx/keyword`

---

xsrst input file: `sphinx/configure.xsrst`

Syntax

- `xsrst.py` *sphinx_dir root_file spelling keyword*
- `xsrst.py` *sphinx_dir root_file spelling keyword line_increment*

Purpose

This is a pseudo sphinx extension that provides the following features:

1. The file name for each section is also an abbreviated title used in the navigation bar and for linking to the section. This makes the navigation bar more useful while also having long descriptive titles. It also makes cross reference linking from other sections easier.
2. Enables documentation in the comments for source code even when multiple computer languages are used for one package. Allows the documentation for one section to span multiple locations in the source code; see *suspend command*.
3. Allows for multiple sections (rst output files) to be specified by one input file. In addition, one section can be the parent for the other sections in a file.
4. Generates the table of contents from the specification of which files are included; see *child commands*. Generates a jump table to the headings for each section so that the navigation bar need not include this information.
5. Includes a configurable *spell checker* and index. Words in each heading are automatically included in the index.
6. Makes it easy to include source code, that also executes, from directly below the *code command* or from a different location in a *file*. Uses tokens in the source, not line numbers in the source, to signify start and stop of inclusion from a file.

Requirements

- `pip install --user pyspellchecker`
- `pip install --user sphinx`
- The directory *cppad_prefix* `/bin` must be in your execution path where *cppad_prefix* is the install prefix for cppad.

Notation

White Space

We define white space to be a sequence of space characters; e.g., tabs are not consider white space by xsrst.

Beginning of a Line

We say that a string *text* is a the beginning of a line if only white space, or nothing, comes before *text* in the line.

Command Line Arguments

root_file

The command line argument *root_file* is the name of a file, relative to the top git repository directory.

root_section

If there is only one section in the *root_file* it is called the *root_section*; i.e., it is the top section in that table of contents. If there is more than one section in the *root_file*, the file must have a *Parent Section* and it is the *root_section*. The file *sphinx_dir* `/index.rst` must contain the line

`xsrst/` *section_name*

where *section_name* is the name of the *root_section*.

sphinx_dir

The command line argument *sphinx_dir* is a sub-directory, of the top git repository directory. The sphinx `conf.py`, `index.rst`, *spelling*, and *keyword* files are located in this directory. Any files that have names ending in `.rst`, and that are in the directory *sphinx_dir* `/xsrst`, are removed at the beginning of execution of `xsrst.py`. All the `.rst` files in *sphinx_dir* `/xsrst` were extracted from the source code the last time that `xsrst.py` was executed.

Example Configuration Files

conf.py: *4.9.1 Example conf.py*
index.rst: *4.9.2 Example index.rst*
keyword: *4.9.4 Example Keyword File*
spelling: *4.9.3 Example Spelling File*

spelling

The command line argument *spelling* is the name of a file, relative to the *sphinx_dir* directory. This file contains a list of words that the spell checker will consider correct for all sections (it can be an empty file). A line that begins with # is a comment (not included in the list). The words are one per line and leading and trailing white space in a word are ignored. Special words, for a particular section, are specified using the *spell command*.

keyword

The command line argument *keyword* is the name of a file, relative to the *sphinx_dir* directory. This file contains a list of python regular expressions for heading tokens that should not be included in the index (it can be an empty file). A heading token is any sequence of non space or new line characters with upper case letters converted to lower case. For example, a heading might contain the token `The` but you might not want to include `the` as a entry in the genindex. In this case you could have a line containing just `the` in *keyword*. For another example, you might want to exclude all tokens that are numbers. In this case you could have a line containing just `[0-9]*` in *keyword*. The regular expressions are one per line and leading and trailing spaces are ignored. A line that begins with # is a comment (not included in the list of python regular expressions).

line_increment

This optional argument helps find the source of errors reported by sphinx. If the argument *line_increment* is present, a table is generated at the end of each output file. This table maps line numbers in the output file to line numbers in the corresponding xsrst input file. The argument *line_increment* is a positive integer specifying the minimum difference between xsrst input line numbers for entries in the table. The value `1` will give the maximum resolution. For example, the sphinx warning

```
... /xsrst/children_exam.rst:30:  WARNING:...
```

corresponds to line number 30 in the file `children_exam.rst`. The table at the bottom of that file maps line numbers in `children_exam.rst` to line numbers in the corresponding xsrst input file.

Table Of Contents

toctree

The sphinx `toctree` directives are automatically generated for sections. The only such directive you should directly edit is in the file *sphinx_dir* `/index.rst`. One entry in this file specifies the *root_section*. Other entries are for `.rst` files that are not extracted by `xsrst.py`.

Parent Section

A single input file may contain multiple *sections*. One (and at most one) of these sections may use begin with a *parent begin* command. In this case, the other sections in the file are children of this section and this section is a child of the section containing the *child command* that included this file.

If there is no parent section for a file, all the sections in the file are children of the section containing the child command that included the file.

Heading Links

- For each word in a heading, a link is included in the index from the word to the heading.
- Each word in a heading is added to the html keyword meta data.
- A cross reference label is defined for linking from anywhere to a heading. The details of how to use these labels are described below.
- Headings can also be used to help find links to children of the current section; see the heading *Children* below.

Section Level

Each *section* can have only one header at the first level which is a title for the section. The *section_name* is automatically used as a label for linking the title for a section; i.e., the following will link to the title for *section_name*:

`:ref:` ` *linking_text* < *section_name* > `

where *linking_text* is the text the user sees.

Other Levels

The label for linking a heading that is not at the first level is the label for the heading directly above it plus a dot character `.`, plus a lower case version of the heading with spaces converted to underbars _. For example, the label for the heading for this paragraph is

`xsrst_py.headers_and_links.other_levels.`

This may seem verbose, but it helps keep the links up to date. If a heading changes, all the links to that heading will break. This identifies the links that should be checked to make sure they are still valid.

Children

If a xsrst input file has a *parent section* the other sections in the file are children of the parent.

- If a section has a *child link or list command* links to all the children of the section are placed where the child link command is located.
- If a section has a *children command* no automatic links to the children of the current section are generated.
- Otherwise, the links to the children of a section are placed at the end of the section.

You can place a heading directly before the links to make them easier to find.

Example

4.8 Heading Example

Indentation

If there are a number of spaces before all of the xsrst documentation for a section, those characters are not included in the xsrst output. This enables one to indent the xsrst so it is grouped with the proper code block in the source. An error message will result if you use tabs in the indentation.

Example

- *4.2.1.1.2.1 Indent Example*

Wish List

The following is a wish list for future improvements to `xsrst.py`:

Tabs

Tabs in a code blocks get expanded to 8 spaces; see stackoverflow. It would be nice to have a way to control the size of tabs in the code blocks displayed by *4.5 Code Command* and *4.6 File Command*. Perhaps it would be good to support tabs as a method for indenting xsrst input sections.

Module

Convert the program into a python module and provide a pip distribution for it. It would at least be nice for cppad_py to install the `xsrst.py` program so that users would not have to copy it to a directory in their execution path.

Commands

- *4.1 Begin and End Commands*
- *4.2 Children Commands*
- *4.3 Spell Command*
- *4.4 Suspend and Resume Commands*
- *4.5 Code Command*
- *4.6 File Command*
- *4.7 Comment Character Command*

---

xsrst input file: `bin/xsrst.py`

whats_new_2020

5 CppAD Py Changes During 2020

- *Previous Years*
- *09-13*
- *09-12*
- *07-31*
- *07-30*
- *07-29*
- *07-28*
- *07-25*
- *07-24*
- *07-18*

- *07-05*
- *05-17*
- *05-16*
- *05-15*
- *05-14*
- *05-12*
- *05-09*
- *05-08*
- *05-07*
- *05-06*
- *05-05*
- *05-04*
- *04-28*
- *04-27*
- *04-26*
- *04-25*
- *04-24*
- *04-23*
- *04-22*
- *04-20*
- *04-19*
- *04-18*
- *04-13*
- *04-12*
- *04-10*

whats_new_2018

5.1 Cppad Py Changes During 2018

- *11-10*
- *11-09*
- *11-07*
- *11-05*
- *08-13*
- *07-31*
- *07-26*

- *07-19*
- *07-15*
- *07-14*
- *07-13*
- *07-12*
- *07-10*
- *07-08*
- *07-07*
- *07-03*

11-10

1. Added the *dynamic* argument to the `independent` function and the *f_new_dynamic* function.
2. Added the sections *more_cpp* and *more_py*.

11-09

Include binary operations where first operand is an `a_double` and second operand is a `double` ; see *a_double_binary*, *a_double_compare*, and *a_double_assign*.

11-07

1. Advance to cppad-20181106 (must re-run `bin/get_cppad.sh`).
2. The *a_fun* objects were added. This support all the operations that *d_fun* support all the *py_fun* operations with the exception of *optimize*. The *py_sparse* operations are not yet supported.

11-05

1. **API Change** : change `cppad_py.a_fun` to `cppad_py.d_fun` because it is a function that evaluates in `double` and plan to use `cppad_py.a_fun` for a function that evaluates in `a_double` .
2. Advance to cppad-20181105 (must re-run `bin/get_cppad.sh`).

08-13

1. Change `cppad_cxx_flags` to *extra_cxx_flags* in the user configuration settings. Also remove `swig_cxx_flags` from these user settings (set automatically).
2. Improve the install instructions; see *setup_py*.

07-31

1. The general purpose Swig example `swig_xam` , which was not specific to cppad_py, has been removed.
2. A change to `setup.py` was making the *get_cppad_sh* script fail. This has been fixed.
3. The python tests are no run in `example/python` (not copied to the build directory).

07-26

Change `cppad_py/python_major_version` to `cppad_py/python_version` because it now contains both major and minor version numbers.

07-19

Add `-std=c++11` to the default compile flags in *setup_py*. Add `-py3` to the `swig` builds when running `setup.py` using Python 3; see swig python 3.

07-15

1. Better error reporting for type and size errors in `numpy.array` arguments to `cppad_py` functions using *numpy2vec*.
2. Finish converting all vectors and matrices in *py_lib* to numpy arrays. Note the *py_utility* routines do the conversion.

07-14

1. Continue conversion of python library to using `numpy.array` ; see *py_fun_forward*, *py_fun_reverse*.
2. Add the *f_size_order()* function.

07-13

Continue conversion of python library to using `numpy.array` ; see *py_fun_jacobian*, *py_fun_hessian*.

07-12

1. Continue conversion of python library to using `numpy.array` ; see *py_independent*, *py_fun_ctor*.
2. Add the file `cppad_py/python_major_version` which contains the major version of python that this build of `cppad_py` is for.
3. Automatically use *build_type* control how setup.py compiles C++ (one used to use flags on setup.py command line).
4. setup.py was using the git repository version of Cppad instead of the one installed in `build/prefix` by *get_cppad_sh*. This has been fixed.
5. Change `size_ind` to *size_domain* and `size_dep` to *size_range*.

07-10

Start conversion of python library from using `vector` to using `numpy.array` ; see *py_independent*.

07-08

1. Advanced to `cppad-20180703` .
2. Fix the following warning: `catching polymorphic type class 'std::runtime_error' by value` .
3. Suppress the following warning when running *setup_py*: `clearing an object of non-trivial type class 'cppad_py::a_double'` ; see swig issue 1259.
4. Move all settings to *setup_py* and run `cmake` from there.
5. The file `cppad_py/__init__.py` was not being created. This has been fixed.

07-07

Change build of Python module to use Python 3 (now it can use either 2 or 3).

07-03

Birthday when Cppad Py was first extracted from Cppad Swig.

---

xsrst input file: `whats_new/2018.xsrst`

Previous Years

whats_new_2018

09-13

1. The text `xsrst:` was added to the beginning of each whats new item (see below) that only pertained to the *xsrst.py* program.
2. xsrst: The order of the sections in the pdf file and the table of contents was corrected.
3. xsrst: Put spaces around table of contents levels that have more that one entry. Also remove the level of indentation in table of contents by one because the root section stands out has having not number in front of it.
4. The latex numbering of sections is incorrect, so suppress it in the pdf version of the output.

09-12

1. xsrst: A table of contents that only contains section titles, not headings within a section, was added.
2. xsrst: The following restriction was added: a *section_name* can not begin with `xsrst_`.

07-31

1. xsrst: An empty line at beginning of xsrst input file was causing it to crash (this has been fixed).
2. xsrst: Putting version number in the documentation navigation side bar changes every gh-pages file when version changes. Move it to the first heading under the title *cppad_py*
3. xsrst: spell checking is not done for a *url* of the form ` < *url* > `_ or ` *text* < *url* > `_.

07-30

xsrst: The words 'anl', 'dir', 'mcs' were removed from the xsrst dictionary. The word 'initialization' was added to the xsrst dictionary.

07-29

1. xsrst: Add the optional command line argument *line_increment*. This is an aid for finding the source of errors and warnings reported by sphinx.
2. xsrst: Sometimes xsrst would not recognize a command that came directly after a Fix *4.6 File Command*. This has been fixed.

07-28

1. xsrst: Move version number from the title in *cppad_py* to the documentation navigation side bar.
2. xsrst: Add the *4.9 Example xsrst Configuration Files* section.
3. xsrst: The *xsrst begin command* was not recognized at the beginning of a file (when there is no new line before it). This has been fixed.
4. xsrst: Change the *index_file* to *keyword* and *spell_file* to *spelling*

07-25

1. xsrst: Install *xsrst.py* during the *install* procedure (not yet available from `pip`).
2. xsrst: Add a *purpose* paragraph to xsrst documentation.
3. Move the `*.omh` files to `*.xsrst` files (they have been converted from omhelp input files to xsrst input files).
4. xsrst: Change the *keyword* to use python regular expressions and remove the whats new month-day headings from the index.

07-24

1. xsrst: Add the *xsrst.py* program which extracts sphinx `*.rst` files from source code. This program is not yet installed but you can use it by placing the script `bin/xsrst.py` in your execution path.
2. Convert the cppad_py documentation to xsrst input files, which are mostly sphinx rst with a few extra xsrst commands.
3. Use xsrst and sphinx to generate this documentation for cppad_py.

07-18

1. The experimental `bin/xsrst.py` program was added. This program runs its tests and builds its documentation (in the `sphinx`) directory with the command `bin/check_xsrst.sh`. The intention is to convert the cppad_py documentation from omhelp to sphinx.
2. Convert all the cppad_py source code to use spaces instead of tabs (with tab stops at multiples of 4 spaces).

07-05

1. The *prototype* for `vec2a_double` was the same as for *vec2cppad_double*. This has been fixed.
2. The limits for the *fun_from_json_xam_py* and *fun_to_json_xam_py* example code were incorrect (this has been fixed).
3. The in *fun_from_json_xam_cpp* the title was corrected; to be specific, to_json was changed to from_json

05-17

The `abs` *a_double_unary_fun* was added (works the same as the `fabs` function).

05-16

1. A discussion of some subtle issues, when interpolation is used to define an ODE, was added to the `ode_multi_step` ; see *set_t_all_index*.
2. Add the *alpha* parameter to the SEIRWD model.

05-15

The file `ode_solve.py` was moved to `ode_multi_step.py` and *numeric_ode_multi_step* was extended to allow for different ODE solvers (steppers).

05-14

1. The example semi-stiff integrator *numeric_rosen3_step* was added.
2. The example file `runge4.py` was moved to `ode_solve.py` in preparation for other solvers.
3. Add the *numeric_simple_inv* example routine for AD inversion of matrices.
4. Add mention of which *numeric_xam* routines can take `a_double` values.
5. Simplify the *numeric_seirwd_model_xam_py* example using numpy vector operations.
6. More improvements to were made to *covid-19* example.

05-12

1. More improvements to were made to *covid-19* example.
2. The *n_step* option was added to the SEIRWD model example.
3. A delay between when is no longer infectious and when one dies was added to the SEIRWD model using a W compartment (Will die compartment).

05-09

The *covid-19_modeling_example* application of AD is changing on a regular basis, so the details of the changes will no longer be tracked in this file, just see current example if you are interested.

05-08

1. The `numeric_seirs_fit_xam.py` example was renamed *numeric_covid_19_xam_py*. And it was change to fit covariate multipliers that model the infectious rate β.
2. The `numeric_seirs_model` was renamed *numeric_seirwd_model*, the D compartment was added to track total deaths, the data was changed from Infectious to cumulative death, and noise was added to the data.
3. In `numeric_covid_a9_xam` , reduce to just E and I as unknowns in initial conditions. Initial R = D = 0, S = 1 - E - R.
4. Use the observed information matrix to estimate the covariance of the optimal parameters.

05-07

1. The function `scipy.misc.factorial` has been deprecated so change it to `scipy.special.factorial` .
2. Add the `numeric_seirs_model` utility example which allows rates in the ODE to change with time. This enabled changing the *numeric_covid_19_xam_py* example to estimate a β coefficient that changes with time.
3. The `numeric_seirs_class` example was removed because `numeric_seirwd_model` replaces it with more general capabilities.

05-06

1. Add the following utility examples: *numeric_optimize_fun_class*, *numeric_runge4_step*, *numeric_ode_multi_step*, `numeric_seirs_class` . (The `numeric_seirs_class` was later replaced by *numeric_seirwd_model*.)
2. Add mention of using `DYLD_LIBRARY_PATH` in the *cppad_library_missing* install instructions for the Mac.

05-05

Add the *numeric_runge4_step_xam_py* example.

05-04

Add the *numeric_optimize_fun_xam_py* example.

04-28

The *f_from_json()* function was added. In addition, the *empty_function* constructor was added.

04-27

The `setup.py` program now installs a separate copy of CppAD and provides instructions at the end for modifying your `LD_LIBRARY_PATH` ; see *cppad_library_missing*.

04-26

The `setup.py` program was modified to try to automatically solve the *cppad_library_missing* problem.

04-25

The *f_to_json()* function was added.

04-24

The newer Mac systems seems to require that one use `-stdlib=libc++` compile and link flag. The install has been changed to check for and adapt to this condition. In addition, *setup_py* now runs the `cmake` command; i.e., the user no longer needs to run `cmake` to test the *c++* library.

04-23

Add an *install_error* section to the documentation.

04-22

1. Instructions were added for *installing_using_pip*.
2. The binary operators were extended to include *x* `op` *ay* where: *x* is a double (python `float`), *ay* is an `a_double` , and *op* is `+` , `-` , `*` , `/` , or `**` . Note that this automatically transfers to numpy arrays; e.g *x* `*` *ay* where *x* is a double and *ay* is a numpy array of `a_double` .

04-20

1. Move configuration setting from *setup_py* to *bin/get_cppad_sh*.
2. First version that installs using `pip` . Install instructions for pip will be added soon.

04-19

1. Move the python source that gets distributed from `lib/python` to `lib/python/cppad_py` so that more like a standard python package.
2. Drop support for python2. It is not consistent with python3 in some of the `setup.py` actions.

04-18

1. Change *yq* to *xq* , correct documentation, for *xq* in the *c++* and *python* reverse mode documentation.
2. Remove the `--inplace` option from the *syntax* for building the cppad_py python module.

04-13

1. The *z* = `pow` (*x* , *y*) functions was added; see *a_double_binary*.
2. Add the *var2par* function and improve the notation in the *near_equal* notation.

04-12

1. Add the `erf` function was added to the list of `a_double` unary *fun* that have been implemented.
2. The dynamic parameter argument was missing from the *syntax* for the python version of the `independent` function. This has been fixed.
3. Improve the *test* and *install* discussion in `setup.py` .

04-10

1. Change the documentation display on the web using a more recent version of the documentation program `omhelp-20200130`.
2. Add *caching* to the Cppad install.

---

xsrst input file: `whats_new/2020.xsrst`

1. setup_py: *1 Configure and Build the cppad_py Python Module*
2. numeric_xam: *2 Numerical Examples*
3. library: *3 The Cppad Py Libraries*
4. xsrst_py: *4 Extract Sphinx RST*
5. whats_new_2020: *5 CppAD Py Changes During 2020*

---

xsrst input file: `doc.xsrst`

INDEX

Symbols

__init__.py, 123
2020.9.13, 9

Numbers

2018, 254
2020, 253

A

a_both, 158
a_double, 70, 124, 126–128, 130, 131, 133–140, 142, 143, 146, 147, 161, 216, 217
a_double_assign, 138
a_double_assign_xam_cpp, 139
a_double_assign_xam_py, 140
a_double_binary, 131
a_double_binary_xam_cpp, 133
a_double_binary_xam_py, 134
a_double_compare, 135
a_double_compare_xam_cpp, 136
a_double_compare_xam_py, 137
a_double_cond_assign, 144
a_double_cond_assign_xam_cpp, 146
a_double_cond_assign_xam_py, 147
a_double_ctor, 124
a_double_property, 128
a_double_property_xam_cpp, 130
a_double_property_xam_py, 131
a_double_unary_fun, 141
a_double_unary_fun_xam_cpp, 142
a_double_unary_fun_xam_py, 143
a_double_unary_op, 126
a_double_unary_op_xam_cpp, 127
a_double_unary_op_xam_py, 127
a_fun, 71, 72, 162
a_fun_xam_py, 72
a_other, 125
abort, 70, 160, 161
actual, 54
ad, 18, 67, 77, 80, 83, 86, 90, 92, 124, 125, 128, 131, 133, 134, 141–144, 156, 168, 171, 174, 178, 182, 184
ad:, 78, 81, 84, 88, 169, 172, 176, 180
add, 121, 220
adynamic, 68
af, 72, 163
ainv, 18
alpha, 46
an, 18, 52, 77, 80, 90, 128, 131, 168, 171, 182, 183, 216, 217
another, 244
arguments, 249
array, 118–120
assignment, 138–140, 144
assignment:, 146, 147
ax, 68, 72, 126, 132, 135, 138, 141, 158, 162
ay, 72, 126, 132, 136, 139, 141, 162
az, 132

B

b, 136
begin, 224
begin_cmd, 224
beginning, 242, 249
beta(t), 53
binary, 131, 133, 134
block, 218
bounds, 54
build, 10, 16
build_type, 14

C

c, 97, 190
c++, 9, 10, 16, 124, 216, 220
c++:, 127, 130, 133, 136, 139, 142, 146, 151, 154, 159, 161, 166, 169, 172, 176, 180, 183, 185, 186, 192, 198, 201, 204, 209, 214, 218
caching, 15
calculation, 188
capital, 232
catch, 218
ch, 241
change, 167
changes, 253, 254

character, 241–243
check_rosen3_step, 32
checking, 237, 239
child, 228, 244
child_cmd, 225
children, 10, 18, 66, 67, 95, 118, 124, 149, 156, 188, 216, 225, 229, 252
children.py, 227, 228
children_exam, 229
children_res, 229
class, 38
code, 19, 20, 23, 25, 28, 31, 36, 38, 41, 43, 49, 58, 236–238
code_cmd, 236
code_exam, 237
code_res, 238
coefficient, 55, 83, 174
col, 97, 101, 191, 196
col_major, 98, 102, 192, 197
command, 231, 236, 238, 241, 249
commands, 224, 225, 234, 253
comment, 241–243
comment_ch_cmd, 241
comment_ch_exam, 242
comment_ch_res, 243
comparison, 135–137
compatible, 18
component, 108, 204
computation, 92
computational, 184
computing, 19, 23, 27, 35, 110, 112, 114, 206, 209, 211
conditional, 144, 146, 147
conf.py, 245
conf_py, 245
configuration, 245, 250
configure, 10, 16, 245
conservation, 46
constraint_ad, 39
constraint_fun, 40
constraint_hess, 40
constraint_jac, 40
constructor, 124
constructors, 149
contents, 251
context, 119
convert, 118, 120, 216, 217
cop, 145
covariates, 53
covid-19, 52
cpp_abort_recording, 160
cpp_fun, 156
cpp_fun_ctor, 162
cpp_fun_forward, 174
cpp_fun_hessian, 171
cpp_fun_jacobian, 168
cpp_fun_json, 184
cpp_fun_new_dynamic, 167
cpp_fun_optimize, 182
cpp_fun_property, 164
cpp_fun_reverse, 178
cpp_independent, 157
cpp_jac_sparsity, 199
cpp_lib, 124
cpp_sparse_hes, 211
cpp_sparse_jac, 206
cpp_sparse_rc, 188
cpp_sparse_rcv, 193
cpp_sparsity, 202
cpp_utility, 216
cppad, 9, 12, 14, 16, 66, 67, 124, 149, 156, 188, 218, 219, 253, 254
cppad::ad<double>, 216, 217
cppad_prefix, 14
cppad_py, 9, 10, 16, 118, 120
current, 71, 162

D

d, 125
d_fun, 71, 75, 162
d_fun:, 90, 183
data, 55
data_file, 55
death, 44
debug, 57
declare, 67, 157
defines, 38
dense, 78, 81, 169, 172
derivative, 23, 27, 28, 35, 36
derivatives, 19
directory, 14, 16
display, 57
documentation, 121, 221
double, 232, 233
download, 15
dtype, 119
during, 11, 253, 254
dynamic, 68, 77, 157, 159, 167, 168

E

element_type, 153
elements, 153
elements:, 154, 155
empty, 72, 163
end, 224
entire, 105, 200
error, 11
error_message, 217

error_message_xam_cpp, 218
error_message_xam_py, 219
errors, 15, 218
example, 19, 20, 23, 25, 27, 31, 35, 37, 41, 43, 47, 48, 52, 68–73, 75–79, 81, 82, 84, 86, 88–93, 98, 99, 102, 103, 105, 106, 108, 109, 112, 113, 116, 118, 122, 123, 126, 127, 130, 131, 133–140, 142, 143, 146–148, 150–152, 154–156, 159–161, 163, 166–170, 172, 174, 176, 177, 180, 182–186, 192, 193, 198, 201, 202, 204, 206, 209, 210, 214, 215, 218, 219, 221, 222, 226, 227, 229–232, 234, 237, 239, 242, 243, 245–248, 250, 252
examples, 9, 18, 94, 187
exception, 217–219
exposed, 44
external, 11
extra_cxx_flags, 14
extract, 223

F

f, 72, 74, 77, 78, 80, 83, 86, 90, 92, 105, 107, 111, 114, 163, 164, 168, 169, 171, 174, 178, 182, 184, 200, 203, 207, 211
f(x), 78, 80, 83, 87, 169, 171, 175, 178
factorial, 235, 240
file, 220, 225–227, 238–240, 247, 248
file_cmd, 238
file_exam, 239
file_name, 239
file_res, 240
files, 245, 250
first, 28
fit, 57
fitting, 52
for, 38, 52, 105, 108, 200, 204
for_jac_sparsity, 104, 199
fortify, 12
forward, 83, 84, 112, 116, 174, 176, 208, 213
fourth, 21
from_json, 92, 185
from_json:, 93, 186
fun, 22, 26, 34, 141
fun.f, 26
fun.f(t, 35
fun.f_t, 26
fun.f_y, 26
fun.set_t_all_index(index), 34
fun_abort_xam_cpp, 161
fun_abort_xam_py, 70
fun_dynamic_xam_cpp, 159
fun_dynamic_xam_py, 68
fun_forward_xam_cpp, 176
fun_forward_xam_py, 84
fun_from_json_xam_cpp, 186
fun_from_json_xam_py, 93
fun_hessian_xam_cpp, 172
fun_hessian_xam_py, 81
fun_jacobian_xam_cpp, 169
fun_jacobian_xam_py, 78
fun_new_dynamic.py, 123
fun_optimize_xam_cpp, 183
fun_optimize_xam_py, 90
fun_property_xam_cpp, 166
fun_property_xam_py, 75
fun_reverse_xam_cpp, 180
fun_reverse_xam_py, 88
fun_to_json_xam_cpp, 185
fun_to_json_xam_py, 92
function, 10, 71, 72, 74, 77, 80, 90, 162–164, 166, 168, 171, 182
functions, 38, 67, 121, 141–143, 156, 220

G

g(t), 87, 179
g(x), 80, 171
get, 14, 16
get_cppad_sh, 14
getting, 9, 153–155
git, 9
graph, 92, 184

H

h, 22, 27, 81, 172
handling, 217
handling:, 218, 219
heading, 243, 244, 251
heading_exam, 243
heading_res, 244
helper, 38
hessian, 80, 81, 107, 108, 116, 171, 172, 202, 204, 214
hessians, 114, 211

I

i, 153
if_false, 146
if_true, 146
implementation, 122, 221
import, 17
in, 218
include, 220
indent, 229, 230
indent_exam, 230
indent_res, 230
indentation, 252
independent, 67, 121, 122, 157, 220, 221
index.rst, 246
index_rst, 246

infectious, 44
initial, 46, 53
input, 242
install, 11, 15, 17
install_error, 11
installation, 11
interface, 9
inverse, 18
inversion, 19

J

j, 78, 169
jacobian, 77, 78, 104, 105, 168, 169, 199–201
jacobians, 110, 206
jacobians:, 112, 209
json, 92, 184

K

k, 97, 101, 190, 195
keyword, 248, 250

L

language, 237
left, 145
letters, 232
level, 244, 245, 251
levels, 252
libraries, 66
library, 9, 12, 66, 67, 124
license, 10
likelihood, 54
line, 237, 242, 249
line_increment, 250
link, 228
links, 226, 245, 251
list, 252

M

mass, 46
math, 233
matrices, 99, 193
matrix, 18, 19, 100, 194
maximum, 54
messages, 11
method, 45
minus, 126
minus:, 127
missing, 12
mode, 83, 84, 86, 88, 174, 176, 178, 180
model, 44, 52–54
module, 10, 253
more, 121, 220
more_cpp, 220
more_py, 121
multiple, 33

N

n, 150, 151
n_step, 46
n_sweep, 111, 115, 208, 213
name, 119
names, 226
nc, 96, 100, 120, 189, 195
near_equal, 129
needed, 38
new, 77
new_dynamic, 121, 123, 220–222
nnz, 96, 101, 189, 195
no, 227
no_parent_exam, 227
no_parent_res, 227
not, 218
notation, 44, 86, 178, 249
nr, 96, 100, 120, 189, 194
numeric_covid_19_xam_py, 52
numeric_ode_multi_step, 33
numeric_ode_multi_step_xam_py, 35
numeric_optimize_fun_class, 38
numeric_optimize_fun_xam_py, 41
numeric_rosen3_step, 25
numeric_rosen3_step_xam_py, 27
numeric_runge4_step, 21
numeric_runge4_step_xam_py, 23
numeric_seirwd_model, 44
numeric_seirwd_model_xam_py, 47
numeric_simple_inv, 18
numeric_simple_inv_xam_py, 19
numeric_xam, 18
numerical, 9, 18
numpy, 118, 120
numpy2vec, 118

O

object, 9, 71, 74, 128, 162, 164
objective_ad, 39
objective_fun, 39
objective_grad, 39
objective_hess, 39
objects:, 72
ode, 21, 23, 25, 27, 28, 33, 35, 45, 53, 54
ok, 27
one, 21, 25
one_step, 34, 35
op, 132, 135, 138
operations:, 70, 161
operators, 131, 133–135, 138
operators:, 136, 137, 139, 140
optimization, 38, 41

optimize, 90, 182, 183
optimize_fun, 39
optimize_fun_class, 41
order, 21, 25
other, 53, 252
output, 57, 225

P

p, 84, 175
p_all, 45
parameter, 129
parameters, 54, 77, 167
parameters:, 68, 159
parent, 225, 227, 251
path, 17
pattern, 96, 100, 111, 115, 189, 194, 208, 212
pattern_in, 105, 200
pattern_out, 105, 108, 200, 204
patterns, 95, 104, 107, 188, 199, 202
patterns:, 98, 102, 105, 108, 116, 192, 198, 201, 204, 214
permissions, 12
pip, 11
plot, 57
plus, 126, 127
previous, 257
printout, 57
problem, 19, 41
product, 236
properties, 74, 128, 164
properties:, 75, 130, 131, 166
prototype, 216, 217
purpose, 9, 18, 22, 26, 34, 39, 44, 68, 70, 72, 90, 104, 107, 110, 114, 121, 125, 144, 150, 157, 160, 182, 199, 203, 206, 211, 220, 226, 231, 234, 236, 238, 241, 248
put, 96, 101, 190, 195
py, 66, 67, 124, 149, 156, 188, 218, 219, 253, 254
py_abort_recording, 70
py_fun, 67
py_fun_ctor, 71
py_fun_forward, 83
py_fun_hessian, 80
py_fun_jacobian, 77
py_fun_json, 92
py_fun_new_dynamic, 77
py_fun_optimize, 90
py_fun_property, 74
py_fun_reverse, 86
py_hes_sparsity, 107
py_independent, 67
py_jac_sparsity, 104
py_lib, 67
py_sparse, 95
py_sparse_hes, 114
py_sparse_jac, 110
py_sparse_rc, 95
py_sparse_rcv, 99
py_utility, 118
python, 9, 10, 16, 17, 67, 92, 93, 95, 118, 121
python:, 68, 70, 72, 75, 78, 81, 84, 88, 90, 98, 102, 105, 108, 112, 116, 127, 131, 134, 137, 140, 143, 147, 152, 155, 219

Q

q, 87, 179

R

r, 97, 115, 190, 212
random, 56
rates, 53
recording, 67, 70, 71, 157, 160–162
recovered, 44
reference, 26, 41
repository, 9
representation, 92, 184
requirements, 11, 236, 249
residuals, 56
resize, 96, 190
rest, 237
restriction, 129
result, 131, 141, 227, 229, 230, 233, 235, 238, 240, 243, 244
result:, 133, 134, 142, 143
results, 57
resume, 234
rev_jac_sparsity, 104, 199
reverse, 86, 88, 178, 180
right, 145
root_file, 249
root_section, 249
rosen3_step, 31
rosenbrock, 25, 27
routine, 18
routines, 95
row, 97, 101, 191, 196
row_major, 97, 101, 191, 196
rst, 223
runge-kutta, 21, 23, 35

S

s, 87, 178
safe, 218
sample_interval, 55
scalars, 124
scipy, 41
second, 28, 244
section, 225, 251

section_name, 225
sections, 244
seed, 56
seirwd, 52
seirwd_all, 46
select_domain, 108, 203
select_range, 108, 203
seris_model, 47
setting, 153–155
settings, 14
setup_py, 10
shape, 119
simulation, 56
size, 151, 152
size_domain, 74, 164
size_op, 75, 165
size_order, 75, 77, 84, 165, 168, 175
size_range, 74, 164
size_var, 74, 165
solution, 23, 27, 28, 35, 36
solved, 11
solver, 54
source, 12, 14, 16, 19, 20, 23, 25, 28, 31, 36, 38, 41, 43, 49, 58
space, 239, 249
sparse, 99, 110, 112, 114, 188, 193, 206, 209, 211
sparse_hes_pattern_xam_cpp, 204
sparse_hes_pattern_xam_py, 108
sparse_hes_xam_cpp, 214
sparse_hes_xam_py, 116
sparse_jac_for, 110, 207
sparse_jac_pattern_xam_cpp, 201
sparse_jac_pattern_xam_py, 105
sparse_jac_rev, 111, 207
sparse_jac_xam_cpp, 209
sparse_jac_xam_py, 112
sparse_rc_xam_cpp, 192
sparse_rc_xam_py, 98
sparse_rcv_xam_cpp, 198
sparse_rcv_xam_py, 102
sparsity, 95, 98, 102, 104, 105, 107, 108, 116, 188, 192, 198–202, 204, 214
speed, 10
spell, 231–233, 237, 239
spell_cmd, 231
spell_exam, 232
spell_res, 233
spelling, 232, 247, 250
sphinx, 223
sphinx_dir, 250
square, 241
start, 56, 67, 157, 239
started, 9
step, 21, 25
steps, 33, 121, 220
stop, 71, 162, 239
store, 71, 162
stream, 242
subset, 111, 115, 207, 212
susceptible, 44
suspend, 234, 235
suspend_cmd, 234
suspend_exam, 234
suspend_res, 235
swig, 12
syntax, 10, 14, 18, 22, 25, 33, 38, 44, 67, 70, 71, 74, 77, 78, 80, 83, 86, 90, 92, 95, 100, 104, 107, 110, 114, 119, 120, 125, 126, 128, 131, 135, 138, 141, 144, 149, 151, 153, 157, 160, 162, 164, 167, 168, 171, 174, 178, 182, 184, 188, 194, 199, 203, 206, 211, 216, 217, 224, 225, 231, 234, 236, 238, 241, 248

T

t, 26, 87, 179
t0, 34
t_all, 35, 45
t_step, 34
table, 251
tabs, 253
target, 144
taylor, 83, 174
test, 16, 68, 70, 72, 75, 78, 81, 84, 88, 90, 92, 93, 98, 102, 105, 108, 112, 116, 127, 130, 131, 133, 134, 136, 137, 139, 140, 142, 143, 146, 147, 151, 152, 154, 155, 159, 161, 166, 169, 172, 176, 180, 183, 185, 186, 192, 198, 201, 204, 209, 214, 218, 219
test_cppad, 15
testing, 124, 223
text, 233
that, 38
the, 10, 66, 67, 124, 167
third, 25, 244, 245
thread, 218
ti, 22, 27
to, 9, 118, 120, 121, 216, 217, 220
to_json, 92, 185
to_json:, 92, 185
toctree, 251
top, 14, 16
travis, 13
trust_constr, 41
try, 218

U

u, 153
unary, 126, 127, 141–143

unknown, 54
unsolved, 13
uses, 112, 116, 208, 213
using, 11, 41, 47, 68, 78, 81, 159, 169, 172
utilities, 118, 216

V

v, 101, 151, 154, 195
val, 101, 196
value, 128
values, 53
var2par, 129
variable, 129
variables, 67, 157
variation, 55
vec, 119, 120
vec2a_double, 217
vec2cppad_double, 216
vec2numpy, 120
vec_a_double, 150
vec_bool, 150
vec_double, 150
vec_int, 150
vector, 118, 120, 149, 151, 153–155, 216, 217
vector_ctor, 149
vector_set_get, 153
vector_set_get_xam_cpp, 154
vector_set_get_xam_py, 155
vector_size, 151
vector_size_xam_cpp, 151
vector_size_xam_py, 152
vectors, 149
vectors:, 151, 152
version, 9

W

w, 81, 172
weighted, 56
whats_new_2018, 254
whats_new_2020, 253
white, 239, 249
wise, 108, 204
wish, 252
with, 41, 131, 133, 134, 141–143
words, 232, 233
work, 111, 115, 208, 213

X

x, 67, 78, 80, 104, 107, 111, 115, 154, 157, 169, 171, 200, 203, 207, 212
x(t), 83, 87, 175, 178
x), 27
xp, 84, 175
xq, 87, 179
xsrst, 245
xsrst_py, 223

Y

y, 26, 132, 136, 139, 154
y(t, 27
y(t), 84, 87, 175, 179
y), 35
y0, 34
y1, 34
y_all, 35
y_init, 35
years, 257
yf, 22, 27
yi, 22, 27
yp, 84, 175
yq, 87, 179